Matemática Financeira — Teoria Completa com Python

Teoria, exemplos resolvidos passo a passo e implementação em Python

Sumário

Matemática Financeira — Teoria Completa com Python 1
Capítulo 0 — Como Usar Esta Apostila 3
0.1. Objetivo e método de estudo 3
0.2. Notação e convenções 3
0.3. Revisão de matemática necessária 4
0.4. Roteiro de cinco passos para resolver qualquer problema 5
0.5. Ferramentas: HP-12C, planilha e Python 5
Capítulo 1 — Fundamentos: O Valor do Dinheiro no Tempo 6
1.1. O Princípio Fundamental 6
1.2. Conceitos Primitivos 7
1.3. Fluxo de Caixa 8
1.4. Regimes de Capitalização 10
Capítulo 2 — Juros Simples 12
2.1. Dedução das Fórmulas 12
2.2. Taxas Proporcionais (Juros Simples) 13
2.3. Juros Exatos vs Juros Comerciais 14
2.4. Descontos Simples 16
2.5. Equivalência entre Taxa de Desconto e Taxa de Juros 17
2.6. Prazo Médio de um Conjunto de Títulos 18
Capítulo 3 — Juros Compostos 20
3.1. Dedução das Fórmulas 20
3.2. A Regra dos 72 21
3.3. Taxas Equivalentes (Juros Compostos) 22
3.4. Taxa Nominal vs Taxa Efetiva 24
3.5. Juros Compostos com Períodos Fracionários 26
Capítulo 4 — Séries de Pagamentos 27
4.1. Classificação das Séries 27
4.2. Série Uniforme Postecipada 27
4.3. Série Uniforme Antecipada 30
4.4. Série Diferida (com Carência) 31
4.5. Perpetuidades 32
4.6. Perpetuidade com Crescimento (Modelo de Gordon) 33
Capítulo 5 — Sistemas de Amortização 34
5.1. Conceitos Gerais 34
5.2. Sistema Price (Sistema Francês) 35
5.3. SAC — Sistema de Amortizações Constantes 37
5.4. Comparação Price vs SAC 39
5.5. SAM — Sistema de Amortização Mista 41
5.6. Sistema Americano 41
5.7. Custo Efetivo Total (CET) 42
Capítulo 6 — Inflação e Correção Monetária 43
6.1. Conceito de Inflação 43
6.2. Taxa Nominal vs Taxa Real (Equação de Fisher) 44
6.3. Correção Monetária 45
6.4. Taxa Pré-fixada vs Pós-fixada 47
Capítulo 7 — Análise de Investimentos 48
7.1. Valor Presente Líquido (VPL / NPV) 48
7.2. Taxa Interna de Retorno (TIR / IRR) 50

7.3. Limitações da TIR 51
7.4. Payback Simples e Descontado 54
7.5. Índice de Lucratividade (IL) 55
7.6. VPLA — Valor Presente Líquido Anualizado 56
7.7. Análise de Sensibilidade 57
Capítulo 8 — Tabela Resumo de Fórmulas 58
Capítulo 9 — Exercícios Resolvidos 59
Capítulo 10 — Exercícios para Executar (na mão) 64
Exercício 1 — Juros simples na mão 64
Exercício 2 — Desconto comercial vs. desconto racional 64
Exercício 3 — Juros compostos mês a mês 64
Exercício 4 — Taxa nominal para taxa efetiva 64
Exercício 5 — Prestação de uma série uniforme 64
Exercício 6 — Decisão entre Price e SAC 64
Exercício 7 — Taxa real pela equação de Fisher 64
Exercício 8 — Interpretando VPL e TIR já calculados 65
Capítulo 11 — Exercícios para Executar (em código) 65
Exercício 1 — Juros simples multiuso 65
Exercício 2 — Ponto de cruzamento entre regimes 65
Exercício 3 — Taxas equivalentes compostas 65
Exercício 4 — Nominal para efetiva com capitalizações diferentes 65
Exercício 5 — Simulação com aportes mensais 65
Exercício 6 — Prestação de série postecipada 66
Exercício 7 — Antecipada vs. postecipada 66
Exercício 8 — Série diferida com carência 66
Exercício 9 — Perpetuidade simples 66
Exercício 10 — Perpetuidade com crescimento (Gordon) 66
Exercício 11 — Tabela Price completa 66
Exercício 12 — Tabela SAC completa 66
Exercício 13 — Comparação Price x SAC 66
Exercício 14 — Sistema de Amortização Americano 66
Exercício 15 — SAM (Sistema de Amortização Mista) 66
Exercício 16 — Custo Efetivo Total (CET) 66
Exercício 17 — Equação de Fisher completa 67
Exercício 18 — Correção monetária de uma série 67
Exercício 19 — Pré-fixado vs. pós-fixado 67
Exercício 20 — VPL de um fluxo de caixa 67
Exercício 21 — TIR implementada do zero 67
Exercício 22 — Payback simples e descontado 67
Exercício 23 — Índice de Lucratividade e WACC 67
Exercício 24 — Projeto integrador: relatório de financiamento 67
Capítulo 12 — Aplicações no Mercado Brasileiro 67
12.1. “Sem juros” é sempre sem juros? 67
12.2. CDB, LCI e imposto de renda regressivo 68
12.3. O custo do rotativo do cartão de crédito 69
12.4. Financiamento imobiliário: Price × SAC em 30 anos 69
12.5. Planejamento de aposentadoria: acumular e depois consumir 70
Capítulo 13 — Erros Comuns e Teste Rápido 70
13.1. Os dez erros que mais reprovam 70
13.2. Teste rápido: verdadeiro ou falso (com gabarito) 71
Capítulo 14 — Glossário 71

Capítulo 0 — Como Usar Esta Apostila

0.1. Objetivo e método de estudo

A matemática financeira responde a uma pergunta só, repetida de muitas formas: **quanto vale, hoje, um valor que será pago ou recebido em outra data?** Com essa pergunta resolvem-se financiamentos, aplicações, descontos de títulos, avaliação de projetos e preços de ativos.

Cada tópico desta apostila é apresentado em três camadas, e o estudo rende mais quando você as percorre **na ordem**:

1. **O porquê (teoria).** De onde vem a fórmula, quais hipóteses ela esconde e quando deixa de valer. Decorar fórmulas sem isso leva a erros silenciosos — aquele tipo de erro em que a conta "dá certo" e o resultado está errado.
2. **O como (exemplos resolvidos à mão).** Cada conceito vem com pelo menos um exemplo numérico resolvido passo a passo, com calculadora. Refaça a conta no papel antes de olhar a resposta.
3. **O automatizar (Python).** O código reproduz o exemplo, generaliza para outros valores e serve de conferência. Todos os resultados impressos nesta apostila foram obtidos executando o código mostrado.

Para sinalizar o tipo de conteúdo, usamos caixas coloridas:

Exemplo resolvido — modelo
Um problema numérico, resolvido em etapas. Tente resolver sozinho antes de ler a solução.

Intuição
A ideia por trás da fórmula, em linguagem comum, sem símbolos.

Aprofundando a teoria — modelo
Demonstrações e detalhes que explicam *por que* a fórmula funciona. Podem ser lidos numa segunda passada.

Atenção — erro comum
Um erro que costuma aparecer em provas, no trabalho e em contratos reais.

No mercado brasileiro
Como o conceito aparece na prática do sistema financeiro e da regulação brasileira.

0.2. Notação e convenções

Símbolo	Significado	Observação
PV ou C	Valor presente (capital)	Valor na data zero
FV ou M	Valor futuro (montante)	Valor na data n
J	Juros	$J = \text{FV} - \text{PV}$
i	Taxa de juros por período	Sempre em **forma decimal** nas fórmulas: 2% = 0,02
n	Número de períodos	Na mesma unidade da taxa
PMT	Prestação (pagamento periódico)	Constante nas séries uniformes
SD_k	Saldo devedor após o pagamento k	Capítulo 5
A_k, J_k	Amortização e juros da prestação k	$\text{PMT}_k = A_k + J_k$
d	Taxa de desconto	Capítulo 2
π, i_r, i_n	Inflação, taxa real, taxa nominal	Capítulo 6

Símbolo	Significado	Observação
TMA, TIR, VPL	Taxa mínima de atratividade, taxa interna de retorno, valor presente líquido	Capítulo 7

Siglas de período: a.d. (ao dia), a.m. (ao mês), a.t. (ao trimestre), a.s. (ao semestre), a.a. (ao ano).

Três regras de higiene numérica.

- **Taxa e prazo na mesma unidade.** Taxa mensal exige n em meses. Esta é a fonte de erro mais frequente de todo o assunto.
- **Não arredonde no meio da conta.** Guarde todas as casas decimais até o resultado final. Arredondar uma taxa de $1{,}8088\%$ para $1{,}8\%$ já desloca o resultado na terceira ou quarta casa do valor final.
- **Separadores.** No texto usamos o padrão brasileiro (R$ 1.550,43). As saídas do Python usam o padrão internacional (1550.43 ou 1,550.43), porque é assim que a linguagem imprime.

0.3. Revisão de matemática necessária

Porcentagem como fator multiplicativo
Aumentar um valor em 8% é multiplicá-lo por $1 + 0{,}08 = 1{,}08$. Reduzi-lo em 8% é multiplicá-lo por $1 - 0{,}08 = 0{,}92$. Variações sucessivas **multiplicam-se** não se somam.

> **Exemplo resolvido — variações sucessivas**
> Um preço sobe 25% e depois cai 20%. Volta ao valor original?
>
> **Solução.** Fator total: $1{,}25 \times 0{,}80 = 1{,}00$. Sim — mas **só por coincidência numérica**: os 20% de queda incidem sobre uma base 25% maior. Já uma alta de 25% seguida de queda de 25% dá $1{,}25 \times 0{,}75 = 0{,}9375$, uma perda líquida de 6,25%.

Potências, raízes e logaritmos
Toda a matemática dos juros compostos usa três operações inversas entre si:

$$x = y^n \quad \Leftrightarrow \quad y = x^{1/n} \quad \Leftrightarrow \quad n = \frac{\ln(x)}{\ln(y)}$$

- **Potência** — calcula o montante: $1{,}02^{12} \approx 1{,}2682$.
- **Raiz** ($x^{1/n}$) — calcula a taxa por período: a taxa mensal que dobra um capital em 12 meses é $2^{1/12} - 1 \approx 5{,}95\%$ a.m.
- **Logaritmo** — calcula o prazo. A propriedade $\ln(a^b) = b \cdot \ln(a)$ "desce o expoente": de $1{,}02^n = 2$ obtém-se $n \cdot \ln(1{,}02) = \ln(2)$, logo $n = \frac{\ln(2)}{\ln(1{,}02)} \approx 35{,}0$ meses.

Progressões aritmética e geométrica
Os juros simples formam uma **progressão aritmética** (PA): somam uma constante a cada período. Os compostos formam uma **progressão geométrica** (PG): multiplicam por uma constante. As séries de pagamentos (Capítulo 4) exigem a **soma de uma PG**, que se demonstra em duas linhas.

> **Aprofundando a teoria — soma dos termos de uma PG**
> Seja uma PG de primeiro termo a_1 e razão $q \neq 1$, com n termos:
>
> $$S = a_1 + a_1 q + a_1 q^2 + \ldots + a_1 q^{n-1}$$
>
> Multiplique tudo por q:
>
> $$qS = a_1 q + a_1 q^2 + \ldots + a_1 q^{n-1} + a_1 q^n$$
>
> Subtraia a segunda linha da primeira. Quase tudo se cancela:
>
> $$S - qS = a_1 - a_1 q^n \quad \Rightarrow \quad S = a_1 \frac{1 - q^n}{1 - q} = a_1 \frac{q^n - 1}{q - 1}$$

Se $|q| < 1$ e n cresce sem limite, $q^n \to 0$ e a soma converge para $S_\infty = \frac{a_1}{1-q}$. Esse resultado é a **perpetuidade** da seção 4.5. Exemplo: $1 + \frac{1}{2} + \frac{1}{4} + \ldots = \frac{1}{1-\frac{1}{2}} = 2$.

0.4. Roteiro de cinco passos para resolver qualquer problema

Quase todo problema de matemática financeira se resolve com a mesma sequência. Adote-a como hábito:

1. **Identifique a incógnita** e os dados: qual dos valores (PV, FV, i, n, PMT) foi pedido e quais foram dados?
2. **Padronize as unidades.** Taxa e prazo precisam estar na mesma base de tempo. Se não estiverem, converta a taxa (equivalência, Capítulo 3) ou o prazo.
3. **Desenhe o fluxo de caixa.** Marque a data de cada valor, com entradas para cima e saídas para baixo. Em 80% dos erros, o desenho teria evitado a confusão.
4. **Escolha o regime e a fórmula** (simples ou composto; pagamento único ou série) e leve todos os valores **para uma mesma data**, geralmente a data zero.
5. **Calcule e teste a razoabilidade.** O montante deve ser maior que o capital; a soma das prestações deve ser maior que o valor financiado; uma taxa de 2% a.m. não pode resultar em 90% a.a.

Exemplo resolvido — o roteiro em ação

Uma loja vende um notebook por R$ 3.000 à vista, ou em 5 prestações mensais iguais, sem entrada, com juros de 2% a.m. Qual é a prestação?

1. **Incógnita:** PMT. **Dados:** PV = 3.000; i = 2% a.m.; n = 5.
2. **Unidades:** taxa mensal, prazo em meses — já compatíveis.
3. **Fluxo de caixa** (visão do comprador):



1. **Fórmula:** série uniforme postecipada (Capítulo 4): $\text{PMT} = \text{PV} \cdot i\frac{(1+i)^n}{(1+i)^n-1}$.
2. **Conta:** $1{,}02^5 = 1{,}10408$; fator $= 0{,}02 \times \frac{1{,}10408}{0{,}10408} = 0{,}21216$; logo $\text{PMT} = 3.000 \times 0{,}21216 \approx$ **R$ 636,48**. **Teste:** $5 \times 636{,}48 = 3.182{,}38 > 3.000$ ✓ (a diferença, R$ 182,38, são os juros).

```
caso_pv, caso_i, caso_n = 3000, 0.02, 5
caso_fator = caso_i * (1 + caso_i) ** caso_n / ((1 + caso_i) ** caso_n - 1)
caso_pmt = caso_pv * caso_fator
print(f"Fator de recuperação de capital: {caso_fator:.5f}")
print(f"Prestação: R${caso_pmt:.2f}")
print(f"Total pago: R${caso_pmt * caso_n:.2f} (juros: R${caso_pmt * caso_n -
caso_pv:.2f})")
```

```
Fator de recuperação de capital: 0.21216
Prestação: R$636.48
Total pago: R$3182.38 (juros: R$182.38)
```

0.5. Ferramentas: HP-12C, planilha e Python

Os mesmos cálculos podem ser feitos em três ambientes. Conhecer a correspondência evita reinventar a roda e permite conferir o resultado por dois caminhos independentes.

Cálculo	HP-12C	Excel / Planilhas (pt-BR)	Python (nesta apostila)
Valor futuro	n, i, PV, PMT ⟶ FV	`=VF(taxa; nper; pgto; vp)`	`juros_compostos`
Valor presente	n, i, PMT, FV ⟶ PV	`=VP(taxa; nper; pgto; vf)`	`juros_compostos`
Prestação	n, i, PV ⟶ PMT	`=PGTO(taxa; nper; vp)`	`serie_postecipada`
Número de períodos	i, PV, PMT, FV ⟶ n	`=NPER(taxa; pgto; vp; vf)`	`juros_compostos`

Cálculo	HP-12C	Excel / Planilhas (pt-BR)	Python (nesta apostila)
Taxa	n, PV, PMT, FV → i	`=TAXA(nper; pgto; vp; vf)`	`calcular_cet`
VPL	`f NPV` (g CF0, g CFj)	`=VPL(taxa; valores)` + valor em t=0	`vpl`
TIR	`f IRR`	`=TIR(valores)`	`tir`
Taxa efetiva a partir da nominal	—	`=EFETIVA(nominal; k)`	`nominal_para_efetiva`

Atenção — erro comum
Duas armadilhas clássicas das planilhas. (1) As funções financeiras seguem a convenção de **sinais do fluxo de caixa**: o dinheiro que sai tem sinal negativo. `=VF(2%; 12; 0; -5000)` devolve +6.341,21; sem o sinal negativo no valor presente, o resultado vem negativo. (2) A função `VPL` do Excel **desconta o primeiro valor da lista em $t = 1$**, não em $t = 0$. O investimento inicial, que está em $t = 0$, deve ser somado *fora* da função: `=VPL(taxa; fluxos_1_a_n) + fluxo_0`. A HP-12C não tem esse problema, porque o `CF0` é informado separadamente.

Capítulo 1 — Fundamentos: O Valor do Dinheiro no Tempo

1.1. O Princípio Fundamental

O **valor do dinheiro no tempo** (time value of money — TVM) é o conceito mais importante de toda a matemática financeira. Ele afirma que **uma unidade monetária hoje vale mais do que a mesma unidade monetária em qualquer data futura**.

Por que isso acontece?

1. **Custo de oportunidade** — Se você tem R$ 1.000 hoje, pode investi-los e obter um retorno. Se só receber esse valor daqui a um ano, perdeu a oportunidade de ganhar juros nesse período.
2. **Inflação** — A tendência geral de aumento de preços faz com que o poder de compra da moeda diminua ao longo do tempo. Com R$ 1.000 hoje você compra mais do que compraria com R$ 1.000 daqui a um ano.
3. **Risco** — Existe incerteza quanto ao recebimento futuro. O devedor pode não pagar, a empresa pode quebrar, o governo pode mudar as regras. Essa incerteza precisa ser remunerada.

Desses três fatores, o mais relevante para a matemática financeira é o **custo de oportunidade**, pois ela independe da inflação ou do risco específico da operação.

Aprofundando a teoria — por que o dinheiro tem preço?
O juro é o preço do *tempo*. Três ideias clássicas da economia explicam por que ele existe:

- **Preferência temporal** (Böhm-Bawerk, Irving Fisher). As pessoas preferem consumir hoje a consumir amanhã; para abrir mão do consumo presente exigem uma compensação. Quanto maior a impaciência média da sociedade, maior o juro de equilíbrio.
- **Produtividade do capital.** Recursos aplicados hoje em uma máquina, num imóvel ou num estoque geram mais recursos amanhã. Quem toma emprestado pode pagar juros porque o capital *rende* mais do que custa.
- **Preferência pela liquidez** (Keynes). Abrir mão de dinheiro disponível implica perder flexibilidade; o juro remunera essa renúncia.

Na prática, a taxa cobrada em uma operação pode ser vista como a **composição** de três componentes, que se *multiplicam*:

$$(1 + i) = (1 + r) \cdot (1 + \pi) \cdot (1 + \theta)$$

onde r é a taxa **real** livre de risco (a remuneração pura do tempo), π é a inflação esperada e θ é o **prêmio de risco** (de crédito, de liquidez, de prazo). O Capítulo 6 desenvolve a relação entre i, r e π (equação de Fisher).

Exemplo resolvido — receber hoje ou daqui a um ano?
Você pode receber R$ 10.000 hoje ou R$ 10.800 daqui a 12 meses. Sua melhor alternativa de aplicação rende 1% a.m. Qual oferta é melhor?

Raciocínio 1 — levar tudo para o futuro. Os R$ 10.000 recebidos hoje, aplicados a 1% a.m. por 12 meses, viram $10.000 \times 1{,}01^{12} = \text{R\$ } 11.268{,}25$. Como isso supera R$ 10.800, **receber hoje é melhor**.

Raciocínio 2 — trazer tudo para hoje. Os R$ 10.800 de daqui a um ano valem hoje $\frac{10.800}{1{,}01^{12}} = \text{R\$ } 9.584{,}45$, menos que R$ 10.000. Mesma conclusão.

Raciocínio 3 — comparar taxas. A oferta de R$ 10.800 equivale a uma taxa de $\left(\frac{10.800}{10.000}\right)^{1/12} - 1 = 0{,}643\%$ a.m., menor que os 1% a.m. da alternativa.

Os três raciocínios são equivalentes — e é isso que dá consistência à matemática financeira: **qualquer data pode servir de referência**, desde que todos os valores sejam levados para a mesma.

```python
oferta_hoje, oferta_futura, taxa_alt = 10000, 10800, 0.01   # 1% a.m., prazo de 12 meses
vf_alternativa = oferta_hoje * (1 + taxa_alt) ** 12
vp_oferta_futura = oferta_futura / (1 + taxa_alt) ** 12
taxa_da_oferta = (oferta_futura / oferta_hoje) ** (1 / 12) - 1

print(f"R$10.000 hoje, aplicados por 12 meses -> R${vf_alternativa:,.2f}")
print(f"R$10.800 daqui a 12 meses valem hoje  -> R${vp_oferta_futura:,.2f}")
print(f"Taxa implícita da oferta futura       -> {taxa_da_oferta*100:.4f}% a.m.")
```

```
R$10.000 hoje, aplicados por 12 meses -> R$11,268.25
R$10.800 daqui a 12 meses valem hoje  -> R$9,584.45
Taxa implícita da oferta futura       -> 0.6434% a.m.
```

1.2. Conceitos Primitivos

A matemática financeira opera com alguns conceitos fundamentais que devem ser absolutamente dominados antes de qualquer avanço.

Conceito	Símbolo	Nomes Alternativos	Definição
Capital	C, PV	Valor Presente, Principal, Valor Atual	Quantia disponível ou devida na data de referência inicial da operação
Montante	M, FV	Valor Futuro, Valor de Resgate, Valor Nominal	Quantia disponível ou devida na data de referência final da operação
Juros	J	Remuneração, Encargos, Rendimento	Diferença entre o montante e o capital: $J = \text{FV} - \text{PV}$
Taxa de Juros	i	Taxa de juros, Rentabilidade, Custo	Razão entre os juros de um período e o capital que os produziu
Período	n	Prazo, Tempo, Duração	Intervalo de tempo ao qual a taxa de juros se refere

A **taxa de juros** merece atenção especial. Ela pode ser expressa de duas formas equivalentes:

- **Forma percentual:** $i = 12\%\ a.a.$ (lê-se "12 por cento ao ano")
- **Forma decimal (unitária):** $i = 0{,}12$ (usada em fórmulas)

A conversão é simples:

$$i_{\text{decimal}} = \frac{i_{\text{percentual}}}{100}$$

Importante: a taxa de juros e o período devem estar sempre na **mesma unidade de tempo**. Se a taxa é mensal, o período deve ser medido em meses. Se é anual, em anos.

Exemplo resolvido — lendo um enunciado
"Aplicou-se R$ 5.000 por 8 meses e resgatou-se R$ 5.600."

Traduza para símbolos: $\text{PV} = 5.000$; $\text{FV} = 5.600$; $n = 8$ meses; $J = \text{FV} - \text{PV} = 600$; taxa i *desconhecida.*

- Em **juros simples**: $i = \frac{J}{\text{PV} \cdot n} = \frac{600}{5{,}000 \cdot 8} = 1{,}5\%$ a.m.
- Em **juros compostos**: $i = \left(\frac{\text{FV}}{\text{PV}}\right)^{1/n} - 1 = 1{,}12^{1/8} - 1 \approx 1{,}4267\%$ a.m.

A mesma operação tem taxas diferentes conforme o regime adotado. Por isso, **nunca se compara uma taxa simples com uma composta** sem convertê-las.

Atenção — erro comum
Unidades incompatíveis. "Qual o montante de R$ 1.000 a 24% a.a. durante 18 meses?" Aqui a taxa é anual e o prazo, mensal. Há duas saídas equivalentes: usar $n = \frac{18}{12} = 1{,}5$ **anos** ou converter a taxa para mensal, $(1{,}24)^{1/12} - 1 \approx 1{,}8088\%$ a.m., e usar $n = 18$. O resultado é o mesmo (R$ 1.380,81). O erro típico é digitar $n = 18$ com a taxa anual — e obter um valor absurdo (veja a última linha da saída abaixo).

```
capital_ex, taxa_aa, meses_ex = 1000, 0.24, 18
via_anos = capital_ex * (1 + taxa_aa) ** (meses_ex / 12)
taxa_mensal_eq = (1 + taxa_aa) ** (1 / 12) - 1
via_meses = capital_ex * (1 + taxa_mensal_eq) ** meses_ex
errado = capital_ex * (1 + taxa_aa) ** meses_ex      # taxa anual com prazo em meses!
print(f"Prazo em anos : R${via_anos:.2f}")
print(f"Prazo em meses: R${via_meses:.2f}")
print(f"Errado (misturou unidades): R${errado:,.2f}")
```

```
Prazo em anos : R$1380.81
Prazo em meses: R$1380.81
Errado (misturou unidades): R$48,038.60
```

1.3. Fluxo de Caixa

O **fluxo de caixa** (FC, cash flow — CF) é a representação das entradas e saídas de dinheiro ao longo do tempo. É a ferramenta gráfica e analítica mais importante da matemática financeira.

Convenção: - **Entradas de caixa** (recebimentos) ⟶ seta para **cima** (+) - **Saídas de caixa** (pagamentos) ⟶ seta para **baixo** (–) - **Linha horizontal** ⟶ escala do tempo - **t = 0** ⟶ momento presente

```
R$ (entradas)
   ↑           ↑           ↑           ↑
   |-----------|-----------|-----------|---------> tempo
   |           1           2           3
   ↓
R$ (saídas)
   t = 0
```

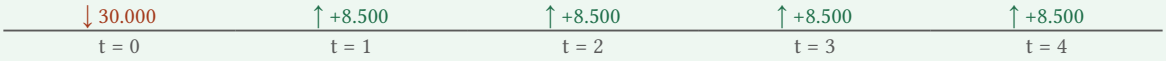
Tipos de fluxo de caixa:

Tipo	Descrição	Exemplo
Uniforme	Prestações iguais e periódicas	Financiamento em 12 parcelas fixas
Não uniforme	Valores diferentes em cada período	Fluxo de caixa de um projeto
Postecipado	Pagamentos ao final do período	Prestação sem entrada
Antecipado	Pagamentos no início do período	Aluguel pago no começo do mês
Diferido	Pagamentos com carência	Primeira prestação após 3 meses
Perpétuo	Pagamentos infinitos	Ação que paga dividendos para sempre

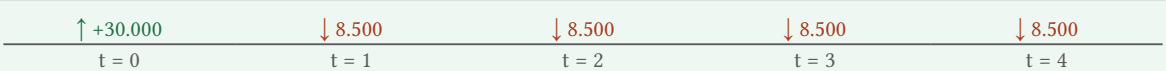
Intuição
Um fluxo de caixa é uma **fotografia da operação financeira**, tirada de um ponto de vista. O mesmo empréstimo é uma *entrada* para quem toma e uma *saída* para quem empresta — e os dois desenhos são espelhos um do outro. Antes de qualquer conta, decida de quem é o ponto de vista.

Exemplo resolvido — o mesmo empréstimo, duas visões
Um banco empresta R$ 30.000 e recebe 4 prestações mensais de R$ 8.500.

Visão do banco (credor): sai dinheiro em $t = 0$, entra em $t = 1$ a 4.



Visão do cliente (devedor): entra dinheiro em $t = 0$, saem prestações depois.



Os dois fluxos têm a **mesma taxa interna**: a taxa i que satisfaz $8.500 \cdot a_{4\,|\,i} = 30.000$ (a notação $a_{n|i}$ aparece na seção 4.2). Resolvendo numericamente, $i \approx 5{,}20\%$ a.m. — um juro alto que dificilmente seria percebido olhando só as prestações.

```python
def bisseccao(funcao, baixo, alto, passos=200):
    """Raiz de uma função contínua por bissecção (usada em vários capítulos)."""
    for _ in range(passos):
        meio = (baixo + alto) / 2
        if funcao(baixo) * funcao(meio) > 0:
            baixo = meio
        else:
            alto = meio
    return (baixo + alto) / 2

def vp_das_parcelas(taxa, parcela, n):
    return sum(parcela / (1 + taxa) ** t for t in range(1, n + 1))

taxa_do_emprestimo = bisseccao(lambda r: vp_das_parcelas(r, 8500, 4) - 30000, 1e-9, 1)
print(f"Taxa do empréstimo: {taxa_do_emprestimo*100:.4f}% a.m.")
```

```
Taxa do empréstimo: 5.2015% a.m.
```

A regra de ouro dos fluxos de caixa. Só é possível **somar, subtrair ou comparar** valores que estejam na **mesma data**. Dois pagamentos de R$ 1.000, um hoje e outro daqui a 12 meses, **não somam R$ 2.000** a 1% a.m., valem $1.000 + \frac{1.000}{1{,}01^{12}} =$ R$ 1.887,45 em valores de hoje (ou R$ 2.126,83 em valores de daqui a 12 meses). A data escolhida para a comparação chama-se **data focal** o resultado de qualquer problema não depende dela, desde que todos os valores sejam levados para a mesma.

1.4. Regimes de Capitalização

O regime de capitalização define **como os juros são calculados e incorporados ao capital** ao longo do tempo. Existem dois regimes fundamentais:

Regime de Capitalização Simples (Juros Simples): - Os juros de cada período incidem **apenas sobre o capital inicial** - O crescimento é **linear** (progressão aritmética) - Usado principalmente em operações de curto prazo (descontos, hot money)

Regime de Capitalização Composta (Juros Compostos): - Os juros de cada período incidem **sobre o montante do período anterior** - Ocorre "juros sobre juros" (anatocismo) - O crescimento é **exponencial** (progressão geométrica) - É o regime **universalmente usado** no sistema financeiro

```
import numpy as np
import matplotlib.pyplot as plt

def regime_comparativo(pv=1000, i=0.02, n_max=30):
    """
    Compara os dois regimes de capitalização graficamente.
    pv: capital inicial
    i: taxa de juros por período
    n_max: número máximo de períodos
    """
    periodos = range(n_max + 1)
    simples = [pv * (1 + i * n) for n in periodos]
    compostos = [pv * (1 + i) ** n for n in periodos]

    plt.figure(figsize=(12, 5))
    plt.plot(periodos, simples, 'b--', label='Juros Simples (linear)', linewidth=2)
    plt.plot(periodos, compostos, 'r-', label='Juros Compostos (exponencial)', linewidth=2)
    plt.fill_between(periodos, simples, compostos, alpha=0.1, color='purple',
                     label='Diferença (ganho do composto)')
    plt.axhline(pv, color='gray', linestyle=':', alpha=0.5)
    plt.xlabel('Períodos (n)')
    plt.ylabel('Montante (R$)')
    plt.title(f'Capitalização Simples vs Composta — PV = R${pv:,.0f}, i = {i*100:.0f}%')
    plt.legend()
    plt.grid(alpha=0.3)
    plt.savefig("figuras06/regimes.png", dpi=170, bbox_inches="tight")
    plt.show()

regime_comparativo()
```

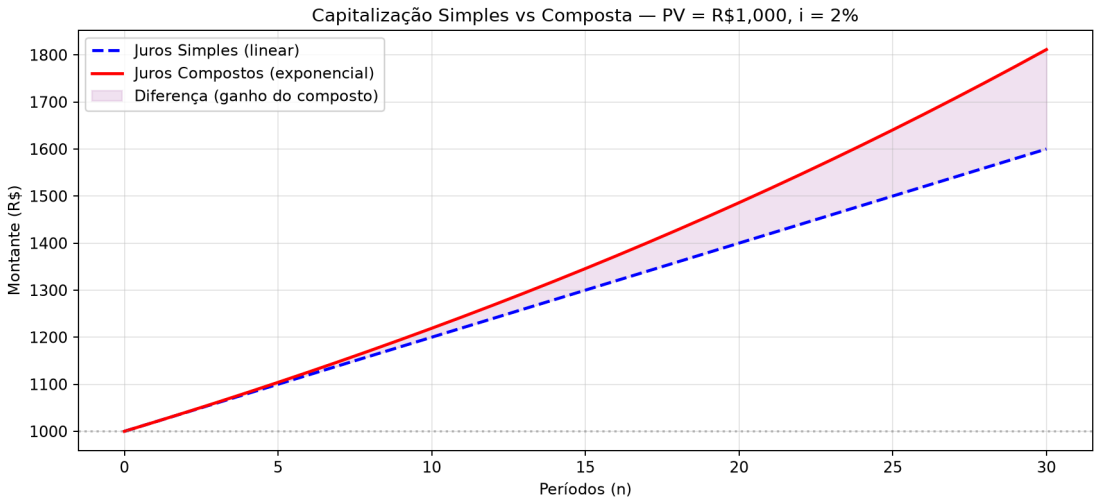


Figura 1: Montante em juros simples (reta) e compostos (curva) para o mesmo capital e a mesma taxa. A área sombreada é o ganho do "juros sobre juros".

Observação fundamental: Para n = 1, os dois regimes produzem o **mesmo montante**. Para n > 1, os juros compostos produzem montante **maior** (e a diferença cresce com n). Para n < 1 (frações de período), os juros simples podem produzir montante maior que o composto, dependendo da convenção adotada.

Comparando com números. A tabela mostra o montante de R$ 1.000 aplicados a 10% por período nos dois regimes.

n	Montante simples	Montante composto	Diferença
0	R$ 1.000,00	R$ 1.000,00	R$ 0,00
1	R$ 1.100,00	R$ 1.100,00	R$ 0,00
2	R$ 1.200,00	R$ 1.210,00	R$ 10,00
3	R$ 1.300,00	R$ 1.331,00	R$ 31,00
4	R$ 1.400,00	R$ 1.464,10	R$ 64,10
5	R$ 1.500,00	R$ 1.610,51	R$ 110,51

Tabela 1.1 — R$ 1.000 a 10% por período. Os dois regimes coincidem em $n = 0$ e $n = 1$ e se separam a partir daí.

Aprofundando a teoria — por que o composto vence a partir do segundo período

No regime simples, o ganho a cada período é **constante em reais** ($J = \text{PV} \cdot i$). No composto, o ganho a cada período é **constante em proporção ao saldo** (o saldo cresce $i\%$ ao período), de modo que o ganho em reais cresce junto com o saldo. A prova algébrica usa o **binômio de Newton**. Para n inteiro,

$$(1+i)^n = 1 + ni + \binom{n}{2} i^2 + \binom{n}{3} i^3 + \ldots + i^n$$

Os dois primeiros termos, $1 + ni$, são exatamente o fator de capitalização *simples*. Todos os termos restantes são positivos (para $i > 0$ e $n \geq 2$); logo $(1+i)^n > 1 + ni$. A diferença é o "juros sobre juros": $\binom{n}{2} i^2$ é o juro do juro do 1º período incidindo sobre o 2º, e assim por diante.

Para $0 < n < 1$ a desigualdade se inverte: $(1+i)^n < 1 + ni$ (desigualdade de Bernoulli aplicada à função côncava $x \mapsto x^n$). Com $i = 10\%$ e $n = 0{,}5$, o montante simples é R$ 1.050,00 e o composto, R$ 1.048,81. É a razão pela qual a convenção adotada para prazos fracionários importa (seção 3.5).

No mercado brasileiro

Juros sobre juros é legal no Brasil? Esse tema é recorrente em disputas judiciais. A Súmula 121 do STF veda a capitalização de juros ("anatocismo") ainda que pactuada; porém, a MP 2.170-36/2001 autorizou a

capitalização em periodicidade inferior a um ano nas operações do Sistema Financeiro Nacional, e o STJ firmou entendimento (Súmulas 539 e 541) de que ela é válida quando expressamente pactuada — bastando que a taxa anual contratada supere o duodécuplo da mensal. Na prática, todo crédito bancário no país é composto. Os juros **simples** sobrevivem em situações específicas, como juros de mora e certos descontos de curtíssimo prazo. Como regras legais mudam, confirme a norma vigente antes de usá-la em qualquer contexto profissional.

Em resumo

- O dinheiro tem preço no tempo por três razões: custo de oportunidade, inflação e risco. Só o primeiro é "juro puro".
- Taxa e prazo devem estar na mesma unidade; valores só se somam na **mesma data**.
- Simples = crescimento linear (PA); composto = exponencial (PG). Eles coincidem em $n = 1$; para $n > 1$ o composto é maior; para $0 < n < 1$ o simples é maior.
- Desenhar o fluxo de caixa — do ponto de vista correto — evita a maior parte dos erros.

Capítulo 2 — Juros Simples

2.1. Dedução das Fórmulas

Seja PV o capital inicial aplicado a uma taxa i por período, durante n períodos, no regime de juros simples.

Período 1: $J_1 = \text{PV} \times i$

Período 2: $J_2 = \text{PV} \times i$ (ainda sobre o capital inicial)

Período n: $J_n = \text{PV} \times i$

O juro total é simplesmente a soma dos juros de cada período:

$$J = J_1 + J_2 + \ldots + J_n = \text{PV} \times i \times n$$

O montante (valor futuro) é:

$$\text{FV} = \text{PV} + J = \text{PV} + \text{PV} \times i \times n = \text{PV} \times (1 + i \times n)$$

Fórmulas fundamentais (juros simples):

Objetivo	Fórmula
Juros totais	$J = \text{PV} \times i \times n$
Montante	$\text{FV} = \text{PV} \times (1 + i \times n)$
Capital inicial	$\text{PV} = \frac{\text{FV}}{1+i\times n}$
Taxa de juros	$i = \frac{\text{FV/PV} - 1}{n}$
Período	$n = \frac{\text{FV/PV} - 1}{i}$

```
def juros_simples(pv=None, fv=None, i=None, n=None):
    """
    Calcula qualquer termo faltante nos juros simples.
    Forneça exatamente 3 dos 4 parâmetros.
    """
    if pv is None:
        return fv / (1 + i * n)
    elif fv is None:
        return pv * (1 + i * n)
    elif i is None:
```

```
        return (fv / pv - 1) / n
    elif n is None:
        return (fv / pv - 1) / i

# Exemplo: Montei R$1.100 a partir de R$1.000 em 5 meses. Qual a taxa?
i = juros_simples(pv=1000, fv=1100, n=5)
print(f"Taxa: {i*100:.2f}% ao mês")
```

```
Taxa: 2.00% ao mês
```

Intuição
Pense em um aluguel: o proprietário cobra R$ 800 por mês, *sempre* R$ 800, mesmo que o inquilino fique cinco anos. O ganho em reais é constante, e o ganho total cresce em linha reta com o tempo. É isso que os juros simples fazem: o juro de cada período é uma constante ($\text{PV} \cdot i$), e o juro total é essa constante multiplicada pelo número de períodos.

Exemplo resolvido — multa e juros de mora de um boleto
Um boleto de R$ 1.500,00 foi pago com 17 dias de atraso. O contrato prevê multa de 2% (cobrada uma só vez) e juros de mora de 1% a.m., proporcionais ao número de dias ("*pro rata die*"). Quanto se paga?

- Multa: $1.500 \times 2\% = 30{,}00$.
- Juros de mora: taxa diária proporcional $= 1\frac{\%}{30}$; em 17 dias, $1.500 \times 1\frac{\%}{30} \times 17 = 8{,}50$.
- Total: $1.500 + 30{,}00 + 8{,}50 =$ **R$ 1.538,50**.

Repare que a taxa foi dividida por 30 — uma **proporção** simples, característica do regime simples.

```
def boleto_atrasado(valor, dias, multa=0.02, mora_am=0.01):
    """Multa fixa + juros de mora simples pro rata die (mês comercial de 30 dias)."""
    v_multa = valor * multa
    v_mora = valor * mora_am / 30 * dias
    return v_multa, v_mora, valor + v_multa + v_mora

m, j, total_boleto = boleto_atrasado(1500, 17)
print(f"Multa: R${m:.2f} | Juros de mora: R${j:.2f} | Total: R${total_boleto:.2f}")
```

```
Multa: R$30.00 | Juros de mora: R$8.50 | Total: R$1538.50
```

Exemplo resolvido — incógnitas diferentes
Um capital rendeu R$ 480,00 de juros simples em 6 meses a 1,6% a.m. De quanto era o capital?

$$J = \text{PV} \cdot i \cdot n \quad \Rightarrow \quad \text{PV} = \frac{J}{i \cdot n} = \frac{480}{0{,}016 \cdot 6} = 5.000$$

E se o capital fosse conhecido (R$ 5.000) e o prazo, não? A fórmula explicita $n = \frac{J}{\text{PV} \cdot i} = \frac{480}{5.000 \cdot 0{,}016} = 6$ meses. A mesma equação resolve as quatro situações — por isso os livros costumam apresentar um único "triângulo" $J = \text{PV} \cdot i \cdot n$.

2.2. Taxas Proporcionais (Juros Simples)

Duas taxas são ditas **proporcionais** quando a relação entre elas é igual à relação entre os períodos a que se referem. Em juros simples, taxas proporcionais são **equivalentes** (produzem o mesmo montante para o mesmo capital e mesmo prazo total).

$$\frac{i_1}{i_2} = \frac{n_1}{n_2}$$

Convenções de ano: - **Ano comercial (bancário):** 360 dias, 12 meses de 30 dias - **Ano civil (exato):** 365 dias (ou 366 em ano bissexto)

```
def taxas_proporcionais():
    """
    Tabela de taxas proporcionais para 24% a.a.
    """
    i_aa = 0.24
    print(f"Taxa de referência: {i_aa*100}% a.a.\n")
    linhas = [
        ("Ano", i_aa, 2, "0.24"),
        ("Semestre", i_aa / 2, 2, "0.24/2"),
        ("Trimestre", i_aa / 4, 2, "0.24/4"),
        ("Mês", i_aa / 12, 2, "0.24/12"),
        ("Dia (comercial)", i_aa / 360, 4, "0.24/360"),
        ("Dia (exato)", i_aa / 365, 4, "0.24/365"),
    ]
    print(f"{'Período':<16} {'Taxa proporcional':<19} Cálculo")
    print("-" * 52)
    for nome, taxa, casas, calculo in linhas:
        texto = f"{taxa*100:.{casas}f}%"
        print(f"{nome:<16} {texto:<19} {calculo}")

taxas_proporcionais()
```

```
Taxa de referência: 24.0% a.a.

Período          Taxa proporcional   Cálculo
----------------------------------------------------
Ano              24.00%              0.24
Semestre         12.00%              0.24/2
Trimestre        6.00%               0.24/4
Mês              2.00%               0.24/12
Dia (comercial)  0.0667%             0.24/360
Dia (exato)      0.0658%             0.24/365
```

Exemplo resolvido — proporcional não é só uma regra de três
Uma taxa de 18% a.a. em juros simples equivale a 1,5% a.m. e a 4,5% a.t. Para R$ 1.000 em um ano:

- Taxa anual: $1.000 \times (1 + 0{,}18 \times 1) = 1.180{,}00$
- Taxa mensal: $1.000 \times (1 + 0{,}015 \times 12) = 1.180{,}00$
- Taxa trimestral: $1.000 \times (1 + 0{,}045 \times 4) = 1.180{,}00$

Todas dão o mesmo montante: em juros simples, taxas proporcionais são **equivalentes**. Essa propriedade **não vale** no regime composto: 1,5% a.m. capitalizados 12 vezes produzem $1{,}015^{12} = 1{,}1956$ (19,56% a.a.), não 18%.

2.3. Juros Exatos vs Juros Comerciais

Juros comerciais (ordinários): utilizam o ano de 360 dias. São os mais usados em operações bancárias de curto prazo.

$$J_{\text{comercial}} = \text{PV} \times i_{\text{anual}} \times \frac{\text{dias}}{360}$$

Juros exatos: utilizam o ano civil de 365 dias.

$$J_{\text{exato}} = \text{PV} \times i_{\text{anual}} \times \frac{\text{dias}}{365}$$

```python
def comparar_juros(pv, i_aa, dias):
    j_com = pv * i_aa * dias / 360
    j_exa = pv * i_aa * dias / 365
    print(f"Capital: R${pv:,.2f} | Taxa: {i_aa*100:.1f}% a.a. | Período: {dias} dias\n")
    print(f"Juros comerciais (360 dias): R${j_com:,.2f}")
    print(f"Juros exatos (365 dias):     R${j_exa:,.2f}")
    print(f"Diferença:                   R${j_com - j_exa:,.4f}")
    print(f"Relação J_comercial / J_exato: {j_com/j_exa:.4f} (= 365/360 = {365/360:.4f})")

comparar_juros(10000, 0.12, 45)
```

```
Capital: R$10,000.00 | Taxa: 12.0% a.a. | Período: 45 dias

Juros comerciais (360 dias): R$150.00
Juros exatos (365 dias):     R$147.95
Diferença:                   R$2.0548
Relação J_comercial / J_exato: 1.0139 (= 365/360 = 1.0139)
```

Observação: Os juros comerciais são sempre **maiores** que os exatos para um mesmo capital, taxa e prazo, pois o ano de 360 dias divide o numerador por um denominador menor. A relação entre eles é sempre $J_{\text{com}}/J_{\text{ex}} = 365/360 \approx 1{,}0139$.

Aprofundando a teoria — o que significa "ano de 360 dias"
O mercado usa três convenções de contagem de dias, e a escolha altera o valor dos juros:

Convenção	Como conta	Fórmula do juro simples
Comercial (30/360)	Cada mês tem 30 dias; o ano, 360	$J = \text{PV} \cdot i_{aa} \cdot \frac{d_{30/360}}{360}$
Bancária (real/360)	Dias corridos reais; ano de 360 dias	$J = \text{PV} \cdot i_{aa} \cdot \frac{d_{\text{real}}}{360}$
Exata (real/365)	Dias corridos reais; ano de 365 dias	$J = \text{PV} \cdot i_{aa} \cdot \frac{d_{\text{real}}}{365}$

A convenção **bancária** é a mais usada em operações de crédito de curto prazo: ela divide por 360, mas conta os dias reais do calendário. Isso a torna **mais cara** que a exata em 365/360 ≈ 1,39%.

```python
from datetime import date

def dias_30_360(d1, d2):
    """Contagem de dias pelo método comercial: todo mês tem 30 dias."""
    return (d2.year - d1.year) * 360 + (d2.month - d1.month) * 30 + (min(d2.day, 30) -
min(d1.day, 30))

inicio, fim = date(2026, 3, 10), date(2026, 6, 18)
capital_ex2, taxa_aa_ex2 = 50000, 0.12
d_real = (fim - inicio).days
d_com = dias_30_360(inicio, fim)

print(f"Dias reais: {d_real} | dias na contagem 30/360: {d_com}")
print(f"Comercial 30/360 : R${capital_ex2 * taxa_aa_ex2 * d_com / 360:,.2f}")
print(f"Bancária real/360: R${capital_ex2 * taxa_aa_ex2 * d_real / 360:,.2f}")
print(f"Exata real/365   : R${capital_ex2 * taxa_aa_ex2 * d_real / 365:,.2f}")
```

```
Dias reais: 100 | dias na contagem 30/360: 98
Comercial 30/360 : R$1,633.33
Bancária real/360: R$1,666.67
Exata real/365   : R$1,643.84
```

2.4. Descontos Simples

Desconto é o abatimento concedido pelo pagamento antecipado de um título. Existem duas modalidades fundamentais.

2.4.1. Desconto Comercial (por Fora)

O desconto é calculado **sobre o valor nominal** (valor futuro, de face) do título. É o mais usado pelo mercado bancário brasileiro.

$$D_c = \text{FV} \times i \times n$$

$$\text{PV} = \text{FV} - D_c = \text{FV} \times (1 - i \times n)$$

Onde: - FV = valor nominal (valor de face, valor no vencimento) - PV = valor atual (valor descontado, valor líquido) - i = taxa de desconto - n = período de antecipação

Restrição: a operação só faz sentido se $i \times n < 1$. Caso contrário, PV seria zero ou negativo.

2.4.2. Desconto Racional (por Dentro)

O desconto é calculado **sobre o valor presente**. É o "verdadeiro" desconto, pois usa a mesma lógica dos juros.

$$D_r = \text{PV} \times i \times n$$

$$\text{PV} = \frac{\text{FV}}{1 + i \times n}$$

2.4.3. Relação entre os Descontos

- $D_c > D_r$ para o mesmo FV, i e n
- $\text{PV}_c < \text{PV}_r$ (o desconto comercial concede um valor líquido menor)

```python
def descontos(fv, i, n):
    """
    Calcula os dois tipos de desconto simples.
    """
    d_com = fv * i * n
    pv_com = fv - d_com
    pv_rac = fv / (1 + i * n)
    d_rac = fv - pv_rac

    print(f"{'Variável':<20} {'Comercial':<15} {'Racional':<15}")
    print("-" * 50)
    print(f"{'Valor Nominal (FV)':<20} R${fv:<10.2f} R${fv:<10.2f}")
    print(f"{'Valor Atual (PV)':<20} R${pv_com:<10.2f} R${pv_rac:<10.2f}")
    print(f"{'Desconto (D)':<20} R${d_com:<10.2f} R${d_rac:<10.2f}")
    t_com = f"{d_com / pv_com * 100:.2f}%"
    t_rac = f"{d_rac / pv_rac * 100:.2f}%"
    print(f"{'Taxa efetiva (prazo)':<20} {t_com:<15} {t_rac}")

descontos(5000, 0.03, 2)
```

```
Variável             Comercial       Racional
--------------------------------------------------
Valor Nominal (FV)   R$5000.00     R$5000.00
Valor Atual (PV)     R$4700.00     R$4716.98
Desconto (D)         R$300.00      R$283.02
Taxa efetiva (prazo) 6.38%            6.00%
```

> **Intuição**
> Descontar um título é **comprá-lo com deságio**. Quem desconta recebe hoje menos que o valor de face, e a diferença é o ganho de quem compra o título. A pergunta decisiva é: *sobre que valor* a taxa incide?

No desconto **comercial** (por fora), incide sobre o valor *futuro*, que é maior que o valor efetivamente emprestado — por isso a taxa "de desconto" **subestima** o custo real.

Exemplo resolvido — antecipação de uma duplicata
Uma empresa antecipa uma duplicata de R$ 20.000 que vence em 90 dias. O banco cobra taxa de desconto comercial de 2,5% a.m.

- Desconto: $D_c = 20.000 \times 0{,}025 \times 3 = 1.500{,}00$.
- Valor líquido recebido: $20.000 - 1.500 = 18.500{,}00$.
- Taxa de juros efetiva (simples): a empresa pegou R$ 18.500 e devolve R$ 20.000 em 3 meses, logo $i = \frac{1.500}{18.500 \cdot 3} = 2{,}70\%$ a.m. — maior que os 2,5% anunciados. Pela fórmula da seção 2.5, $i = \frac{d}{1-dn} = \frac{0{,}025}{1-\,0{,}075} = 2{,}70\%$.
- Em juros **compostos**, a taxa efetiva seria $\left(\frac{20.000}{18.500}\right)^{1/3} - 1 = 2{,}63\%$ a.m.

Pelo desconto **racional** à mesma taxa, o valor líquido seria $\frac{20.000}{1+\,0{,}025\,\times 3} = 18.604{,}65$: R$ 104,65 a mais para a empresa.

```
face, d_mensal, meses_dup = 20000, 0.025, 3
desc_comercial = face * d_mensal * meses_dup
liquido_comercial = face - desc_comercial
liquido_racional = face / (1 + d_mensal * meses_dup)

print(f"Desconto comercial: R${desc_comercial:,.2f} | líquido: R${liquido_comercial:,.2f}")
print(f"Desconto racional : R${face - liquido_racional:,.2f} | líquido:
R${liquido_racional:,.2f}")
print(f"Taxa efetiva simples : {desc_comercial / liquido_comercial / meses_dup * 100:.2f}%
a.m.")
print(f"Taxa efetiva composta: {((face / liquido_comercial) ** (1 / meses_dup) - 1) *
100:.2f}% a.m.")
```

```
Desconto comercial: R$1,500.00 | líquido: R$18,500.00
Desconto racional : R$1,395.35 | líquido: R$18,604.65
Taxa efetiva simples : 2.70% a.m.
Taxa efetiva composta: 2.63% a.m.
```

Aprofundando a teoria — a relação entre os dois descontos
Seja FV o valor de face. Pelas definições,

$$D_c = \text{FV} \cdot i \cdot n \quad \text{e} \quad D_r = \frac{\text{FV} \cdot i \cdot n}{1 + in}$$

(o desconto racional é calculado sobre $\text{PV} = \frac{\text{FV}}{1+in}$). Dividindo uma expressão pela outra, obtém-se $D_c = D_r \cdot (1 + in)$. Como $1 + in > 1$, o desconto comercial é sempre **maior** que o racional, **à mesma taxa**. Verificação com a tabela da seção 2.4: $283{,}02 \times 1{,}06 = 300{,}00$ ✓.

No mercado brasileiro
Desconto de recebíveis. Empresas antecipam duplicatas, cheques pré-datados e recebíveis de cartão cotando a *taxa de desconto* em % ao mês. Além do deságio, incidem IOF e tarifas, o que eleva o custo efetivo acima do $\frac{d}{1-dn}$ calculado aqui. A comparação correta entre propostas de dois bancos é sempre pela **taxa efetiva** (ou CET, seção 5.7), nunca pela taxa de desconto anunciada.

2.5. Equivalência entre Taxa de Desconto e Taxa de Juros

A taxa de desconto d e a taxa de juros i são diferentes. Para um mesmo valor nominal FV e mesmo prazo n:

$$i = \frac{d}{1 - d \times n}$$

$$d = \frac{i}{1 + i \times n}$$

```python
def taxa_desconto_para_juros(d, n):
    """Converte taxa de desconto comercial para taxa de juros efetiva"""
    return d / (1 - d * n)

def taxa_juros_para_desconto(i, n):
    """Converte taxa de juros para taxa de desconto comercial"""
    return i / (1 + i * n)

# Uma taxa de desconto de 3% a.m. em 2 meses equivale a que taxa de juros?
d, n = 0.03, 2
i_eq = taxa_desconto_para_juros(d, n)
print(f"Taxa de desconto {d*100}% a.m. = Taxa de juros {i_eq*100:.2f}% a.m.")
```

```
Taxa de desconto 3.0% a.m. = Taxa de juros 3.19% a.m.
```

Exemplo resolvido — de desconto para juros
Um título de R$ 10.000 vence em 60 dias e é descontado a 3,5% a.m. (desconto comercial).

- Valor líquido: $10.000 \times (1 - 0{,}035 \times 2) = 9.300{,}00$.
- Taxa de juros equivalente: $i = \frac{d}{1-dn} = \frac{0{,}035}{1-\,0{,}07} = 3{,}76\%$ a.m.

Demonstração da fórmula. No desconto comercial, $\text{PV} = \text{FV}(1 - dn)$. Na ótica dos juros simples, $\text{FV} = \text{PV}(1 + in)$. Substituindo uma na outra: $(1 - dn)(1 + in) = 1$, e isolando i, $i = \frac{d}{1-dn}$ — que existe apenas se $dn < 1$.

2.6. Prazo Médio de um Conjunto de Títulos

Quando se deseja substituir vários títulos por um único, calcula-se o **prazo médio**, que é a média ponderada dos prazos pelos valores nominais.

$$n_{\text{médio}} = \frac{\Sigma(\text{FV}_k \times n_k)}{\Sigma \text{FV}_k}$$

```python
def prazo_medio(titulos):
    """
    titulos: lista de (valor_nominal, prazo_em_meses)
    """
    soma_valores = sum(fv for fv, _ in titulos)
    soma_ponderada = sum(fv * n for fv, n in titulos)
    return soma_ponderada / soma_valores

titulos = [(5000, 30), (8000, 60), (7000, 90)]
pm = prazo_medio(titulos)
print(f"Prazo médio: {pm:.2f} dias")
```

```
Prazo médio: 63.00 dias
```

Aprofundando a teoria — equivalência de capitais e a data focal
Dois valores em datas distintas são **equivalentes** quando valem o mesmo na data focal escolhida. É o princípio que sustenta a troca de um conjunto de títulos por outro (renegociação, refinanciamento).

Em juros simples há um problema sutil: o resultado **depende da data focal** escolhida. Exemplo: substituir dois títulos de R$ 6.000 (vencimentos em 30 e 60 dias) por um único título com vencimento em 90 dias, a 2% a.m. de juros simples.

- Focal em **hoje** (desconto racional): $\text{VP} = \frac{6.000}{1{,}02} + \frac{6.000}{1{,}04} = 11.651{,}58$; no dia 90, $11.651{,}58 \times 1{,}06 = 12.350{,}68$.
- Focal no **dia 90** (capitalizando): $6.000 \times 1{,}04 + 6.000 \times 1{,}02 = 12.360{,}00$.

Os dois resultados diferem em R$ 9,32. Em juros **compostos** a diferença desaparece: dinheiro cresce exponencialmente e o resultado é independente da data focal — uma das razões de o regime composto ser o padrão no mercado.

Exemplo resolvido — prazo médio na prática
Três títulos (R$ 5.000 em 30 dias, R$ 8.000 em 60 e R$ 7.000 em 90) serão substituídos por um único de R$ 20.000. Em que prazo ele deve vencer para que a troca seja justa?

$$n_{\text{médio}} = \frac{5.000 \cdot 30 + 8.000 \cdot 60 + 7.000 \cdot 90}{5.000 + 8.000 + 7.000} = 1.260.\frac{000}{20.000} = 63 \text{ dias}$$

Cuidado: o prazo médio é **exato apenas no desconto comercial simples**. Com desconto de 2% a.m.: os três títulos valem hoje 19.160,00 e o título único de 63 dias também ($20.000 \times (1 - 0{,}02 \times 2{,}1) = 19.160{,}00$). No desconto **racional**, os valores são R$ 19.198,04 e R$ 19.193,86: próximos, mas não iguais.

```
titulos_ex = [(5000, 30), (8000, 60), (7000, 90)]   # (valor de face, dias)
taxa_dia = 0.02 / 30
soma_face = sum(v for v, _ in titulos_ex)
pm = sum(v * d for v, d in titulos_ex) / soma_face

vp_com_orig = sum(v * (1 - taxa_dia * d) for v, d in titulos_ex)
vp_com_unico = soma_face * (1 - taxa_dia * pm)
vp_rac_orig = sum(v / (1 + taxa_dia * d) for v, d in titulos_ex)
vp_rac_unico = soma_face / (1 + taxa_dia * pm)

print(f"Prazo médio: {pm:.1f} dias")
print(f"Desconto comercial -> originais: R${vp_com_orig:,.2f} | título único:
R${vp_com_unico:,.2f}")
print(f"Desconto racional  -> originais: R${vp_rac_orig:,.2f} | título único:
R${vp_rac_unico:,.2f}")
```

```
Prazo médio: 63.0 dias
Desconto comercial -> originais: R$19,160.00 | título único: R$19,160.00
Desconto racional  -> originais: R$19,198.04 | título único: R$19,193.86
```

Em resumo
- Juros simples: $J = \text{PV} \cdot i \cdot n$ e $\text{FV} = \text{PV}(1 + in)$; o crescimento é linear. Taxas proporcionais são equivalentes.
- O número de dias depende da **convenção**: 30/360, real/360 ou real/365. Em crédito bancário usa-se geralmente real/360.
- Desconto comercial: $D_c = \text{FV} \cdot d \cdot n$. Desconto racional: $D_r = \frac{D_c}{1+in}$. O comercial é maior, e a taxa efetiva do comercial é $\frac{d}{1-dn}$.
- Prazo médio e equivalência de capitais dependem da data focal em juros simples — outra razão para preferir o regime composto.

Capítulo 3 — Juros Compostos

3.1. Dedução das Fórmulas

No regime de juros compostos, a **base de cálculo dos juros se renova a cada período**. Os juros do período anterior são incorporados ao capital para o cálculo dos juros do período seguinte.

Período 1:

$$\mathrm{FV}_1 = \mathrm{PV} + \mathrm{PV} \times i = \mathrm{PV} \times (1 + i)$$

Período 2:

$$\mathrm{FV}_2 = \mathrm{FV}_1 + \mathrm{FV}_1 \times i = \mathrm{FV}_1 \times (1 + i) = \mathrm{PV} \times (1 + i)^2$$

Período n:

$$\mathrm{FV}_n = \mathrm{PV} \times (1 + i)^n$$

Fórmulas fundamentais (juros compostos):

Objetivo	Fórmula
Montante	$\mathrm{FV} = \mathrm{PV} \times (1 + i)^n$
Capital inicial	$\mathrm{PV} = \frac{\mathrm{FV}}{(1+i)^n} = \mathrm{FV} \times (1 + i)^{-n}$
Juros totais	$J = \mathrm{FV} - \mathrm{PV} = \mathrm{PV} \times [(1 + i)^n - 1]$
Taxa de juros	$i = \left(\frac{\mathrm{FV}}{\mathrm{PV}}\right)^{1/n} - 1$
Período	$n = \frac{\ln(\mathrm{FV}/\mathrm{PV})}{\ln(1+i)}$

```python
import math

def juros_compostos(pv=None, fv=None, i=None, n=None):
    """
    Calcula qualquer termo faltante nos juros compostos.
    Forneça exatamente 3 dos 4 parâmetros.
    """
    if pv is None:
        return fv / (1 + i) ** n
    elif fv is None:
        return pv * (1 + i) ** n
    elif i is None:
        return (fv / pv) ** (1 / n) - 1
    elif n is None:
        return math.log(fv / pv) / math.log(1 + i)

# Exemplo 1: R$5.000 a 2% a.m. por 12 meses
fv = juros_compostos(pv=5000, i=0.02, n=12)
print(f"Ex.1: R$5.000 a 2% a.m. por 12m = R${fv:.2f}")

# Exemplo 2: Qual taxa transforma R$1.000 em R$2.000 em 24 meses?
i = juros_compostos(pv=1000, fv=2000, n=24)
print(f"Ex.2: Taxa para dobrar em 24m = {i*100:.4f}% a.m.")

# Exemplo 3: Quanto tempo para R$1.000 virar R$3.000 a 1,5% a.m.?
n = juros_compostos(pv=1000, fv=3000, i=0.015)
print(f"Ex.3: Tempo para triplicar a 1,5% a.m. = {n:.2f} meses")
```

```
Ex.1: R$5.000 a 2% a.m. por 12m = R$6341.21
Ex.2: Taxa para dobrar em 24m = 2.9302% a.m.
Ex.3: Tempo para triplicar a 1,5% a.m. = 73.79 meses
```

Intuição
No regime composto, cada período multiplica o saldo pelo **fator de capitalização** $(1+i)$. Aplicar n períodos é multiplicar n vezes pelo mesmo fator — daí o expoente. O processo inverso, trazer um valor futuro para hoje, é **dividir** por esse fator n vezes, ou multiplicar pelo **fator de descapitalização** $(1+i)^{-n}$.

Exemplo resolvido — construindo a fórmula período a período
R$ 1.000 aplicados a 3% a.m. Em cada mês, multiplica-se o saldo por 1,03:

Mês	Cálculo	Juros do mês	Saldo final
0	—	—	1.000,00
1	$1.000 \times 1{,}03$	30,00	1.030,00
2	$1.030 \times 1{,}03$	30,90	1.060,90
3	$1.060{,}90 \times 1{,}03$	31,83	1.092,73
4	$1.092{,}73 \times 1{,}03$	32,78	1.125,51

Pela fórmula fechada: $1.000 \times 1{,}03^4 = 1.125{,}51$ ✓. Note que os juros do mês crescem (30,00 ⟶ 32,78): é o efeito "juros sobre juros".

Exemplo resolvido — valor presente de uma dívida
Uma dívida de R$ 20.000 vence em 18 meses. Se a taxa de mercado é de 1,5% a.m., quanto vale essa dívida hoje, ou seja, quanto seria preciso pagar para quitá-la agora?

$$\text{PV} = \frac{\text{FV}}{(1+i)^n} = \frac{20.000}{1{,}015^{18}} = \frac{20.000}{1{,}30734} \approx \text{R\$ } 15.298{,}23$$

Interpretação: R$ 15.298,23 aplicados hoje a 1,5% a.m. rendem exatamente R$ 20.000 em 18 meses. A diferença de R$ 4.701,77 é o **desconto** pela antecipação.

```python
vf_divida, taxa_div, meses_div = 20000, 0.015, 18
vp_divida = vf_divida / (1 + taxa_div) ** meses_div
print(f"Valor presente: R${vp_divida:,.2f} | desconto: R${vf_divida - vp_divida:,.2f}")
print(f"Conferência (capitalizando de volta): R${vp_divida * (1 + taxa_div) **
meses_div:,.2f}")
```

```
Valor presente: R$15,298.23 | desconto: R$4,701.77
Conferência (capitalizando de volta): R$20,000.00
```

Taxas variáveis ao longo do tempo. Quando a taxa muda de um período para outro, o fator acumulado é o **produto** dos fatores de cada período:

$$(1+i_{\text{acum}}) = (1+i_1)(1+i_2)\cdots(1+i_n)$$

Exemplo: inflações mensais de 0,4%, 0,5% e 0,3% acumulam $1{,}004 \times 1{,}005 \times 1{,}003 - 1 = 1{,}2047\%$ no trimestre — não 1,2% (a soma simples) — porque cada mês incide sobre o preço já corrigido.

3.2. A Regra dos 72
Uma aproximação útil para estimar o tempo de duplicação de um capital a uma dada taxa:

$$n_{\text{dobro}} \approx \frac{72}{i_{\text{percentual}}}$$

```python
def regra_72(taxa_percentual):
    n_aproximado = 72 / taxa_percentual
    n_exato = math.log(2) / math.log(1 + taxa_percentual / 100)
    erro = abs(n_aproximado - n_exato) / n_exato * 100
    return n_aproximado, n_exato, erro

for taxa in [1, 2, 3, 5, 10, 15, 20]:
    aprox, exato, erro = regra_72(taxa)
    print(f"Taxa {taxa:>2}%: Regra 72={aprox:.2f}m, Exato={exato:.2f}m, Erro={erro:.1f}%")
```

```
Taxa  1%: Regra 72=72.00m, Exato=69.66m, Erro=3.4%
Taxa  2%: Regra 72=36.00m, Exato=35.00m, Erro=2.8%
Taxa  3%: Regra 72=24.00m, Exato=23.45m, Erro=2.3%
Taxa  5%: Regra 72=14.40m, Exato=14.21m, Erro=1.4%
Taxa 10%: Regra 72=7.20m, Exato=7.27m, Erro=1.0%
Taxa 15%: Regra 72=4.80m, Exato=4.96m, Erro=3.2%
Taxa 20%: Regra 72=3.60m, Exato=3.80m, Erro=5.3%
```

Aprofundando a teoria — de onde vem o número 72

Para dobrar um capital, queremos $(1+i)^n = 2$, ou seja, $n = \ln \frac{2}{\ln(1+i)}$, com $\ln 2 \approx 0{,}693$. Para taxas pequenas, a série de Taylor dá $\ln(1+i) \approx i - \frac{i^2}{2}$, de modo que

$$n \approx \frac{0{,}693}{i\left(1 - \frac{i}{2}\right)} \approx \left(\frac{69{,}3}{100i}\right)\left(1 + \frac{i}{2}\right)$$

onde, na última expressão, a taxa é percentual. O fator de correção $\left(1 + \frac{i}{2}\right)$ vale cerca de 1,04 para taxas próximas de 8%, o que leva 69,3 a aproximadamente 72. A "regra dos 72" é, portanto, uma **calibração do 69,3 para a faixa de taxas de 6% a 10%**, onde ela é praticamente exata (a 8%, o prazo exato é 9,006 períodos; a regra dá 9,000). Para taxas muito baixas, a "regra dos 70" ou do 69,3 é mais precisa; para taxas altas (acima de 15%), a regra dos 72 subestima o prazo.

Exemplo resolvido — usando a regra

- A poupança rende cerca de 6,17% a.a. Quanto tempo para dobrar o capital? $\frac{72}{6{,}17} \approx 11{,}7$ anos (exato: 11,58).
- A inflação média é de 6% a.a. Em quanto tempo o poder de compra da moeda cai pela metade? $\frac{72}{6} = 12$ anos (exato: 11,9). Ou seja, em 12 anos, R$ 100 compram o que R$ 50 compram hoje — é o que significa $\frac{1}{1{,}06^{12}} \approx 0{,}497$.

3.3. Taxas Equivalentes (Juros Compostos)

Em juros compostos, **taxas proporcionais NÃO são equivalentes**. Duas taxas são equivalentes quando, aplicadas ao mesmo capital pelo mesmo prazo total, produzem o mesmo montante, **mas em períodos de capitalização diferentes**.

A relação fundamental:

$$(1 + i_1)^{n_1} = (1 + i_2)^{n_2}$$

Generalizando:

$$i_{\text{maior}} = (1 + i_{\text{menor}})^{k} - 1$$

$$i_{\text{menor}} = (1 + i_{\text{maior}})^{1/k} - 1$$

Onde k é o número de períodos menores contidos no período maior.

```python
def taxa_equivalente(i, periodo_origem, periodo_destino):
    """
    Converte taxas no regime composto.
    Períodos: 'ano', 'semestre', 'trimestre', 'mes', 'dia'
    """
    periodos = {'ano': 1, 'semestre': 2, 'trimestre': 4, 'mes': 12, 'dia': 360}
    k = periodos[periodo_destino] / periodos[periodo_origem]
    return (1 + i) ** (1 / k) - 1 if k > 0 else (1 + i) ** (-k) - 1

# 24% a.a. para diversas bases
i_aa = 0.24
print("Conversão de 24% a.a. para outras bases (regime composto):")
print(f"  Mensal:  {taxa_equivalente(i_aa, 'ano', 'mes')*100:.4f}% a.m.")
print(f"  Diária:  {taxa_equivalente(i_aa, 'ano', 'dia')*100:.4f}% a.d.")
print(f"  Semestral: {taxa_equivalente(i_aa, 'ano', 'semestre')*100:.4f}% a.s.")

# E no caminho inverso: 2% a.m. para anual
i_am = 0.02
i_aa_equiv = taxa_equivalente(i_am, 'mes', 'ano')
print(f"\n2% a.m. = {i_aa_equiv*100:.2f}% a.a. (equivalente composto)")
print(f"2% a.m. = {i_am*12*100:.2f}% a.a. (proporcional simples — incorreto em composto)")
```

```
Conversão de 24% a.a. para outras bases (regime composto):
  Mensal:  1.8088% a.m.
  Diária:  0.0598% a.d.
  Semestral: 11.3553% a.s.

2% a.m. = 26.82% a.a. (equivalente composto)
2% a.m. = 24.00% a.a. (proporcional simples — incorreto em composto)
```

Aprofundando a teoria — a prova da equivalência

Aplique R$ 1 por um ano, de duas formas: (a) à taxa anual i_a, uma vez; (b) à taxa mensal i_m, capitalizada 12 vezes. Os montantes são $(1+i_a)$ e $(1+i_m)^{12}$. As duas taxas são **equivalentes** se os montantes forem iguais:

$$(1+i_a) = (1+i_m)^{12} \quad \Rightarrow \quad i_m = (1+i_a)^{1/12} - 1$$

A generalização é imediata: para k períodos menores dentro de um maior, $(1+i_{\text{maior}}) = (1+i_{\text{menor}})^{k}$. O erro clássico é aplicar a proporcionalidade dos juros simples ($i_m = \frac{i_a}{12}$) a taxas compostas: isso dá uma taxa mensal **maior** que a equivalente. Com $i_a = 12{,}68\%$, a taxa proporcional seria $1{,}057\%$ a.m., e 12 capitalizações dariam 13,45% a.a., não 12,68%.

Exemplo resolvido — conversões do dia a dia

- **1% a.m. em taxa anual:** $1{,}01^{12} - 1 = 12{,}6825\%$ a.a. (e não 12%).
- **Poupança, 0,5% a.m.:** $1{,}005^{12} - 1 = 6{,}1678\%$ a.a.
- **5% a.t. em taxa mensal:** $1{,}05^{1/3} - 1 = 1{,}6396\%$ a.m.; em taxa anual, $1{,}05^{4} - 1 = 21{,}55\%$ a.a.
- **Inflação:** um IPCA de 4,44% em 12 meses corresponde a uma inflação **média** de $1{,}0444^{1/12} - 1 = 0{,}3627\%$ a.m. (dado real do IPCA acumulado em 12 meses até julho de 2026; série SGS 13522 do BCB, arquivo `ipca_12m.json` desta pasta).

No mercado brasileiro

Taxas "over" e base 252. No mercado de renda fixa brasileiro, o CDI e a Selic são cotados em % a.a. na **base de 252 dias úteis**. A taxa diária equivalente a 13,25% a.a. é $1{,}1325^{1/252} - 1 = 0{,}04939\%$ a.d. Um mês com 21 dias úteis rende $1{,}1325^{21/252} - 1 = 1{,}0423\%$ — note que o rendimento de um "mês" **depende**

de quantos dias úteis ele tem. Já a taxa "over", usada em operações compromissadas, é uma taxa diária multiplicada por 30 (proporção simples), exceção histórica ao padrão composto.

```
def converte_taxa(taxa, de_dias, para_dias):
    """Converte taxa composta entre bases de tempo (em dias da mesma base: úteis ou
corridos)."""
    return (1 + taxa) ** (para_dias / de_dias) - 1

cdi_aa = 0.1325
print(f"CDI 13,25% a.a. -> diária (252 du): {converte_taxa(cdi_aa, 252, 1)*100:.5f}%")
print(f"CDI 13,25% a.a. -> 21 dias úteis : {converte_taxa(cdi_aa, 252, 21)*100:.4f}%")
print(f"CDI 13,25% a.a. -> 63 dias úteis : {converte_taxa(cdi_aa, 252, 63)*100:.4f}%")
print(f"IPCA 12m 4,44% -> média mensal  : {converte_taxa(0.0444, 12, 1)*100:.4f}% a.m.")
```

```
CDI 13,25% a.a. -> diária (252 du): 0.04939%
CDI 13,25% a.a. -> 21 dias úteis : 1.0423%
CDI 13,25% a.a. -> 63 dias úteis : 3.1596%
IPCA 12m 4,44% -> média mensal  : 0.3627% a.m.
```

3.4. Taxa Nominal vs Taxa Efetiva

Taxa Nominal: é a taxa declarada no contrato, mas cujo período de capitalização é **diferente** do período a que ela se refere. Exemplo: "24% ao ano **capitalizados mensalmente**".

Taxa Efetiva: é a taxa que efetivamente será aplicada no período considerado.

Para converter nominal em efetiva:

$$i_{\text{efetiva}} = \left(1 + \frac{i_{\text{nominal}}}{k}\right)^{k} - 1$$

Onde k = número de capitalizações dentro do período nominal.

```
def nominal_para_efetiva(i_nominal, capitalizacoes_por_periodo):
    """
    Converte taxa nominal para taxa efetiva.
    Ex: nominal 24% a.a. capitalizada mensalmente -> capitalizacoes_por_periodo = 12
    """
    return (1 + i_nominal / capitalizacoes_por_periodo) ** capitalizacoes_por_periodo - 1

def efetiva_para_nominal(i_efetiva, capitalizacoes_por_periodo):
    """
    Converte taxa efetiva para taxa nominal.
    """
    return ((1 + i_efetiva) ** (1 / capitalizacoes_por_periodo) - 1) *
capitalizacoes_por_periodo

print("Conversões de taxas nominais para efetivas:")
for taxa_nominal in [0.12, 0.15]:
    for cap in [2, 4, 6, 12]:
        ef = nominal_para_efetiva(taxa_nominal, cap)
        print(f"  {taxa_nominal*100:.0f}% a.a. cap. {cap}x/ano -> efetiva {ef*100:.2f}%
a.a.")
    print()
```

```
Conversões de taxas nominais para efetivas:
  12% a.a. cap. 2x/ano -> efetiva 12.36% a.a.
  12% a.a. cap. 4x/ano -> efetiva 12.55% a.a.
  12% a.a. cap. 6x/ano -> efetiva 12.62% a.a.
```

```
  12% a.a. cap. 12x/ano -> efetiva 12.68% a.a.

  15% a.a. cap. 2x/ano -> efetiva 15.56% a.a.
  15% a.a. cap. 4x/ano -> efetiva 15.87% a.a.
  15% a.a. cap. 6x/ano -> efetiva 15.97% a.a.
  15% a.a. cap. 12x/ano -> efetiva 16.08% a.a.
```

Implicação importante: quanto **maior** o número de capitalizações dentro do período, **maior** será a taxa efetiva. O limite teórico é a **capitalização contínua**:

$$i_{\text{contínua}} = e^{i_{\text{nominal}}} - 1$$

onde $e \approx 2{,}71828$ (número de Euler).

```python
import math

def capitalizacao_continua(i_nominal):
    return math.exp(i_nominal) - 1

i_nom = 0.20
print(f"20% a.a. com capitalização contínua: {capitalizacao_continua(i_nom)*100:.4f}%
a.a.")
print(f"20% a.a. com capitalização mensal: {nominal_para_efetiva(i_nom, 12)*100:.2f}%
a.a.")
print(f"20% a.a. com capitalização diária: {nominal_para_efetiva(i_nom, 360)*100:.4f}%
a.a.")
```

```
20% a.a. com capitalização contínua: 22.1403% a.a.
20% a.a. com capitalização mensal: 21.94% a.a.
20% a.a. com capitalização diária: 22.1335% a.a.
```

Aprofundando a teoria — capitalização contínua e o número e
Quanto vale, no limite, a capitalização mais frequente possível? Se uma taxa nominal x é capitalizada k vezes ao ano, o fator anual é $\left(1+\frac{x}{k}\right)^k$. Tomando o logaritmo e usando $\ln(1+u) \approx u - \frac{u^2}{2}$:

$$\ln\left(1+\frac{x}{k}\right)^k = k\ln\left(1+\frac{x}{k}\right) \approx k\left(\frac{x}{k} - \frac{x^2}{2k^2}\right) = x - \frac{x^2}{2k}$$

À medida que $k \to \infty$, o termo $\frac{x^2}{2k}$ some e o logaritmo tende a x; portanto $\left(1+\frac{x}{k}\right)^k \to e^x$. Isso mostra duas coisas: a taxa efetiva **cresce** com k (o termo subtraído diminui) e tem um **teto** — a capitalização contínua, $e^x - 1$.

Exemplo resolvido — quanto a frequência importa
Uma taxa nominal de 12% a.a., conforme a frequência de capitalização:

Capitalização	k	Taxa efetiva
Anual	1	12,0000%
Semestral	2	12,3600%
Trimestral	4	12,5509%
Mensal	12	12,6825%
Diária	365	12,7475%
Contínua	∞	12,7497%

Passar de capitalização anual para mensal acrescenta 0,68 ponto percentual; de mensal para diária, só 0,07. Os ganhos são decrescentes, e o teto é 12,7497%.

Atenção — erro comum
"Juros de 12% ao ano" pode ter dois significados. Num contrato, "12% a.a." pode ser uma taxa **efetiva** (cobrada de fato no ano) ou **nominal** (taxa mensal proporcional de 1% a.m., que equivale a 12,68% a.a.). Quando o contrato diz "capitalizados mensalmente", ou quando a taxa mensal é multiplicada por 12 para ser apresentada como anual, trata-se de taxa nominal. Compare sempre pela **efetiva**. Exemplo extremo: o rotativo do cartão de crédito a 12% a.m. equivale a $1{,}12^{12} - 1 = 289{,}60\%$ a.a. — e não a "144% ao ano".

3.5. Juros Compostos com Períodos Fracionários

Em operações reais, o prazo n pode não ser inteiro. Existem duas convenções:

Convenção Linear (exponencial para parte inteira, linear para fração):

$$\text{FV} = \text{PV} \times (1+i)^{n_{\text{int}}} \times (1 + i \times n_{\text{frac}})$$

Convenção Exponencial (potência para o prazo total):

$$\text{FV} = \text{PV} \times (1+i)^{n}$$

```python
def fv_convencao(pv, i, n, convencao='exponencial'):
    n_int = int(n)
    n_frac = n - n_int
    if convencao == 'linear':
        return pv * (1 + i) ** n_int * (1 + i * n_frac)
    else:
        return pv * (1 + i) ** n

# R$1.000 a 2% a.m. por 6 meses e 15 dias (n = 6.5)
pv, i, n = 1000, 0.02, 6.5
print(f"Convenção exponencial: R${fv_convencao(pv, i, n, 'exponencial'):.2f}")
print(f"Convenção linear: R${fv_convencao(pv, i, n, 'linear'):.2f}")
```

```
Convenção exponencial: R$1137.37
Convenção linear: R$1137.42
```

A convenção linear produz montante **levemente maior** (a prova está no quadro "Aprofundando a teoria" logo abaixo). Na prática do mercado brasileiro de renda fixa predomina a convenção **exponencial** (taxa equivalente elevada ao prazo, em dias úteis/252 ou corridos/365); a linear aparece sobretudo em livros-texto e em cálculos simplificados.

Aprofundando a teoria — por que a convenção linear nunca perde
Seja f a fração do período ($0 < f < 1$) e $i > 0$. A função $g(f) = (1+i)^f$ é **convexa** em f e coincide com a reta $1 + if$ nos pontos $f = 0$ e $f = 1$ (onde ambas valem 1 e $1+i$). Uma função convexa fica **abaixo** da corda que une dois de seus pontos; logo $(1+i)^f \leq 1 + if$ para $0 \leq f \leq 1$ (essa é a desigualdade de Bernoulli). Por isso, na parte fracionária do prazo, a convenção linear dá um fator **maior ou igual** ao da exponencial.

Exemplo resolvido — a diferença em reais
R$ 100.000 a 2% a.m. por 6,5 meses: a convenção exponencial dá $100.000 \times 1{,}02^{6{,}5} = 113.736{,}83$ e a linear dá $100.000 \times 1{,}02^{6} \times (1 + 0{,}02 \times 0{,}5) = 113.742{,}40$. A diferença (R$ 5,57) é pequena para um prazo curto, mas cresce com o valor e com a taxa — e o contrato deve dizer qual convenção vale.

Em resumo

- $\text{FV} = \text{PV}(1+i)^n$ e $\text{PV} = \frac{\text{FV}}{(1+i)^n}$. O expoente conta quantas vezes o fator é aplicado.
- Taxas compostas **equivalentes**: $(1+i_{\text{maior}}) = (1+i_{\text{menor}})^k$. Taxas proporcionais **não** são equivalentes em juros compostos.
- Taxa nominal capitalizada k vezes: $i_{\text{ef}} = \left(1+\frac{i_{\text{nom}}}{k}\right)^k - 1$; quanto maior k, maior a efetiva, até o teto $e^{i_{\text{nom}}} - 1$.
- Regra dos 72: tempo de duplicação $\approx \frac{72}{i}\%$, boa entre 6% e 10%.
- Bases de tempo (252 dias úteis, 365 dias corridos, 360 comercial) devem estar **explícitas** — mudar a base muda o resultado.

Capítulo 4 — Séries de Pagamentos

4.1. Classificação das Séries

Uma **série de pagamentos** (ou anuidade) é uma sucessão de capitais disponíveis em diferentes datas. Classifica-se segundo vários critérios:

Critério	Tipos
Período	Periódicos (intervalos iguais) vs Não Periódicos
Valor	Constantes (uniformes) vs Variáveis
Início	Postecipados (1º pgto ao final do 1º período) vs Antecipados (1º pgto no início) vs Diferidos (carência)
Duração	Finitas (temporárias) vs Infinitas (perpétuas)
Forma de cálculo	Imediatas vs Diferidas

Intuição

Uma série de pagamentos é um **pacote de pagamentos únicos**. Cada um deles pode ser trazido a valor presente com $\frac{\text{FV}}{(1+i)^n}$ — e a série vale a **soma** desses valores presentes. As fórmulas deste capítulo são apenas atalhos algébricos para essa soma, que é uma progressão geométrica. Se alguma vez a fórmula parecer misteriosa, volte à soma "por extenso".

Três perguntas para classificar qualquer série (e escolher a fórmula):

1. **Quando cai o primeiro pagamento?** Um período depois da data zero (postecipada), na própria data zero (antecipada) ou mais tarde, após uma carência (diferida).
2. **Os valores são iguais?** Se sim, é uma série *uniforme* se não, some os valores presentes um a um.
3. **Quantos pagamentos?** Finito, ou infinito (perpetuidade).

Situação	Tipo de série
Financiamento "sem entrada", 1ª parcela em 30 dias	Postecipada
Financiamento "com entrada" de valor igual à parcela	Antecipada
Aluguel pago no início do mês	Antecipada
Carnê com 1ª parcela em 120 dias	Diferida (carência de 3 períodos)
Ação que paga dividendos indefinidamente	Perpétua

4.2. Série Uniforme Postecipada

A mais comum: pagamentos iguais ao **final** de cada período.

```
     PMT    PMT    PMT    PMT    PMT
      ↑      ↑      ↑      ↑      ↑
|-----|------|------|------|------|----->
0     1      2      3      4      5
↓
PV
```

Valor Presente:

Cada pagamento PMT é descontado a valor presente:

$$\text{PV} = \frac{\text{PMT}}{(1+i)^1} + \frac{\text{PMT}}{(1+i)^2} + \ldots + \frac{\text{PMT}}{(1+i)^n}$$

A soma da progressão geométrica resulta em:

$$\text{PV} = \text{PMT} \times \frac{(1+i)^n - 1}{i \times (1+i)^n}$$

Valor Futuro:

$$\text{FV} = \text{PMT} \times \frac{(1+i)^n - 1}{i}$$

Prestação (PMT):

$$\text{PMT} = \text{PV} \times \frac{i \times (1+i)^n}{(1+i)^n - 1}$$

O fator $\frac{i \times (1+i)^n}{(1+i)^n - 1}$ é chamado **Fator de Recuperação de Capital (FRC)**.

```python
def serie_postecipada(pv=None, pmt=None, fv=None, i=None, n=None):
    """
    Calcula séries uniformes postecipadas.
    """
    if pmt is None:
        if pv is not None:
            fator = (i * (1 + i) ** n) / ((1 + i) ** n - 1)
            return pv * fator
        else:
            fator = i / ((1 + i) ** n - 1)
            return fv * fator
    elif pv is None and fv is None:
        return None

# Exemplo: Financiar R$30.000 em 24x a 1,8% a.m.
pv, i, n = 30000, 0.018, 24
pmt = serie_postecipada(pv=pv, i=i, n=n)
total_pago = pmt * n
juros_pagos = total_pago - pv

print(f"Valor financiado: R${pv:,.2f}")
print(f"Taxa: {i*100:.1f}% a.m.")
print(f"Prazo: {n} meses")
print(f"Prestação: R${pmt:.2f}")
print(f"Total pago: R${total_pago:,.2f}")
print(f"Juros totais: R${juros_pagos:,.2f}")
print(f"Taxa efetiva total: {(total_pago/pv - 1)*100:.2f}%")
print(f"Custo mensal médio: R${juros_pagos/n:.2f} de juros por mês")
```

```
Valor financiado: R$30,000.00
Taxa: 1.8% a.m.
Prazo: 24 meses
Prestação: R$1550.43
Total pago: R$37,210.20
Juros totais: R$7,210.20
Taxa efetiva total: 24.03%
Custo mensal médio: R$300.43 de juros por mês
```

Aprofundando a teoria — dedução do valor presente da série postecipada
Seja $v = \frac{1}{1+i}$ o fator de desconto de um período. O valor presente da série é a soma dos n pagamentos descontados:

$$\text{PV} = \text{PMT} \cdot \left(v + v^2 + \cdots + v^n\right)$$

A expressão entre parênteses é uma PG de primeiro termo v e razão v. Pela fórmula da soma (Capítulo 0), $S = v\frac{1-v^n}{1-v}$. Como $1 - v = \frac{i}{1+i}$ e $v = \frac{1}{1+i}$, tem-se $\frac{v}{1-v} = \frac{1}{i}$, e então

$$\text{PV} = \text{PMT} \cdot \frac{1 - v^n}{i} = \text{PMT} \cdot \frac{1 - (1+i)^{-n}}{i}$$

Define-se o **fator de valor presente** da série como $a_{n\,|\,i} = \frac{1-(1+i)^{-n}}{i}$. Assim:

- $\text{PV} = \text{PMT} \cdot a_{n\,|\,i}$ e $\text{PMT} = \frac{\text{PV}}{a_{n\,|\,i}}$ (o inverso de $a_{n|i}$ é o FRC);
- $\text{FV} = \text{PMT} \cdot s_{n\,|\,i}$, com $s_{n\,|\,i} = \frac{(1+i)^n - 1}{i} = a_{n\,|\,i}(1+i)^n$.

A última igualdade diz que o valor futuro da série é o valor presente capitalizado por n períodos — como deveria ser.

Exemplo resolvido — somando por extenso (n pequeno)
Três pagamentos de R$ 100, ao fim de cada mês, a 10% a.m. Quanto valem hoje?

$$\text{PV} = \frac{100}{1{,}1} + \frac{100}{1{,}1^2} + \frac{100}{1{,}1^3} = 90{,}91 + 82{,}64 + 75{,}13 = 248{,}69$$

Pela fórmula: $100 \times \frac{1-1{,}1^{-3}}{0{,}1} = 100 \times 2{,}48685 = 248{,}69$ ✓. A fórmula só evita escrever a soma quando n é grande (por exemplo, 360 parcelas de um financiamento imobiliário).

Exemplo resolvido — valor futuro: quanto acumula quem poupa todo mês?
Depositar R$ 500 ao fim de cada mês, durante 10 anos (120 meses), a 0,6% a.m. Quanto se acumula?

$$\text{FV} = 500 \times \frac{1{,}006^{120} - 1}{0{,}006} = 500 \times 175{,}003 \approx \text{R\$ } 87.501{,}50$$

Dos R$ 87.501,50, apenas R$ 60.000 foram depositados; R$ 27.501,50 (31%) vêm dos rendimentos.

Exemplo resolvido — incógnita é o prazo
Uma dívida de R$ 20.000 é paga com parcelas mensais de R$ 800 a 2% a.m. Em quantos meses será quitada?

Isolando n em $\text{PV} = \text{PMT} \cdot \frac{1-(1+i)^{-n}}{i}$:

$$n = -\frac{\ln\left(1 - \frac{\text{PV} \cdot i}{\text{PMT}}\right)}{\ln(1+i)} = -\frac{\ln\left(1 - \frac{20.000 \cdot\, 0{,}02}{800}\right)}{\ln(1{,}02)} = -\frac{\ln(0{,}5)}{\ln(1{,}02)} \approx 35{,}0 \text{ meses}$$

Condição de existência: a expressão só tem solução se $\text{PMT} > \text{PV} \cdot i$, isto é, se a parcela for maior que os juros do primeiro mês (aqui, R$ 400). Com parcela de R$ 300 a dívida *nunca* seria quitada — ela cresceria.

```python
import math

def periodos_da_serie(valor_presente, parcela, taxa):
    """Número de parcelas necessário para quitar valor_presente (série postecipada)."""
    juros_do_mes = valor_presente * taxa
    if parcela <= juros_do_mes:
        return math.inf      # a parcela não cobre nem os juros: a dívida nunca acaba
    return -math.log(1 - juros_do_mes / parcela) / math.log(1 + taxa)

print(f"Parcela R$800: {periodos_da_serie(20000, 800, 0.02):.2f} meses")
print(f"Parcela R$401: {periodos_da_serie(20000, 401, 0.02):.1f} meses")
print(f"Parcela R$300: {periodos_da_serie(20000, 300, 0.02)} meses")
```

```
Parcela R$800: 35.00 meses
Parcela R$401: 302.7 meses
Parcela R$300: inf meses
```

Exemplo resolvido — incógnita é a taxa
Um carro de R$ 20.000 à vista é vendido em 24 parcelas de R$ 1.000. Qual é a taxa de juros?

Não existe fórmula fechada para i (a equação $20.000 = 1.000 \cdot a_{24\,|\,i}$ é polinomial de grau 24). Resolve-se por tentativas — exatamente o que a HP-12C e o Excel (`=TAXA`) fazem internamente. Em Python usamos a bissecção definida no Capítulo 1. A resposta é $i \approx 1{,}5131\%$ a.m. ($\approx 19{,}75\%$ a.a.).

```python
taxa_do_carro = bisseccao(lambda r: vp_das_parcelas(r, 1000, 24) - 20000, 1e-9, 1)
print(f"Taxa: {taxa_do_carro*100:.4f}% a.m. = {((1 + taxa_do_carro) ** 12 - 1)*100:.2f}%
a.a.")
```

```
Taxa: 1.5131% a.m. = 19.75% a.a.
```

4.3. Série Uniforme Antecipada

Pagamentos no **início** de cada período. A primeira parcela é paga no ato (entrada).

```
PMT     PMT     PMT     PMT     PMT
 ↑       ↑       ↑       ↑       ↑
|-------|-------|-------|-------|-------|----->
0       1       2       3       4       5
```

Valor Presente:

$$\text{PV}_{\text{ant}} = \text{PMT} \times \frac{(1+i)^n - 1}{i \times (1+i)^n} \times (1+i)$$

Ou alternativamente:

$$\text{PV}_{\text{ant}} = \text{PMT} + \text{PMT} \times \frac{(1+i)^{n-1} - 1}{i \times (1+i)^{n-1}}$$

```python
def serie_antecipada(pv, i, n):
    """Calcula prestação de série antecipada (com entrada)."""
    fator = ((1 + i) ** n - 1) / (i * (1 + i) ** n)
    return pv / (fator * (1 + i))

# Mesmo financiamento com entrada (série antecipada)
pv, i, n = 30000, 0.018, 24
pmt_ant = serie_antecipada(pv, i, n)
```

```
print(f"Série postecipada (sem entrada): R${pmt:.2f}")
print(f"Série antecipada (com entrada):   R${pmt_ant:.2f}")
print(f"Diferença: R${pmt - pmt_ant:.2f} por mês")
```

```
Série postecipada (sem entrada): R$1550.43
Série antecipada (com entrada):   R$1523.01
Diferença: R$27.41 por mês
```

Aprofundando a teoria — antecipada = postecipada capitalizada uma vez
Numa série antecipada, cada pagamento ocorre **um período mais cedo** que na postecipada. Pagar mais cedo custa mais: cada parcela vale $(1+i)$ vezes mais em valor presente. Logo

$$\text{PV}_{\text{ant}} = \text{PV}_{\text{post}} \cdot (1+i) \quad \Rightarrow \quad \text{PMT}_{\text{ant}} = \frac{\text{PMT}_{\text{post}}}{1+i}$$

Interpretação alternativa, muito útil em provas: uma série antecipada de n parcelas é **uma parcela paga hoje** mais uma série **postecipada de $n-1$ parcelas**: $\text{PV} = \text{PMT} + \text{PMT} \cdot a_{n-1 \mid i}$.

Atenção — erro comum
"Com entrada" nem sempre é série antecipada. A série antecipada exige que a entrada seja **igual às demais parcelas**. Se o vendedor pede, por exemplo, 30% de entrada e o restante em parcelas iguais postecipadas, a entrada é apenas um abatimento do valor financiado: calcula-se a série sobre o *saldo*, com as fórmulas postecipadas.

```
fin_valor, fin_taxa, fin_n = 30000, 0.018, 24
parcela_post = fin_valor / ((1 - (1 + fin_taxa) ** -fin_n) / fin_taxa)
parcela_ant = parcela_post / (1 + fin_taxa)
# conferência pela interpretação "1 parcela hoje + série postecipada de n-1":
fator_n_menos_1 = (1 - (1 + fin_taxa) ** -(fin_n - 1)) / fin_taxa
parcela_ant_alt = fin_valor / (1 + fator_n_menos_1)
print(f"Postecipada: R${parcela_post:.2f} | antecipada: R${parcela_ant:.2f} | alternativa:
R${parcela_ant_alt:.2f}")
```

```
Postecipada: R$1550.43 | antecipada: R$1523.01 | alternativa: R$1523.01
```

4.4. Série Diferida (com Carência)

Quando há um período de carência antes do início dos pagamentos.

$$\text{PV}_{\text{diferida}} = \frac{\text{PMT} \times \frac{(1+i)^n - 1}{i \times (1+i)^n}}{(1+i)^m}$$

Onde m = número de períodos de carência.

```
def serie_diferida(pv, i, n, carencia):
    """
    Calcula prestação com período de carência.
    carencia: número de períodos sem pagamento
    """
    fator_padrao = (i * (1 + i) ** n) / ((1 + i) ** n - 1)
    fator_carencia = (1 + i) ** carencia
    pmt_na_carencia = pv * fator_padrao * fator_carencia
    return pmt_na_carencia

# R$30.000, 1,8% a.m., 24 parcelas, carência de 3 meses
pmt_dif = serie_diferida(30000, 0.018, 24, 3)
```

```
print(f"Prestação com 3 meses de carência: R${pmt_dif:.2f}")
print(f"Prestação sem carência:             R${pmt:.2f}")
print(f"Diferença: R${pmt_dif - pmt:.2f}")
```

```
Prestação com 3 meses de carência: R$1635.66
Prestação sem carência:            R$1550.43
Diferença: R$85.24
```

Exemplo resolvido — pagar juros na carência ou capitalizá-los?
Voltando ao financiamento de R$ 30.000 a 1,8% a.m. em 24 parcelas, com carência de 3 meses. Há duas formas de tratar a carência:

- **Juros capitalizados** (padrão da fórmula acima): nada é pago nos 3 primeiros meses; os juros se acumulam ao saldo, que chega a $30.000 \times 1{,}018^3 = 31.649{,}33$ no início do pagamento. A parcela sobe para R$ 1.635,66 e o total pago é $24 \times 1.635{,}66 =$ R$ 39.255,94.
- **Juros pagos na carência:** o cliente paga só os juros mensais ($30.000 \times 1{,}8\% =$ R$ 540,00) nos 3 primeiros meses; o saldo continua R$ 30.000 e as 24 parcelas seguintes valem R$ 1.550,43. O total pago é $3 \times 540 + 24 \times 1.550{,}43 =$ R$ 38.830,20.

Pagar os juros na carência custa menos no total (R$ 425,74 a menos), porque evita que o juro vire "juros sobre juros". Em ambos os casos a taxa efetiva é a mesma, 1,8% a.m.: a diferença é só de **quando** o dinheiro sai.

```
saldo_apos_carencia = fin_valor * (1 + fin_taxa) ** 3
parcela_cap = parcela_post * (1 + fin_taxa) ** 3
total_cap = parcela_cap * fin_n
total_pago_juros = 3 * fin_valor * fin_taxa + parcela_post * fin_n
print(f"Saldo ao fim da carência: R${saldo_apos_carencia:,.2f}")
print(f"Juros capitalizados: parcela R${parcela_cap:,.2f} | total pago R${total_cap:,.2f}")
print(f"Juros pagos na carência: parcela R${parcela_post:,.2f} | total pago
R${total_pago_juros:,.2f}")
```

```
Saldo ao fim da carência: R$31,649.33
Juros capitalizados: parcela R$1,635.66 | total pago R$39,255.94
Juros pagos na carência: parcela R$1,550.43 | total pago R$38,830.20
```

4.5. Perpetuidades

Uma perpetuidade é uma série de pagamentos **infinitos** e iguais.

Valor Presente da Perpetuidade:

$$\text{PV} = \frac{\text{PMT}}{i}$$

```
def perpetuidade(pmt=None, i=None, pv=None):
    if pv is None:
        return pmt / i
    elif pmt is None:
        return pv * i
    elif i is None:
        return pmt / pv

# Quanto preciso ter para receber R$5.000/mês para sempre a 0,8% a.m.?
pv = perpetuidade(pmt=5000, i=0.008)
print(f"Valor necessário para perpetuidade: R${pv:,.2f}")
```

```python
# Se tenho R$1.000.000, quanto posso sacar por mês a 0,6% a.m.?
pmt = perpetuidade(pv=1000000, i=0.006)
print(f"Saque mensal perpétuo: R${pmt:.2f}")
```

```
Valor necessário para perpetuidade: R$625,000.00
Saque mensal perpétuo: R$6000.00
```

Aprofundando a teoria — a perpetuidade como limite
O valor presente de n pagamentos é $\text{PMT} \cdot \frac{1-(1+i)^{-n}}{i}$. Se o número de pagamentos cresce sem limite, $(1+i)^{-n} \to 0$ (porque $1+i > 1$) e o valor presente converge para

$$\text{PV}_\infty = \frac{\text{PMT}}{i}$$

Isso confirma uma intuição: se você tem PV aplicado a i por período, pode sacar os juros $\text{PV} \cdot i$ todo período *sem nunca tocar no principal* — e o saque é eterno. Perpetuidades puras são raras (o governo britânico emitiu "consols" no século XVIII), mas a fórmula é a base de avaliações de imóveis, ações e fundações.

Exemplo resolvido — quanto vale um imóvel alugado?
Um imóvel gera R$ 2.400 por mês de aluguel líquido. Se o investidor exige 0,6% a.m. de retorno, o valor do imóvel é $\frac{2.400}{0{,}006} = \text{R\$ } 400.000$. Se a taxa exigida sobe para 0,8% a.m., o valor cai para $\frac{2.400}{0{,}008} = \text{R\$ } 300.000$ — uma queda de 25%.

Este é o mecanismo do **risco de taxa de juros**: o preço de ativos de longa duração cai quando a taxa de desconto sobe. Reaparece no Capítulo 6 (títulos pré-fixados) e na apostila de Finanças.

Atenção — erro comum
A fórmula $\frac{\text{PMT}}{i}$ vale para pagamentos **postecipados**. Se o primeiro pagamento ocorre *hoje* (perpetuidade antecipada), some um pagamento: $\text{PV} = \text{PMT} + \frac{\text{PMT}}{i}$.

4.6. Perpetuidade com Crescimento (Modelo de Gordon)

Quando os pagamentos crescem a uma taxa constante g:

$$\text{PV} = \frac{\text{PMT}}{i - g}$$

Restrição: $i > g$ (senão o valor presente explode para infinito).

```python
def perpetuidade_crescimento(pmt, i, g):
    if i <= g:
        raise ValueError("i deve ser maior que g")
    return pmt / (i - g)

# Empresa pagará dividendo de R$2,00 este ano, crescendo 4% a.a., Ke=11% a.a.
pv = perpetuidade_crescimento(2, 0.11, 0.04)
print(f"Preço justo da ação (Gordon): R${pv:.2f}")

# Qual a taxa de crescimento implícita se a ação está a R$40 e próximo dividendo é R$3?
# PV = PMT / (i - g) -> g = i - PMT/PV
g = 0.11 - 3/40
print(f"Crescimento implícito no preço de R$40,00: {g*100:.2f}% a.a.")
```

```
Preço justo da ação (Gordon): R$28.57
Crescimento implícito no preço de R$40,00: 3.50% a.a.
```

Aprofundando a teoria — dedução do modelo de Gordon
Seja D_1 o **próximo** dividendo (pago em $t = 1$), que cresce à taxa g ao período. O valor presente de todos os dividendos é

$$P_0 = \frac{D_1}{1+i} + \frac{D_1(1+g)}{(1+i)^2} + \frac{D_1(1+g)^2}{(1+i)^3} + \cdots$$

É uma PG infinita de primeiro termo $a_1 = \frac{D_1}{1+i}$ e razão $q = \frac{1+g}{1+i}$. Se $g < i$ então $q < 1$ e a soma converge:

$$P_0 = \frac{a_1}{1-q} = \frac{\frac{D_1}{1+i}}{\frac{i-g}{1+i}} = \frac{D_1}{i-g}$$

Se $g \geq i$, a série diverge: nenhum preço finito remunera o fluxo.

Exemplo resolvido — sensibilidade do preço ao crescimento
Dividendo esperado $D_1 = \text{R\$ } 2{,}00$; taxa exigida $i = 11\%$ a.a.

Crescimento g	Preço justo	Variação vs. $g = 4\%$
0%	R$ 18,18	−36,4%
2%	R$ 22,22	−22,2%
4%	R$ 28,57	—
6%	R$ 40,00	+40,0%
8%	R$ 66,67	+133,3%

Uma variação de apenas 2 pontos percentuais em g muda o preço em 40% — e quando g se aproxima de i o preço "explode". Por isso o modelo de Gordon é **extremamente sensível** à premissa de crescimento, e o analista deve sempre testar cenários.

Atenção — erro comum
No modelo de Gordon, PMT (ou D_1) é o **próximo** pagamento, o do período 1. Se o enunciado fornece o dividendo *já pago* hoje (D_0), é preciso crescê-lo antes: $D_1 = D_0(1+g)$.

No mercado brasileiro
A fórmula $P_0 = \frac{D_1}{i-g}$ é o ponto de partida de valuation por dividendos descontados (apostila 07). Reordenando, a taxa de retorno exigida implícita no preço é $i = \frac{D_1}{P_0} + g$: o **dividend yield** mais o crescimento.

Em resumo
- Série postecipada: $\text{PV} = \text{PMT} \cdot a_{n|i}$, com $a_{n|i} = \frac{1-(1+i)^{-n}}{i}$; o VP fica **um período antes do 1º pagamento**.
- Antecipada: $\text{PV}_{\text{ant}} = \text{PV}_{\text{post}}(1+i)$. Diferida (juros capitalizados): $\text{PMT} = \text{PV}\ \frac{(1+i)^m}{a_{n|i}}$.
- Perpetuidade: $\text{PV} = \frac{\text{PMT}}{i}$. Com crescimento: $\text{PV} = \frac{\text{PMT}_1}{i-g}$, exigindo $i > g$.
- Quando a incógnita é i não há fórmula fechada: use métodos numéricos (HP-12C, `=TAXA`, bissecção).

Capítulo 5 — Sistemas de Amortização

5.1. Conceitos Gerais

Amortização é o processo de **devolução do capital emprestado** por meio de pagamentos periódicos. Cada prestação (PMT) é composta de duas partes:

$$\text{Prestação} = \text{Amortização} + \text{Juros}$$

$$\text{PMT}_k = A_k + J_k$$

Onde: - A_k = parcela que abate o saldo devedor (amortização do principal) - J_k = parcela que remunera o credor (juros sobre o saldo devedor)

Intuição
Todo mês o devedor paga uma prestação. Ela faz duas coisas, **nesta ordem**: primeiro quita os juros do mês (taxa × saldo devedor) e, com o que sobrar, **abate a dívida** (amortização). Os sistemas de amortização diferem apenas na *regra de divisão* entre juros e amortização — e, portanto, no formato da sequência de prestações.

Aprofundando a teoria — o saldo devedor é o valor presente das prestações que faltam
O empréstimo foi concedido por um valor PV exatamente igual ao valor presente de todas as prestações, à taxa i. Isso vale para qualquer sistema e para **qualquer data posterior** (método prospectivo): logo após o pagamento k, a dívida remanescente é o valor presente, à mesma taxa, das prestações que ainda faltam. No Price, com prestação constante,

$$\text{SD}_k = \text{PMT} \cdot a_{\overline{n-k}\,|\,i} = \text{PMT} \cdot \frac{1-(1+i)^{-(n-k)}}{i}$$

Esta fórmula é a ferramenta para calcular o **saldo para quitação antecipada** sem montar a tabela inteira.

No mercado brasileiro
Pelo Código de Defesa do Consumidor (art. 52, § 2º), o consumidor pode liquidar antecipadamente o débito, total ou parcialmente, com **redução proporcional dos juros**. Na prática, o valor da quitação é o saldo devedor da data, calculado pela fórmula acima: os juros dos meses futuros não são devidos.

Sistema	Prestação	Amortização	Juros	Uso típico
Price	Constante	Crescente	Decrescentes	Crédito direto, veículos
SAC	Decrescente	Constante	Decrescentes	Financiamento imobiliário
SAM	Média Price/SAC	Média Price/SAC	Decrescentes	Alguns financiamentos habitacionais
Americano	Só juros, até o fim	Tudo no final	Constantes	Debêntures, bonds

5.2. Sistema Price (Sistema Francês)

Características: - Prestações **constantes** (fixas) durante todo o período - Juros **decrescentes** com o tempo - Amortização **crescente** com o tempo - Amplamente usado em financiamento imobiliário, consórcios, crédito direto

Cálculo da prestação:

$$\text{PMT} = \text{PV} \times \frac{i \times (1+i)^n}{(1+i)^n - 1}$$

Evolução da dívida:

$$J_k = \text{SD}_{k-1} \times i$$

$$A_k = \text{PMT} - J_k$$

$$\text{SD}_k = \text{SD}_{k-1} - A_k$$

Onde SD_k é o saldo devedor após o pagamento da k-ésima prestação.

```python
def tabela_price(pv, i, n):
    """
    Gera a tabela completa de amortização pelo Sistema Price.
    Retorna lista de dicionários com período, prestação, juros, amortização, saldo.
    """
    pmt = pv * (i * (1 + i) ** n) / ((1 + i) ** n - 1)
    sd = pv
    tabela = []
    total_juros = 0
    for periodo in range(1, n + 1):
        juros = sd * i
        amortizacao = pmt - juros
        sd -= amortizacao
        total_juros += juros
        tabela.append({
            'periodo': periodo,
            'prestacao': round(pmt, 2),
            'juros': round(juros, 2),
            'amortizacao': round(amortizacao, 2),
            'saldo_devedor': round(abs(sd), 2) if sd < 0.01 else round(sd, 2)
        })
    return tabela, round(pmt * n - pv, 2)

# Exemplo: R$50.000 em 12x a 2% a.m.
pv, i, n = 50000, 0.02, 12
tabela, total_juros = tabela_price(pv, i, n)

print(f"PRICE — R${pv:,.2f} em {n}x a {i*100:.1f}% a.m.")
print(f"Prestação: R${tabela[0]['prestacao']:.2f}")
print(f"Total de juros: R${total_juros:,.2f}")
print()
print(f"{'Per':<5} {'Prestação':<12} {'Juros':<12} {'Amort':<12} {'Saldo Dev':<12}")
print("-" * 53)
for row in tabela:
    print(f"{row['periodo']:<5} R${row['prestacao']:<8.2f} R${row['juros']:<8.2f} "
          f"R${row['amortizacao']:<8.2f} R${row['saldo_devedor']:<8.2f}")
```

```
PRICE — R$50,000.00 em 12x a 2.0% a.m.
Prestação: R$4727.98
Total de juros: R$6,735.76

Per   Prestação    Juros        Amort        Saldo Dev
-----------------------------------------------------
1     R$4727.98  R$1000.00  R$3727.98  R$46272.02
2     R$4727.98  R$925.44   R$3802.54  R$42469.48
3     R$4727.98  R$849.39   R$3878.59  R$38590.89
4     R$4727.98  R$771.82   R$3956.16  R$34634.73
5     R$4727.98  R$692.69   R$4035.29  R$30599.44
6     R$4727.98  R$611.99   R$4115.99  R$26483.45
7     R$4727.98  R$529.67   R$4198.31  R$22285.14
8     R$4727.98  R$445.70   R$4282.28  R$18002.86
9     R$4727.98  R$360.06   R$4367.92  R$13634.94
10    R$4727.98  R$272.70   R$4455.28  R$9179.66
11    R$4727.98  R$183.59   R$4544.39  R$4635.27
12    R$4727.98  R$92.71    R$4635.27  R$0.00
```

Aprofundando a teoria — por que as amortizações do Price crescem em PG
Seja SD_{k-1} o saldo antes do pagamento k. Então $A_k = \mathrm{PMT} - i \cdot \mathrm{SD}_{k-1}$. Para o pagamento seguinte, $A_{k+1} = \mathrm{PMT} - i \cdot \mathrm{SD}_k$. Subtraindo,

$$A_{k+1} - A_k = -i(\mathrm{SD}_k - \mathrm{SD}_{k-1}) = -i \cdot (-A_k) = i \cdot A_k$$

pois o saldo cai exatamente a amortização. Logo $A_{k+1} = A_k(1+i)$: **cada amortização é a anterior multiplicada por $(1+i)$**. Com $A_1 = \mathrm{PMT} - \mathrm{PV} \cdot i$:

$$A_k = A_1(1+i)^{k-1}$$

Exemplo resolvido — as três primeiras linhas da tabela, na mão
R$ 50.000 em 12 parcelas a 2% a.m. (Price).

Prestação: $\mathrm{PMT} = 50.000 \times 0{,}02 \times \frac{1{,}02^{12}}{1{,}02^{12}-1} = 50.000 \times 0{,}094560 = 4.727{,}98$.

- **Mês 1:** juros $= 50.000 \times 0{,}02 = 1.000{,}00$; amortização $= 4.727{,}98 - 1.000{,}00 = 3.727{,}98$; saldo $= 50.000 - 3.727{,}98 = 46.272{,}02$.
- **Mês 2:** juros $= 46.272{,}02 \times 0{,}02 = 925{,}44$; amortização $= 3.802{,}54$; saldo $= 42.469{,}48$.
- **Mês 3:** juros $= 849{,}39$; amortização $= 3.878{,}59$; saldo $= 38.590{,}89$.

Conferências. (i) $A_2 = A_1 \times 1{,}02 = 3.727{,}98 \times 1{,}02 = 3.802{,}54$ ✓. (ii) A soma de todas as amortizações deve ser 50.000 e o saldo final, zero — veja a tabela completa na saída abaixo.

Atenção — erro comum
"Pagar prestação não é pagar dívida." No Price, a primeira prestação é composta quase só de juros, especialmente em prazos longos. Um financiamento de R$ 200.000 em 360 meses a 1% a.m. tem prestação perto de R$ 2.057, mas só cerca de R$ 57 amortizam o saldo no primeiro mês (o resto, R$ 2.000, são juros). Veja o cálculo no código.

```
fin_longo_pv, fin_longo_i, fin_longo_n = 200000, 0.01, 360
fin_longo_pmt = fin_longo_pv * fin_longo_i / (1 - (1 + fin_longo_i) ** -fin_longo_n)
juros_1 = fin_longo_pv * fin_longo_i
print(f"Prestação: R${fin_longo_pmt:,.2f}")
print(f"1º mês -> juros R${juros_1:,.2f} ({juros_1 / fin_longo_pmt:.1%}) | amortização
R${fin_longo_pmt - juros_1:,.2f}")
print(f"Total pago ao fim de 30 anos: R${fin_longo_pmt * fin_longo_n:,.2f} ({fin_longo_pmt
* fin_longo_n / fin_longo_pv:.2f}x o valor financiado)")
```

```
Prestação: R$2,057.23
1º mês -> juros R$2,000.00 (97.2%) | amortização R$57.23
Total pago ao fim de 30 anos: R$740,601.07 (3.70x o valor financiado)
```

5.3. SAC — Sistema de Amortizações Constantes

Características: - Amortização **constante** em todos os períodos - Prestações **decrescentes** (juros diminuem com o saldo) - Menor custo total (juros totais) que o Price para mesma taxa - Usado no SFH (Sistema Financeiro de Habitação) e financiamentos imobiliários

Cálculo da amortização:

$$A = \frac{\mathrm{PV}}{n}$$

Evolução da dívida:

$$J_k = \mathrm{SD}_{k-1} \times i$$

$$\text{PMT}_k = A + J_k$$

$$\text{SD}_k = \text{SD}_{k-1} - A$$

```python
def tabela_sac(pv, i, n):
    """
    Gera tabela completa de amortização pelo SAC.
    """
    amortizacao = pv / n
    sd = pv
    tabela = []
    total_juros = 0
    for periodo in range(1, n + 1):
        juros = sd * i
        pmt = amortizacao + juros
        sd -= amortizacao
        total_juros += juros
        tabela.append({
            'periodo': periodo,
            'prestacao': round(pmt, 2),
            'juros': round(juros, 2),
            'amortizacao': round(amortizacao, 2),
            'saldo_devedor': round(sd, 2)
        })
    return tabela, round(total_juros, 2)

# Mesmo exemplo: R$50.000 em 12x a 2% a.m.
tabela_s, tj_sac = tabela_sac(pv, i, n)

print(f"SAC — R${pv:,.2f} em {n}x a {i*100:.1f}% a.m.")
print(f"Amortização constante: R${pv/n:.2f}")
print(f"1ª prestação: R${tabela_s[0]['prestacao']:.2f}")
print(f"Última prestação: R${tabela_s[-1]['prestacao']:.2f}")
print(f"Total de juros: R${tj_sac:,.2f}")
print()
print(f"{'Per':<5} {'Prestação':<12} {'Juros':<12} {'Amort':<12} {'Saldo Dev':<12}")
print("-" * 53)
for row in tabela_s:
    print(f"{row['periodo']:<5} R${row['prestacao']:<8.2f} R${row['juros']:<8.2f} "
          f"R${row['amortizacao']:<8.2f} R${row['saldo_devedor']:<8.2f}")
```

```
SAC — R$50,000.00 em 12x a 2.0% a.m.
Amortização constante: R$4166.67
1ª prestação: R$5166.67
Última prestação: R$4250.00
Total de juros: R$6,500.00

Per   Prestação    Juros        Amort        Saldo Dev
-----------------------------------------------------
1     R$5166.67  R$1000.00  R$4166.67  R$45833.33
2     R$5083.33  R$916.67   R$4166.67  R$41666.67
3     R$5000.00  R$833.33   R$4166.67  R$37500.00
4     R$4916.67  R$750.00   R$4166.67  R$33333.33
5     R$4833.33  R$666.67   R$4166.67  R$29166.67
6     R$4750.00  R$583.33   R$4166.67  R$25000.00
7     R$4666.67  R$500.00   R$4166.67  R$20833.33
8     R$4583.33  R$416.67   R$4166.67  R$16666.67
9     R$4500.00  R$333.33   R$4166.67  R$12500.00
10    R$4416.67  R$250.00   R$4166.67  R$8333.33
```

```
11    R$4333.33  R$166.67   R$4166.67  R$4166.67
12    R$4250.00  R$83.33    R$4166.67  R$0.00
```

Aprofundando a teoria — fórmulas fechadas do SAC
Como a amortização é constante, $A = \frac{\text{PV}}{n}$ e o saldo após k pagamentos é $\text{SD}_k = \text{PV}\ \frac{n-k}{n}$. Portanto

$$J_k = i \cdot \text{SD}_{k-1} = i \cdot \text{PV} \cdot \frac{n-k+1}{n} \quad \text{e} \quad \text{PMT}_k = \frac{\text{PV}}{n} \cdot (1 + i(n-k+1))$$

As prestações formam uma **PA decrescente** de razão $-i \cdot \frac{\text{PV}}{n} = -iA$: cada prestação é menor que a anterior em iA (os juros do período encolhem porque o saldo encolheu A). O total de juros é a soma de uma PA:

$$\sum_{k=1}^{n} J_k = i \cdot A \cdot (n + (n-1) + \cdots + 1) = i \cdot A \cdot \frac{n(n+1)}{2} = i \cdot \text{PV} \cdot \frac{n+1}{2}$$

Exemplo resolvido — SAC na mão
Mesmos R$ 50.000, 12 parcelas, 2% a.m.

- Amortização constante: $A = \frac{50.000}{12} = 4.166{,}67$.
- 1ª prestação: $4.166{,}67 + 50.000 \times 0{,}02 = 5.166{,}67$.
- Decréscimo mensal: $iA = 0{,}02 \times 4.166{,}67 = 83{,}33$ (5.166,67 ⟶ 5.083,33 ⟶ 5.000,00 ⟶ ...).
- Última (12ª): $4.166{,}67 + 4.166{,}67 \times 0{,}02 = 4.250{,}00$.
- Total de juros: $0{,}02 \times 50.000 \times \frac{13}{2} = 6.500{,}00$.

Comparando com o Price (juros totais R$ 6.735,76): o SAC economiza R$ 235,76 de juros nesta operação.

5.4. Comparação Price vs SAC

```
def comparar_sistemas(pv, i, n):
    """
    Comparação completa entre Price e SAC.
    """
    t_price, j_price = tabela_price(pv, i, n)
    t_sac, j_sac = tabela_sac(pv, i, n)

    print(f"{'Indicador':<35} {'PRICE':<15} {'SAC':<15}")
    print("-" * 65)
    print(f"{'Valor financiado':<35} R${pv:<11,.2f} R${pv:<,.2f}")
    print(f"{'Prestação inicial':<35} R${t_price[0]['prestacao']:<10.2f} R${t_sac[0]
['prestacao']:<.2f}")
    print(f"{'Prestação final':<35} R${t_price[-1]['prestacao']:<10.2f} R${t_sac[-1]
['prestacao']:<.2f}")
    print(f"{'Total prestações':<35} R${pv + j_price:<10,.2f} R${pv + j_sac:<,.2f}")
    print(f"{'Juros totais':<35} R${j_price:<10,.2f} R${j_sac:<,.2f}")
    print(f"{'Diferença de juros':<35} {'---':<15} R${j_price - j_sac:<,.2f}")

    # Gráfico comparativo
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(range(1, n+1), [r['prestacao'] for r in t_price], 'b-o', label='Price')
    plt.plot(range(1, n+1), [r['prestacao'] for r in t_sac], 'r-s', label='SAC')
    plt.xlabel('Período'); plt.ylabel('Prestação (R$)')
    plt.title('Evolução das Prestações'); plt.legend(); plt.grid(alpha=0.3)

    plt.subplot(1, 2, 2)
    plt.plot(range(1, n+1), [r['saldo_devedor'] for r in t_price], 'b--', label='Price')
    plt.plot(range(1, n+1), [r['saldo_devedor'] for r in t_sac], 'r-', label='SAC')
```

```
    plt.xlabel('Período'); plt.ylabel('Saldo Devedor (R$)')
    plt.title('Evolução do Saldo Devedor'); plt.legend(); plt.grid(alpha=0.3)
    plt.tight_layout()
    plt.savefig("figuras06/price_vs_sac.png", dpi=170, bbox_inches="tight")
    plt.show()

comparar_sistemas(50000, 0.02, 12)
```

```
Indicador                          PRICE          SAC
-----------------------------------------------------------------
Valor financiado                   R$50,000.00   R$50,000.00
Prestação inicial                  R$4727.98    R$5166.67
Prestação final                    R$4727.98    R$4250.00
Total prestações                   R$56,735.76  R$56,500.00
Juros totais                       R$6,735.76   R$6,500.00
Diferença de juros                 ---             R$235.76
```

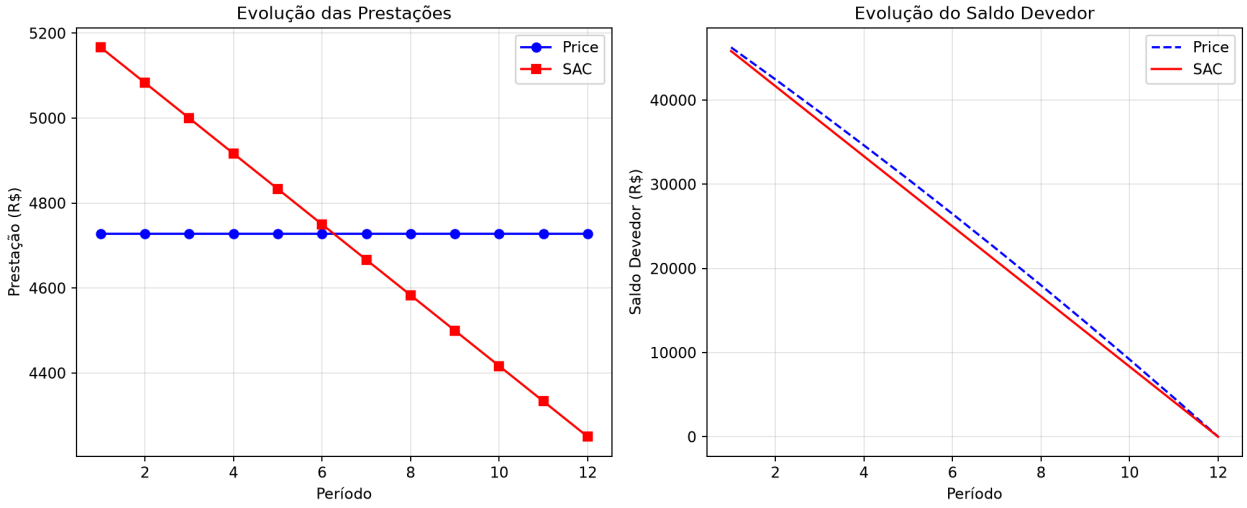


Figura 2: Price (azul) × SAC (vermelho) para R$ 50.000 em 12 parcelas a 2% a.m.: o SAC começa com prestação maior e reduz o saldo mais depressa.

Intuição
O SAC e o Price têm o **mesmo custo financeiro por real emprestado por mês de empréstimo**: a taxa i é a mesma, e o valor presente das prestações de ambos, descontado a i, é exatamente PV. Então por que o SAC paga menos juros no total? Porque ele **devolve o capital mais depressa**: o saldo devedor médio é menor, e os juros incidem sobre um saldo menor. Pagar menos juros no total, *por si só*, não é "ser mais barato" — você também pagou mais cedo.

```
def vp_dos_fluxos(taxa, prestacoes):
    return sum(p / (1 + taxa) ** (k + 1) for k, p in enumerate(prestacoes))

prest_price = [r["prestacao"] for r in tabela_price(50000, 0.02, 12)[0]]
prest_sac = [r["prestacao"] for r in tabela_sac(50000, 0.02, 12)[0]]
print(f"VP das prestações Price a 2%: R${vp_dos_fluxos(0.02, prest_price):,.2f}")
print(f"VP das prestações SAC   a 2%: R${vp_dos_fluxos(0.02, prest_sac):,.2f}")

cruz = next(k + 1 for k, (a_, b_) in enumerate(zip(prest_price, prest_sac)) if b_ < a_)
print(f"A prestação SAC fica menor que a do Price a partir da parcela {cruz}")
```

```
VP das prestações Price a 2%: R$50,000.00
VP das prestações SAC   a 2%: R$50,000.00
A prestação SAC fica menor que a do Price a partir da parcela 7
```

Exemplo resolvido — qual sistema escolher?
Os bancos aprovam o crédito com base na **renda comprometida**: a primeira prestação não pode passar de uma fração da renda (30% é uma regra comum). Para o financiamento acima, a renda mínima é $\frac{4.727{,}98}{0{,}30} =$ R$ 15.759,93 no Price e $\frac{5.166{,}67}{0{,}30} =$ R$ 17.222,23 no SAC. Resumo da decisão:

- **Renda apertada no começo, ou crescente:** Price (prestação inicial menor e previsível).
- **Foco em pagar menos juros / quitar rápido:** SAC (menor custo total, prestação cai com o tempo).
- Em ambos, **amortizações extras no início** reduzem muito mais os juros do que no fim.

5.5. SAM — Sistema de Amortização Mista

O SAM é a **média aritmética** entre Price e SAC:

$$\text{PMT}_k^{\text{SAM}} = \frac{\text{PMT}_k^{\text{Price}} + \text{PMT}_k^{\text{SAC}}}{2}$$

$$A_k^{\text{SAM}} = \frac{A_k^{\text{Price}} + A_k^{\text{SAC}}}{2}$$

Exemplo resolvido — SAM: a média dos dois
Para os R$ 50.000 em 12 parcelas a 2% a.m.: a primeira prestação SAM é $\frac{4.727{,}98 + 5.166{,}67}{2} = 4.947{,}32$; a sexta, $\frac{4.727{,}98 + 4.750{,}00}{2} = 4.738{,}99$; a última, $\frac{4.727{,}98 + 4.250{,}00}{2} = 4.488{,}99$. Como a média é linear, o total de juros do SAM também é a média: $\frac{6.735{,}76 + 6.500{,}00}{2} = 6.617{,}88$. A prestação inicial fica entre a do Price e a do SAC, e o sistema "suaviza" os dois extremos.

5.6. Sistema Americano

Características: - Pagamento **apenas dos juros** durante o período - Principal (amortização total) pago ao **final** do contrato - Usado em debêntures, títulos de dívida (bond), operações interbancárias

$$J_k = \text{PV} \times i \quad \text{(constante para todos os períodos)}$$

$$A_n = \text{PV} \quad \text{(apenas na última prestação)}$$

Exemplo resolvido — Americano com fundo de amortização
Uma empresa emite um título de R$ 100.000 por 5 anos, pagando 10% a.a. de juros anualmente e devolvendo o principal no fim.

- Desembolso anual de **juros**: R$ 10.000.
- Para ter R$ 100.000 no ano 5, a empresa abre um **fundo de amortização** (sinking fund) e deposita um valor D por ano, remunerado a 6% a.a.: $D = 100.000 \cdot \frac{0{,}06}{1{,}06^5 - 1} =$ R$ 17.739,64.
- Desembolso anual total: $10.000 + 17.739{,}64 =$ R$ 27.739,64.

Se o mesmo empréstimo fosse pago pelo Price a 10% a.a., a prestação seria $100.000 \cdot \frac{0{,}10}{1 - 1{,}1^{-5}} =$ R$ 26.379,75. O Americano com fundo custa **mais por ano** (R$ 1.359,89 de diferença) porque o fundo rende 6% enquanto a dívida custa 10%: a empresa "empresta barato e toma emprestado caro". Se o fundo rendesse 10%, os dois sistemas coincidiriam.

```
principal_amer, juros_amer, anos_amer, rend_fundo = 100000, 0.10, 5, 0.06
deposito = principal_amer * rend_fundo / ((1 + rend_fundo) ** anos_amer - 1)
price_equivalente = principal_amer * juros_amer / (1 - (1 + juros_amer) ** -anos_amer)
print(f"Depósito anual no fundo: R${deposito:,.2f}")
print(f"Desembolso anual (juros + depósito): R${principal_amer * juros_amer +
```

```
deposito:,.2f}")
print(f"Prestação anual do Price a 10%: R${price_equivalente:,.2f}")
```

```
Depósito anual no fundo: R$17,739.64
Desembolso anual (juros + depósito): R$27,739.64
Prestação anual do Price a 10%: R$26,379.75
```

5.7. Custo Efetivo Total (CET)

O CET vai além da taxa de juros, incluindo todos os encargos, tributos, tarifas e seguros:

$$\text{CET} = i_{\text{efetiva}} \quad \text{que iguala} \quad \text{PV}_{\text{recebido}} \quad \text{a todos os pagamentos futuros}$$

```
def calcular_cet(pv_recebido, pagamentos, periodos):
    """
    Calcula o CET (Custo Efetivo Total) usando método iterativo.
    pagamentos: lista de valores pagos (positivos)
    periodos: lista de períodos correspondentes
    """
    def cet_funcao(taxa):
        vp_pagamentos = sum(pmt / (1 + taxa) ** t for pmt, t in zip(pagamentos, periodos))
        return vp_pagamentos - pv_recebido

    # Método da bissecção para encontrar a taxa
    a, b = 0.0001, 1.0
    for _ in range(100):
        m = (a + b) / 2
        if cet_funcao(m) * cet_funcao(a) > 0:
            a = m
        else:
            b = m
    return (a + b) / 2

# Empréstimo de R$10.000 com 12 prestações de R$950 (já incluindo tarifas)
pv = 10000
pmt = 950
n = 12

# Simular CET
taxa_cet = calcular_cet(pv, [pmt] * n, list(range(1, n+1)))
print(f"CET mensal: {taxa_cet*100:.4f}%")
print(f"CET anual:  {((1 + taxa_cet)**12 - 1)*100:.2f}%")

# Mesmo contrato, mas o banco desconta R$150 (tarifa + IOF) do valor liberado
liberado = pv - 150
cet_liq = calcular_cet(liberado, [pmt] * n, list(range(1, n + 1)))
print(f"CET mensal se o cliente recebe só R${liberado:,.0f}: {cet_liq*100:.4f}%")
print(f"CET anual: {((1 + cet_liq)**12 - 1)*100:.2f}%")
```

```
CET mensal: 2.0757%
CET anual:  27.96%
CET mensal se o cliente recebe só R$9,850: 2.3232%
CET anual: 31.73%
```

Aprofundando a teoria — o que entra no CET

O CET é a **taxa interna de retorno do fluxo de caixa do cliente**: a taxa que iguala o valor realmente *recebido* (o crédito liberado, já líquido de tarifas e tributos cobrados na contratação) ao valor presente de todos os pagamentos futuros (prestações, seguros obrigatórios, tarifas recorrentes). Qualquer despesa

que o cliente **não teria** se não contratasse a operação deve entrar. A divulgação do CET, em % a.m. e % a.a., é exigida pela regulamentação bancária para operações de crédito a pessoas físicas (Resolução CMN 3.517/2007).

Exemplo resolvido — de 2,08% para 2,32% ao mês
Um empréstimo é anunciado como "R$ 10.000 em 12 parcelas de R$ 950".

- O fluxo do cliente é: $+10.000$ em $t = 0$ e -950 em $t = 1, \ldots, 12$. Resolvendo $10.000 = 950 \cdot a_{12|i}$, obtém-se **2,0757% a.m.** (27,96% a.a.).
- Se, porém, o banco desconta R$ 150 (tarifa de cadastro e IOF) do valor liberado, o cliente recebe só R$ 9.850 pelas mesmas 12 parcelas. O CET sobe para **2,3232% a.m.** (31,73% a.a.).

Os R$ 150 "pequenos" elevaram o custo anual em 3,8 pontos percentuais. Por isso a comparação entre ofertas deve ser feita pelo CET, e não pelo valor da parcela nem pela taxa de juros nominal.

Em resumo
- Prestação = juros do saldo + amortização. O saldo devedor é sempre o VP das prestações que faltam.
- **Price:** prestação constante; amortização cresce em PG de razão $(1 + i)$. **SAC:** amortização constante; prestações em PA decrescente; juros totais $= i \cdot \text{PV}\ \frac{n+1}{2}$.
- SAC paga menos juros no total, mas começa com prestação maior; o *custo por real emprestado* é igual nos dois (taxa i).
- **CET** = TIR do fluxo líquido do cliente; inclui tarifas, tributos e seguros. É a medida correta para comparar ofertas.

Capítulo 6 — Inflação e Correção Monetária

6.1. Conceito de Inflação

Inflação é o aumento persistente e generalizado dos preços em uma economia. Seu efeito imediato é a **redução do poder de compra** da moeda.

Índices de preços no Brasil: - **IPCA** (IBGE) — índice oficial, usado para meta de inflação - **INPC** (IBGE) — foco em famílias de baixa renda - **IGP-M** (FGV) — usado em aluguéis e contratos - **INCC** (FGV) — custo da construção civil

Aprofundando a teoria — causas e medição da inflação
A inflação é o aumento **persistente e generalizado** dos preços; um choque pontual (a gasolina sobe num mês) não é inflação, mas pode desencadeá-la. Os economistas costumam distinguir:

- **Inflação de demanda:** a procura agregada excede a capacidade de produção; o aquecimento da economia pressiona os preços.
- **Inflação de custos:** choques de oferta (câmbio, energia, safra) encarecem a produção e são repassados ao consumidor.
- **Inflação inercial e expectativas:** contratos indexados e expectativas elevadas fazem a inflação passada se repetir. É por isso que o regime de **metas** (o CMN fixa 3% a.a., com tolerância de ±1,5 ponto percentual) depende tanto da credibilidade do Banco Central.

Cada índice mede uma "cesta" diferente, e por isso os números divergem: o **IPCA** cobre famílias com renda de 1 a 40 salários mínimos; o **INPC**, de 1 a 5; o **IGP-M** combina preços no atacado (60%), no consumidor (30%) e na construção (10%) — por isso é muito mais volátil, sensível ao câmbio e às commodities.

Intuição
Inflação é um imposto invisível sobre quem guarda dinheiro: com inflação π, o mesmo valor nominal compra $\frac{1}{1+\pi}$ do que comprava antes. A 4,5% a.a., R$ 100 mantidos "debaixo do colchão" por 10 anos compram só $\frac{1}{1{,}045^{10}} = 64{,}4\%$ do que compram hoje. A taxa de juros nominal só gera *riqueza real* se superar a inflação.

6.2. Taxa Nominal vs Taxa Real (Equação de Fisher)

A relação entre taxa nominal (i_n), taxa real (i_r) e inflação (π):

$$(1 + i_n) = (1 + i_r) \times (1 + \pi)$$

Derivando:

$$i_r = \frac{1 + i_n}{1 + \pi} - 1$$

$$i_n = i_r + \pi + i_r \times \pi$$

Aproximação linear (válida para taxas baixas): $i_n \approx i_r + \pi$

```python
def fisher(i_nominal=None, i_real=None, inflacao=None):
    """
    Calcula qualquer termo faltante da equação de Fisher.
    (1 + i_n) = (1 + i_r) * (1 + pi)
    """
    if i_nominal is None:
        return (1 + i_real) * (1 + inflacao) - 1
    elif i_real is None:
        return (1 + i_nominal) / (1 + inflacao) - 1
    elif inflacao is None:
        return (1 + i_nominal) / (1 + i_real) - 1

# Exemplo: CDI rendeu 13,25% com IPCA de 4,50%
i_nom, pi = 0.1325, 0.045
i_real = fisher(i_nominal=i_nom, inflacao=pi)
aprox = i_nom - pi
print(f"Rentabilidade nominal: {i_nom*100:.2f}%")
print(f"Inflação (IPCA): {pi*100:.2f}%")
print(f"Rentabilidade real: {i_real*100:.2f}%")
print(f"Aproximação ({i_nom*100:.2f}% - {pi*100:.2f}%): {aprox*100:.2f}%")
print(f"Erro da aproximação: {(i_real - aprox)*100:.2f} p.p.")
```

```
Rentabilidade nominal: 13.25%
Inflação (IPCA): 4.50%
Rentabilidade real: 8.37%
Aproximação (13.25% - 4.50%): 8.75%
Erro da aproximação: -0.38 p.p.
```

Aprofundando a teoria — dedução da equação de Fisher
Considere R$ 1 aplicado a uma taxa nominal i_n. Daqui a um período:

- o investidor terá $(1 + i_n)$ reais;
- o preço da cesta de consumo, que custava P e é reajustado pela inflação, passa a valer $P(1 + \pi)$.

Antes, R$ 1 comprava $\frac{1}{P}$ unidades da cesta. Depois, $(1 + i_n)$ reais compram $\frac{1+i_n}{P(1+\pi)}$ unidades. O **ganho de poder de compra** é a razão entre as duas quantidades:

$$(1+i_r) = \frac{\frac{1+i_n}{P(1+\pi)}}{\frac{1}{P}} = \frac{1+i_n}{1+\pi}$$

que é a equação de Fisher. Duas leituras: (1) a taxa real é um **ganho em unidades de bens**, não em reais; (2) distinguem-se a taxa real **ex-post** (calculada com a inflação *realizada*) e a **ex-ante** (calculada com a inflação *esperada*, que é a base da decisão de investir).

Atenção — erro comum
"Juro real = nominal menos inflação" só vale para taxas baixas. A subtração ignora o termo cruzado $i_r \cdot \pi$. Com inflação de 4,5% o erro é de décimos de ponto percentual (veja a saída do código da seção); com inflação alta ele é grande. Com juros nominais de 30% a.a. e inflação de 20% a.a., a conta rápida dá 10%, mas a taxa real correta é $\frac{1{,}30}{1{,}20} - 1 = 8{,}33\%$.

Exemplo resolvido — a mesma regra, em outros sentidos
- **Tesouro IPCA+ a 5,5%:** o título paga IPCA mais 5,5% a.a. de taxa *real*. Com IPCA de 3,8%, o retorno nominal é $1{,}055 \times 1{,}038 - 1 = 9{,}51\%$ (e não 9,3%, a soma simples). Com IPCA de 5,0%, $1{,}055 \times 1{,}05 - 1 = 10{,}78\%$.
- **Poupança** a 6,1678% a.a. com IPCA de 4,44% a.a. (12 meses até julho de 2026, arquivo `ipca_12m.json`): taxa real $\frac{1{,}061678}{1{,}0444} - 1 = 1{,}65\%$ a.a. — positiva, mas modesta.
- **Taxa real negativa:** um investimento que rende 4% quando a inflação é 6% tem $\frac{1{,}04}{1{,}06} - 1 = -1{,}89\%$: perde poder de compra mesmo aumentando o saldo em reais.

```
print(f"{'Inflação':>9} {'Nominal':>8} {'Real (Fisher)':>14} {'Real (aprox.)':>14} {'Erro
(p.p.)':>12}")
for infl_ex, nominal_ex in [(0.03, 0.08), (0.045, 0.1325), (0.10, 0.20), (0.20, 0.30),
(0.50, 0.65)]:
    real_ex = fisher(i_nominal=nominal_ex, inflacao=infl_ex)
    aprox_ex = nominal_ex - infl_ex
    print(f"{infl_ex:>8.1%} {nominal_ex:>8.2%} {real_ex:>13.2%} {aprox_ex:>13.2%}
{(aprox_ex - real_ex)*100:>12.2f}")
```

```
 Inflação  Nominal  Real (Fisher)  Real (aprox.)  Erro (p.p.)
    3.0%    8.00%         4.85%         5.00%         0.15
    4.5%   13.25%         8.37%         8.75%         0.38
   10.0%   20.00%         9.09%        10.00%         0.91
   20.0%   30.00%         8.33%        10.00%         1.67
   50.0%   65.00%        10.00%        15.00%         5.00
```

6.3. Correção Monetária

Valor corrigido por um índice de preços:

$$V_{\text{corrigido}} = V_{\text{original}} \times \prod_{t=1}^{n}(1+\pi_t)$$

```
def corrigir_por_indice(valor_original, indices):
    """
    Corrige valor por uma série de índices de inflação.
    indices: lista de variações percentuais decimais
    """
    fator_acumulado = 1
    for idx, inf in enumerate(indices):
        fator_acumulado *= (1 + inf)
        print(f"  Mês {idx+1:2d}: inflação {inf*100:.2f}%, fator acumulado =
```

```
{fator_acumulado:.4f}")
    return valor_original * fator_acumulado

# Aluguel de R$2.000 corrigido por 6 meses de IPCA
valor = 2000
inflacoes_mensais = [0.0056, 0.0048, 0.0062, 0.0039, 0.0051, 0.0043]

print(f"Valor original: R${valor:,.2f}")
print("Correção:")
valor_corrigido = corrigir_por_indice(valor, inflacoes_mensais)
print(f"\nValor corrigido (6 meses): R${valor_corrigido:.2f}")
```

```
Valor original: R$2,000.00
Correção:
  Mês  1: inflação 0.56%, fator acumulado = 1.0056
  Mês  2: inflação 0.48%, fator acumulado = 1.0104
  Mês  3: inflação 0.62%, fator acumulado = 1.0167
  Mês  4: inflação 0.39%, fator acumulado = 1.0207
  Mês  5: inflação 0.51%, fator acumulado = 1.0259
  Mês  6: inflação 0.43%, fator acumulado = 1.0303

Valor corrigido (6 meses): R$2060.55
```

Exemplo resolvido — reajuste de aluguel e perda real de salário

- **Aluguel.** Um contrato de R$ 2.000 é reajustado pelo IPCA acumulado em 12 meses, 4,44% (dado do repositório). Novo aluguel: $2.000 \times 1{,}0444 = \text{R\$ } 2.088{,}80$.
- **Salário.** Para manter o poder de compra, um salário de R$ 3.000 precisaria subir para $3.000 \times 1{,}0444 = \text{R\$ } 3.133{,}20$. Se o reajuste concedido foi de 3%, o salário vai a R$ 3.090,00 e a **variação real** é $\frac{1{,}03}{1{,}0444} - 1 = -1{,}38\%$: o trabalhador perdeu poder de compra mesmo recebendo aumento.

Aprofundando a teoria — valores nominais, valores reais e o deflator

Para comparar valores em datas diferentes, é preciso expressá-los na mesma "moeda de poder de compra". Escolhe-se uma data-base e divide-se o valor nominal pelo **fator acumulado do índice** até aquela data:

$$\text{Valor real}_t = \frac{\text{Valor nominal}_t}{\text{Fator acumulado}_{0 \to t}}$$

O resultado está em **reais da data-base**. Dividir (e não multiplicar) é a operação correta: o dinheiro de hoje é "menos valioso" que o do passado, então o valor atual precisa ser reduzido para ser comparável. Séries econômicas (PIB, salário, renda) só têm sentido comparativo ao longo do tempo quando **deflacionadas**.

```
def deflacionar(valores_nominais, inflacoes_mensais):
    """Converte uma série nominal para reais do 1º mês (índices em decimal)."""
    fator, reais = 1.0, []
    for k, v in enumerate(valores_nominais):
        if k > 0:
            fator *= 1 + inflacoes_mensais[k - 1]
        reais.append(v / fator)
    return reais

salario_nominal = [3000, 3000, 3000, 3090, 3090, 3090]     # reajuste de 3% no 4º mês
inflacao_mensal = [0.004, 0.005, 0.004, 0.005, 0.004]       # IPCA mensal (ilustrativo)
for mes, (nom, real) in enumerate(zip(salario_nominal, deflacionar(salario_nominal,
inflacao_mensal)), 1):
    print(f"Mês {mes}: nominal R${nom:,.2f} | real (R$ do mês 1) R${real:,.2f}")
```

```
Mês 1: nominal R$3,000.00 | real (R$ do mês 1) R$3,000.00
Mês 2: nominal R$3,000.00 | real (R$ do mês 1) R$2,988.05
Mês 3: nominal R$3,000.00 | real (R$ do mês 1) R$2,973.18
Mês 4: nominal R$3,090.00 | real (R$ do mês 1) R$3,050.18
Mês 5: nominal R$3,090.00 | real (R$ do mês 1) R$3,035.00
Mês 6: nominal R$3,090.00 | real (R$ do mês 1) R$3,022.91
```

6.4. Taxa Pré-fixada vs Pós-fixada

Título Pré-fixado: a taxa é conhecida no momento da aplicação:

$$\text{FV} = \text{PV} \times (1 + i)^n$$

Título Pós-fixado: atrelado a um indexador + spread:

$$\text{FV} = \text{PV} \times (1 + \text{indexador} + \text{spread})^n$$

Mas o indexador é desconhecido a priori, então usa-se projeções.

```python
def comparar_prefixado_posfixado(pv, i_prefix, cdi_projetado, spread, n_dias):
    """
    Compara retorno de título pré e pós-fixado.
    """
    # Pré-fixado
    vf_pre = pv * (1 + i_prefix) ** (n_dias / 252)

    # Pós-fixado (CDI + spread)
    cdi_diario = (1 + cdi_projetado) ** (1/252) - 1
    spread_diario = (1 + spread) ** (1/252) - 1   # o spread é anual: converter p/ base
diária
    taxa_pos_diaria = (1 + cdi_diario) * (1 + spread_diario) - 1
    vf_pos = pv * (1 + taxa_pos_diaria) ** n_dias

    print(f"Pré-fixado ({i_prefix*100:.1f}% a.a.): R${vf_pre:,.2f}")
    print(f"Pós-fixado (CDI {cdi_projetado*100:.1f}% + spread {spread*100:.1f}%):
R${vf_pos:,.2f}")
    print(f"Diferença: R${vf_pos - vf_pre:,.2f}")

    if vf_pre > vf_pos:
        print("Pré-fixado vale mais a pena (cenário de queda de juros)")
    else:
        print("Pós-fixado vale mais a pena (cenário de alta de juros)")

comparar_prefixado_posfixado(10000, 0.12, 0.1325, 0.005, 252)
```

```
Pré-fixado (12.0% a.a.): R$11,200.00
Pós-fixado (CDI 13.2% + spread 0.5%): R$11,381.63
Diferença: R$181.63
Pós-fixado vale mais a pena (cenário de alta de juros)
```

Atenção — erro comum

A fórmula $\text{FV} = \text{PV}(1 + \text{indexador} + \text{spread})^n$ da seção 6.4 é uma **simplificação didática**. Um título pós-fixado de verdade rende o **produto dos fatores**: $\text{FV} = \text{PV} \cdot (1 + \text{indexador})^n \cdot (1 + \text{spread})^n$ (indexador e spread se compõem, não se somam). Além disso, "CDI + 0,5%" e "110% do CDI" são coisas diferentes: no primeiro, o spread é uma **taxa** somada ao CDI; no segundo, o percentual multiplica a **taxa diária** do CDI, dia a dia. Por isso, 110% de um CDI de 13,25% a.a. rende **14,67%** a.a. (e não $1{,}10 \times 13{,}25\% = 14{,}575\%$).

Aprofundando a teoria — a inflação implícita (break-even) e a escolha pré × IPCA+
Um investidor tem duas opções de mesmo prazo: um título **pré-fixado** a $i_{\text{pré}}$ e um título **IPCA+** que paga r real mais a inflação realizada. Os dois rendem o mesmo quando

$$(1+i_{\text{pré}}) = (1+r)(1+\pi_{\text{be}}) \quad \Rightarrow \quad \pi_{\text{be}} = \frac{1+i_{\text{pré}}}{1+r} - 1$$

π_{be} é a **inflação implícita** (break-even). Com pré de 12,5% a.a. e IPCA+ de 6,0% a.a., $\pi_{\text{be}} = \frac{1{,}125}{1{,}06} - 1 = 6{,}13\%$. Se o investidor *espera* inflação maior que 6,13% durante o prazo, o IPCA+ é melhor; se espera menos, o pré-fixado vence. Estamos, no fundo, comparando a expectativa do investidor com a expectativa embutida no preço do mercado.

Exemplo resolvido — o risco de marcação a mercado
O Tesouro Prefixado (LTN) paga R$ 1.000 no vencimento. Seu preço unitário (PU) é $\text{PU} = \frac{1.000}{(1+i)^{\frac{\text{du}}{252}}}$. Se a taxa de mercado sobe de 13% para 14% a.a.:

- Vencimento em 252 dias úteis (1 ano): PU passa de R$ 884,96 para R$ 877,19 (−0,88%).
- Vencimento em 504 dias úteis (2 anos): PU passa de R$ 783,15 para R$ 769,47 (−1,75%).

Quem carrega o título até o vencimento recebe os R$ 1.000 prometidos, mas quem precisa vendê-lo antes sofre a perda. Quanto **mais longo** o prazo, **maior** a sensibilidade do preço à taxa — o mesmo mecanismo da perpetuidade do Capítulo 4.

```python
def pu_ltn(taxa_aa, dias_uteis, face=1000):
    return face / (1 + taxa_aa) ** (dias_uteis / 252)

for dias_ex in (252, 504, 1260):
    pu_a, pu_b = pu_ltn(0.13, dias_ex), pu_ltn(0.14, dias_ex)
    print(f"{dias_ex:>4} du: PU a 13% = R${pu_a:7.2f} | a 14% = R${pu_b:7.2f} | variação =
{pu_b / pu_a - 1:+.2%}")
```

```
 252 du: PU a 13% = R$ 884.96 | a 14% = R$ 877.19 | variação = -0.88%
 504 du: PU a 13% = R$ 783.15 | a 14% = R$ 769.47 | variação = -1.75%
1260 du: PU a 13% = R$ 542.76 | a 14% = R$ 519.37 | variação = -4.31%
```

Em resumo
- $(1+i_n) = (1+i_r)(1+\pi)$: a taxa real é o ganho em poder de compra. Aproximar por $i_n - \pi$ só é aceitável com taxas baixas.
- Valores em datas diferentes só se comparam depois de **deflacionados** dividir pelo fator acumulado do índice.
- Pós-fixado: indexador e spread **se compõem** "% do CDI" multiplica a taxa **diária**. Pré-fixado: a taxa é conhecida, mas o preço de mercado oscila com os juros.
- Inflação implícita: $\pi_{\text{be}} = \frac{1+i_{\text{pré}}}{1+r} - 1$.

Capítulo 7 — Análise de Investimentos

7.1. Valor Presente Líquido (VPL / NPV)

O VPL é a soma de todos os fluxos de caixa descontados a valor presente pela taxa mínima de atratividade (TMA):

$$\text{VPL} = \sum_{t=0}^{n} \frac{\text{FC}_t}{(1+\text{TMA})^t}$$

Critério de decisão: - $\text{VPL} > 0 \longrightarrow$ investimento **viável** (cria valor) - $\text{VPL} = 0 \longrightarrow$ indiferente - $\text{VPL} < 0 \longrightarrow$ **inviável** (destrói valor)

```python
def vpl(taxa, fluxos):
    """
    Calcula o Valor Presente Líquido.
    fluxos[0] é o investimento inicial (negativo).
    """
    return sum(fl / (1 + taxa) ** t for t, fl in enumerate(fluxos))

# Projeto A: investir R$15.000, receber R$4.500/ano por 5 anos, TMA=10%
fluxos_a = [-15000, 4500, 4500, 4500, 4500, 4500]
tma = 0.10
vpl_a = vpl(tma, fluxos_a)

# Projeto B: investir R$12.000, receber R$3.800/ano por 5 anos
fluxos_b = [-12000, 3800, 3800, 3800, 3800, 3800]
vpl_b = vpl(tma, fluxos_b)

print(f"Projeto A: VPL = R${vpl_a:.2f} {'VIÁVEL' if vpl_a > 0 else 'INVIÁVEL'}")
print(f"Projeto B: VPL = R${vpl_b:.2f} {'VIÁVEL' if vpl_b > 0 else 'INVIÁVEL'}")
print(f"Melhor projeto: {'A' if vpl_a > vpl_b else 'B'}")
```

```
Projeto A: VPL = R$2058.54 VIÁVEL
Projeto B: VPL = R$2404.99 VIÁVEL
Melhor projeto: B
```

Aprofundando a teoria — por que o VPL é o critério central
Quatro razões fazem do VPL o critério preferido na teoria financeira:

1. **Mede riqueza criada, em reais de hoje.** Um VPL de R$ 2.000 significa que o projeto gera R$ 2.000 de valor *além* do que o capital renderia na melhor alternativa (a TMA).
2. **Considera todos os fluxos**, de todas as datas, sem ignorar nenhum.
3. **Considera o valor do dinheiro no tempo** à taxa de oportunidade.
4. **É aditivo:** $\text{VPL}(A + B) = \text{VPL}(A) + \text{VPL}(B)$. Isso permite avaliar projetos em pacote, ou isolar o efeito de uma decisão. A TIR *não* é aditiva.

A **TMA** não é um número arbitrário: é o **custo de oportunidade do capital** — o retorno que se obteria em outro uso de risco comparável. Para empresas, costuma-se usar o WACC (apostila 07); para pessoas físicas, a melhor alternativa de investimento livre de esforço (por exemplo, o Tesouro Selic ou o CDI).

Exemplo resolvido — VPL na mão: Projeto A
Investimento de R$ 15.000 e entradas de R$ 4.500 por 5 anos; TMA = 10% a.a.



Traga cada entrada a valor presente:

Ano	Cálculo	Valor presente
1	$\frac{4.500}{1{,}10}$	4.090,91
2	$\frac{4.500}{1{,}10^2}$	3.719,01
3	$\frac{4.500}{1{,}10^3}$	3.380,92
4	$\frac{4.500}{1{,}10^4}$	3.073,56

Ano	Cálculo	Valor presente
5	$\frac{4.500}{1{,}10^5}$	2.794,15
	Soma	**17.058,54**

$\text{VPL} = 17.058{,}54 - 15.000 =$ **R$ 2.058,54**. Atalho: como as entradas são uniformes, $\text{VP} = 4.500 \cdot a_{5|10\%} = 4.500 \times 3{,}79079 = 17.058{,}54$. O VPL positivo indica que o projeto **cria valor**.

Atenção — erro comum
"VPL positivo" não significa "melhor projeto" automaticamente. Para projetos *independentes*, aceitam-se todos com VPL > 0. Para projetos *mutuamente excludentes* (só um pode ser escolhido), escolhe-se o de **maior VPL**. Para projetos limitados por orçamento (racionamento de capital), veja o índice de lucratividade (seção 7.5).

7.2. Taxa Interna de Retorno (TIR / IRR)

A TIR é a taxa de desconto que **zera o VPL**:

$$0 = \sum_{t=0}^{n} \frac{\text{FC}_t}{(1 + \text{TIR})^t}$$

Critério de decisão: $\text{TIR} > \text{TMA} \longrightarrow$ viável

```
def tir(fluxos, precisao=1e-10, max_iter=10000):
    """
    Calcula a TIR pelo método de Newton-Raphson.
    """
    taxa = 0.1  # chute inicial
    for _ in range(max_iter):
        vp = sum(fl / (1 + taxa) ** t for t, fl in enumerate(fluxos))
        derivada = sum(-t * fl / (1 + taxa) ** (t + 1) for t, fl in enumerate(fluxos))
        if abs(derivada) < precisao:
            break
        taxa_nova = taxa - vp / derivada
        if abs(taxa_nova - taxa) < precisao:
            return taxa_nova
        taxa = taxa_nova
    return taxa

tir_a = tir(fluxos_a)
tir_b = tir(fluxos_b)

print(f"Projeto A: TIR = {tir_a*100:.2f}% a.a. (TMA={tma*100:.0f}%)")
print(f"Projeto B: TIR = {tir_b*100:.2f}% a.a. (TMA={tma*100:.0f}%)")
print(f"Decisão A: {'VIÁVEL' if tir_a > tma else 'INVIÁVEL'}")
print(f"Decisão B: {'VIÁVEL' if tir_b > tma else 'INVIÁVEL'}")
```

```
Projeto A: TIR = 15.24% a.a. (TMA=10%)
Projeto B: TIR = 17.57% a.a. (TMA=10%)
Decisão A: VIÁVEL
Decisão B: VIÁVEL
```

Intuição
A TIR é a **"taxa de juros própria" do projeto**: a rentabilidade anual média sobre o capital que permanece investido. Compare a TIR com o CET do Capítulo 5: o CET é, tecnicamente, a TIR do fluxo do tomador de

crédito. Um projeto com TIR de 15% rende como uma aplicação que pagasse 15% a.a. durante toda a vida do projeto.

Exemplo resolvido — achando a TIR por interpolação
Para o Projeto A, procuramos a taxa que zera $4.500 \cdot a_{5|i} - 15.000$. Tentativas:

- Com $i = 15\%$: VPL $= +84{,}70$ (ainda positivo: a TIR é maior que 15%).
- Com $i = 16\%$: VPL $= -265{,}68$ (negativo: a TIR é menor que 16%).

Interpolando linearmente: $\text{TIR} \approx 15\% + \frac{84{,}70}{84{,}70 + 265{,}68} \times 1\% = 15{,}24\%$. O resultado exato é **15,2382%**. Como $15{,}24\% > \text{TMA} = 10\%$, o projeto é viável — a mesma conclusão do VPL. Calculadoras e planilhas fazem exatamente esse refinamento por tentativas (Newton-Raphson ou bissecção), com muitas iterações.

Aprofundando a teoria — o método de Newton-Raphson
Para resolver $f(i) = \text{VPL}(i) = 0$, parte-se de um palpite i_0 e melhora-se a estimativa usando a **reta tangente** à curva do VPL:

$$i_{k+1} = i_k - \frac{f(i_k)}{f'(i_k)}$$

com $f'(i) = -\sum_{t=1}^{n} t \cdot \frac{\text{FC}_t}{(1+i)^{t+1}}$. A convergência é rápida (quadrática) quando o palpite é razoável. Para fluxos convencionais (uma só troca de sinal), a curva do VPL é **decrescente e convexa**, o que favorece a convergência (com o cuidado de não sair do domínio $i > -1$).

7.3. Limitações da TIR

1. **Múltiplas TIRs.** A equação que define a TIR é um polinômio de grau n na incógnita $1/(1+\text{TIR})$ — e um polinômio de grau n pode ter até n raízes reais. Pela **regra de sinais de Descartes**, o número de raízes positivas é, no máximo, igual ao número de **trocas de sinal** na sequência de fluxos de caixa $(\text{FC}_0, \text{FC}_1, \ldots, \text{FC}_n)$. Um fluxo convencional (negativo seguido só de positivos, como um investimento simples) troca de sinal **uma única vez** — garantindo, pela regra de Descartes, no máximo uma raiz positiva, e por isso uma TIR bem definida. Já um fluxo **não convencional**, que troca de sinal mais de uma vez (ex.: investimento com desembolso adicional no meio do projeto, como manutenção pesada ou descomissionamento), pode ter mais de uma raiz positiva — e nesse caso "a TIR" deixa de ser um número único bem definido, e o critério "TIR > TMA" perde sentido sem qualificação adicional.
2. **TIR vs VPL: por que o ranking pode divergir.** O VPL é uma função **decrescente** da taxa de desconto (fluxos futuros valem cada vez menos quanto maior a taxa usada para trazê-los a valor presente); dois projetos com perfis de fluxo diferentes (um mais concentrado no início, outro no fim; ou magnitudes muito diferentes) geram curvas de VPL que **cruzam** em algum ponto — a chamada *taxa de Fisher* do cruzamento. Abaixo dessa taxa, um projeto tem VPL maior; acima, o outro. A TIR de cada projeto (onde sua própria curva cruza o zero) não carrega informação sobre **onde esse cruzamento entre os dois projetos** acontece — por isso, ao comparar projetos **mutuamente excludentes** (só um pode ser escolhido), o critério correto é sempre o **maior VPL à TMA relevante**, nunca "a maior TIR": o projeto de maior TIR pode ter o menor VPL, se a TMA estiver do lado "errado" da taxa de Fisher.
3. **Taxa de reinvestimento implícita.** Ao descontar um fluxo futuro por $(1+\text{TIR})^t$, a fórmula da TIR presume implicitamente que qualquer fluxo intermediário recebido é **reinvestido à própria TIR** até o fim do projeto — um pressuposto que só é razoável se a TIR for uma taxa de mercado à qual de fato dá para reinvestir. Para uma TIR muito alta (comum em projetos de curta duração e alto retorno), supor reinvestimento a essa mesma taxa é otimista; o VPL, em contraste, já assume reinvestimento à TMA — taxa que o analista escolhe de forma realista (o custo de oportunidade de capital da empresa) — o que é outro motivo, além do ranking, para preferir o VPL como critério principal e tratar a TIR como um indicador complementar. A TIR Modificada (MTIR, abaixo) corrige exatamente essa premissa, separando a taxa de financiamento dos fluxos negativos da taxa de reinvestimento dos fluxos positivos.

```
# Exemplo de múltiplas TIRs: fluxo não convencional
fluxos_nao_convencionais = [-1000, 2500, -1500]
# Este fluxo pode ter 2 TIRs diferentes

# TIR Modificada (MTIR): assume reinvestimento à TMA
def mtir(fluxos, taxa_reinvestimento, taxa_financiamento=None):
    if taxa_financiamento is None:
        taxa_financiamento = taxa_reinvestimento
    vp_negativos = sum(min(fl, 0) / (1 + taxa_financiamento) ** t for t, fl in
enumerate(fluxos))
    vf_positivos = sum(max(fl, 0) * (1 + taxa_reinvestimento) ** (len(fluxos) - 1 - t)
                       for t, fl in enumerate(fluxos))
    return (vf_positivos / (-vp_negativos)) ** (1 / (len(fluxos) - 1)) - 1

mtir_a = mtir(fluxos_a, tma)
print(f"MTIR do Projeto A: {mtir_a*100:.2f}% a.a.")
```

```
MTIR do Projeto A: 12.87% a.a.
```

Exemplo resolvido — quando a TIR e o VPL discordam: escala
TMA de 10%. Dois projetos mutuamente excludentes:

- **C:** investe R$ 1.000 e recebe R$ 1.300 em um ano: TIR de 30%, VPL $= \frac{1.300}{1{,}1} - 1.000 =$ R$ 181,82.
- **D:** investe R$ 10.000 e recebe R$ 11.800 em um ano: TIR de 18%, VPL $= \frac{11.800}{1{,}1} - 10.000 =$ R$ 727,27.

A TIR prefere C; o VPL prefere D. Quem tem razão? O VPL: com R$ 10.000 disponíveis, escolher C deixa R$ 9.000 aplicados à TMA (VPL zero), e o total de valor criado é só R$ 181,82. A TIR é uma **taxa**, e ignora a **escala** do investimento.

Exemplo resolvido — TIR de dois valores: o fluxo não convencional
Fluxo $(-1.000, +2.500, -1.500)$: investe-se, recebe-se e, no último ano, há um desembolso (por exemplo, descomissionamento). Escreva a TIR com $x = \frac{1}{1+\text{TIR}}$: $-1.000 + 2.500x - 1.500x^2 = 0$, que simplifica para $3x^2 - 5x + 2 = 0$, com raízes $x = 1$ e $x = \frac{2}{3}$. Ou seja: **TIR = 0% e TIR = 50%** — duas taxas, e nenhuma delas diz se o projeto é bom. O VPL ao longo de várias taxas mostra o que acontece:

Taxa	0%	10%	20%	50%
VPL	0,00	33,06	41,67	0,00

Entre 0% e 50% o VPL é positivo; fora desse intervalo é negativo. A regra "TIR > TMA" perde sentido: só o VPL (a uma TMA bem definida) decide.

```
import numpy as np

fluxo_atipico = [-1000, 2500, -1500]
x_raizes = np.roots([-1500, 2500, -1000])          # em x = 1/(1+TIR)
print("TIRs encontradas:", sorted(f"{round((1 / x - 1) * 100, 1) + 0:.1f}%" for x in
x_raizes))
print(f"VPL a 10%: R${vpl(0.10, fluxo_atipico):.2f}")
print(f"MTIR de A reinvestindo a 10%: {mtir(fluxos_a, 0.10)*100:.2f}% | a 6%:
{mtir(fluxos_a, 0.06)*100:.2f}% | TIR: {tir_a*100:.2f}%")
```

```
TIRs encontradas: ['0.0%', '50.0%']
VPL a 10%: R$33.06
MTIR de A reinvestindo a 10%: 12.87% | a 6%: 11.08% | TIR: 15.24%
```

Exemplo resolvido — o cruzamento de Fisher: Projetos A e B

Ao variar a taxa de desconto, os VPLs dos Projetos A e B (seção 7.1) variam de modo diferente. Entre 0% e cerca de 5,4% o Projeto A tem VPL maior; a partir daí, o B. A taxa em que as curvas se cruzam é a **taxa de Fisher** (não confundir com a equação de Fisher do Capítulo 6): aqui, 5,37%. Como a TMA é 10% — acima do cruzamento —, o projeto B é o melhor. Observe também as TIRs: A, 15,24%; B, 17,57%.

```
taxas_perfil = np.linspace(0, 0.25, 200)
curva_a = [vpl(t, fluxos_a) for t in taxas_perfil]
curva_b = [vpl(t, fluxos_b) for t in taxas_perfil]
taxa_fisher = bisseccao(lambda r: vpl(r, fluxos_a) - vpl(r, fluxos_b), 1e-6, 0.5)

plt.figure(figsize=(9, 4.6))
plt.plot(taxas_perfil * 100, curva_a, label="Projeto A", color="#c0392b", linewidth=2)
plt.plot(taxas_perfil * 100, curva_b, label="Projeto B", color="#1d4f91", linewidth=2)
plt.axhline(0, color="gray", linewidth=0.8)
plt.axvline(taxa_fisher * 100, color="purple", linestyle="--", linewidth=1)
plt.annotate(f"Cruzamento (Fisher)\n{taxa_fisher*100:.2f}%", (taxa_fisher * 100,
vpl(taxa_fisher, fluxos_a)),
             textcoords="offset points", xytext=(12, 12), color="purple")
plt.scatter([tir_a * 100, tir_b * 100], [0, 0], color=["#c0392b", "#1d4f91"], zorder=5)
plt.annotate(f"TIR A = {tir_a*100:.2f}%", (tir_a * 100, 0), textcoords="offset points",
xytext=(-85, -26), color="#c0392b", arrowprops=dict(arrowstyle="-", color="#c0392b",
lw=0.8))
plt.annotate(f"TIR B = {tir_b*100:.2f}%", (tir_b * 100, 0), textcoords="offset points",
xytext=(14, 18), color="#1d4f91", arrowprops=dict(arrowstyle="-", color="#1d4f91", lw=0.8))
plt.axvline(10, color="gray", linestyle=":", linewidth=1)
plt.text(10.2, plt.ylim()[1] * 0.85, "TMA = 10%", color="gray")
plt.xlabel("Taxa de desconto (% a.a.)")
plt.ylabel("VPL (R$)")
plt.title("Perfil do VPL dos Projetos A e B")
plt.legend()
plt.grid(alpha=0.3)
plt.savefig("figuras06/perfil_vpl.png", dpi=170, bbox_inches="tight")
print(f"Taxa de Fisher (cruzamento): {taxa_fisher*100:.2f}%")
```

```
Taxa de Fisher (cruzamento): 5.37%
```

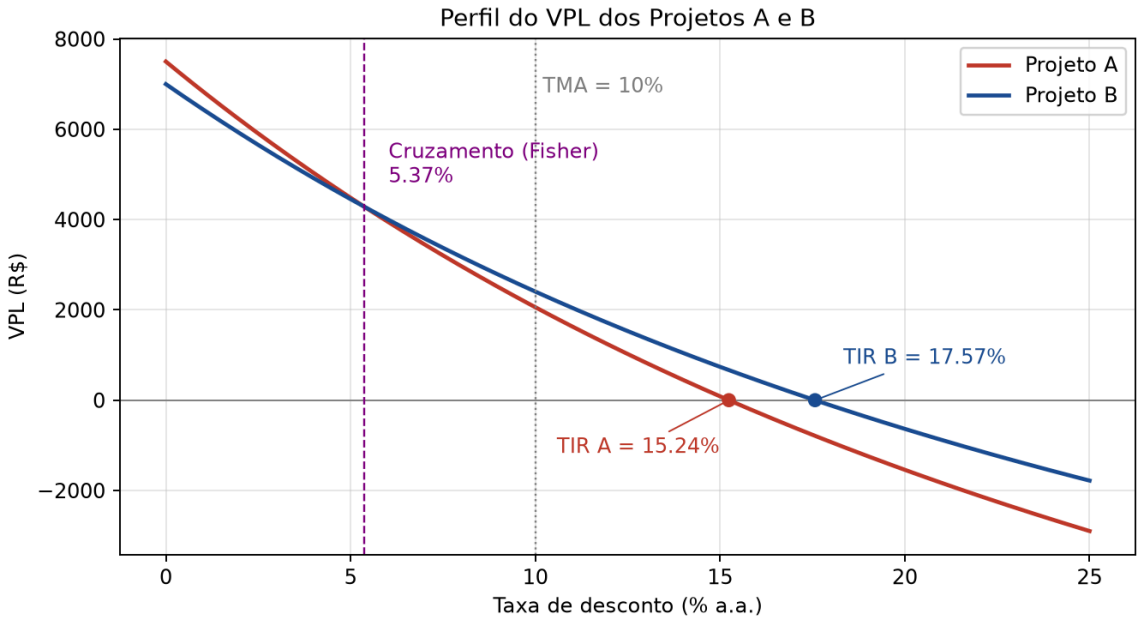


Figura 3: Perfil do VPL: cada curva cruza o eixo na sua TIR; as curvas se cruzam na taxa de Fisher. À esquerda do cruzamento, A é preferível; à direita, B.

7.4. Payback Simples e Descontado

Payback Simples: tempo necessário para recuperar o investimento **sem** considerar o custo do capital.

$$\text{Payback} = \text{menor} \;\; n \;\; \text{tal que} \sum_{t=0}^{n} \text{FC}_t \geq 0$$

Payback Descontado: considera o valor do dinheiro no tempo.

$$\text{Payback}_D = \text{menor} \;\; n \;\; \text{tal que} \sum_{t=0}^{n} \frac{\text{FC}_t}{(1+\text{TMA})^t} \geq 0$$

```
def payback(fluxos, taxa=None):
    """
    Calcula payback simples ou descontado.
    """
    acum = 0
    for t, fl in enumerate(fluxos):
        if taxa is None:
            fl_descontado = fl
        else:
            fl_descontado = fl / (1 + taxa) ** t
        acum += fl_descontado
        if acum >= 0 and t > 0:
            # Interpolação linear para fração do período
            if t > 0:
                acum_anterior = acum - fl_descontado
                fracao = -acum_anterior / fl_descontado if fl_descontado != 0 else 1
                return t - 1 + fracao
            return t
    return None

print(f"Payback simples (Proj A): {payback(fluxos_a):.2f} anos")
print(f"Payback descontado (Proj A): {payback(fluxos_a, 0.10):.2f} anos")
```

```
Payback simples (Proj A): 3.33 anos
Payback descontado (Proj A): 4.26 anos
```

Exemplo resolvido — payback na mão
Payback simples do Projeto A. Saldo acumulado dos fluxos:

Ano	Acumulado simples	Acumulado descontado a 10%
0	−15.000,00	−15.000,00
1	−10.500,00	−10.909,09
2	−6.000,00	−7.190,08
3	−1.500,00	−3.809,17
4	+3.000,00	−735,61
5	+7.500,00	+2.058,54

Simples: o saldo vira positivo durante o ano 4. No início do ano 4 faltavam R$ 1.500 e o fluxo do ano é R$ 4.500; logo payback $= 3 + \frac{1.500}{4.500} =$ **3,33 anos**. Descontado: o saldo só vira positivo no ano 5; faltavam R$ 735,61 e o fluxo descontado do ano é R$ 2.794,15, de modo que payback$_D = 4 + \frac{735,61}{2.794,15} =$ **4,26 anos**. O último valor acumulado descontado (+2.058,54) é o próprio VPL.

Atenção — erro comum
Limitações do payback. (1) Ignora tudo o que ocorre *depois* da recuperação. (2) O simples ignora o valor do dinheiro no tempo. (3) Não mede criação de valor. É um indicador de **liquidez e risco** (quanto tempo o capital fica exposto), não de rentabilidade. Veja dois projetos com o mesmo payback e destinos opostos:

```
projeto_e = [-1000, 600, 400, 100, 100]
projeto_f = [-1000, 500, 500, 5000, 5000]
for nome_p, fl_p in (("E", projeto_e), ("F", projeto_f)):
    print(f"Projeto {nome_p}: payback = {payback(fl_p):.2f} anos | VPL a 10% = R${vpl(0.10,
fl_p):,.2f}")
```

```
Projeto E: payback = 2.00 anos | VPL a 10% = R$19.47
Projeto F: payback = 2.00 anos | VPL a 10% = R$7,039.41
```

7.5. Índice de Lucratividade (IL)

Mede a relação benefício-custo:

$$\text{IL} = \frac{\sum_{t=1}^{n} \frac{\text{FC}_t}{(1+\text{TMA})^t}}{|\text{FC}_0|}$$

Critério: IL $> 1 \longrightarrow$ projeto viável

```
def indice_lucratividade(taxa, fluxos):
    vp_entradas = sum(fl / (1 + taxa) ** t for t, fl in enumerate(fluxos) if t > 0)
    investimento = abs(min(0, fluxos[0]))
    return vp_entradas / investimento

il_a = indice_lucratividade(0.10, fluxos_a)
il_b = indice_lucratividade(0.10, fluxos_b)
print(f"IL A: {il_a:.3f} {'VIÁVEL' if il_a > 1 else 'INVIÁVEL'}")
print(f"IL B: {il_b:.3f} {'VIÁVEL' if il_b > 1 else 'INVIÁVEL'}")
```

```
IL A: 1.137 VIÁVEL
IL B: 1.200 VIÁVEL
```

Aprofundando a teoria — o IL e o racionamento de capital
O índice de lucratividade se relaciona com o VPL por $\text{IL} = 1 + \frac{\text{VPL}}{|\text{FC}_0|}$. Por isso, o IL mede o **VPL por real investido**, e a regra "IL > 1" equivale a "VPL > 0".

Quando o orçamento é limitado (*racionamento de capital*), a seleção ótima maximiza o VPL **total** dentro do orçamento, e o IL ajuda a ordenar os projetos — embora, com projetos indivisíveis, seja preciso comparar as combinações possíveis.

Exemplo resolvido — IL de A e B e a escolha com orçamento limitado
$\text{IL}_A = \frac{17.058{,}54}{15.000} = 1{,}1372$; $\text{IL}_B = \frac{14.404{,}99}{12.000} = 1{,}2004$. O B rende mais VPL por real investido (R$ 0,20 contra R$ 0,14).

Se o orçamento fosse de R$ 15.000, só um dos dois caberia: A (VPL de R$ 2.058,54, usando todo o orçamento) ou B (VPL de R$ 2.404,99, e R$ 3.000 sobrando, que podem ser aplicados à TMA com VPL zero). A melhor escolha é o **B**. Com orçamento de R$ 27.000, os dois cabem, e a soma dos VPLs é R$ 4.463,53 (aditividade).

7.6. VPLA — Valor Presente Líquido Anualizado

Converte o VPL em uma série uniforme equivalente ao longo da vida do projeto:

$$\text{VPLA} = \text{VPL} \times \frac{i \times (1+i)^n}{(1+i)^n - 1}$$

Útil para comparar projetos com **diferentes prazos**.

```python
def vpla(taxa, fluxos, n):
    vpl_ = vpl(taxa, fluxos)
    fator = (taxa * (1 + taxa) ** n) / ((1 + taxa) ** n - 1)
    return vpl_ * fator

# Projetos com prazos diferentes
# Proj X: 3 anos, VPL = R$10.000
# Proj Y: 5 anos, VPL = R$14.000
taxa = 0.10
# Os VPLs de cada projeto já são conhecidos; basta anualizá-los:
vpl_x, n_x = 10000, 3
vpl_y, n_y = 14000, 5

vpla_x = vpl_x * (taxa * (1+taxa)**n_x) / ((1+taxa)**n_x - 1)
vpla_y = vpl_y * (taxa * (1+taxa)**n_y) / ((1+taxa)**n_y - 1)

print(f"Projeto X (3 anos): VPL = R${vpl_x:,.2f}, VPLA = R${vpla_x:.2f}/ano")
print(f"Projeto Y (5 anos): VPL = R${vpl_y:,.2f}, VPLA = R${vpla_y:.2f}/ano")
print(f"Melhor pelo VPLA: {'X' if vpla_x > vpla_y else 'Y'}")
```

```
Projeto X (3 anos): VPL = R$10,000.00, VPLA = R$4021.15/ano
Projeto Y (5 anos): VPL = R$14,000.00, VPLA = R$3693.16/ano
Melhor pelo VPLA: X
```

Aprofundando a teoria — o problema dos prazos diferentes
Comparar VPLs de projetos com vidas diferentes é injusto: um projeto de 5 anos "gasta" mais tempo do capital que um de 3 anos. A solução usual é supor que o projeto será **repetido indefinidamente** (cadeia de reposição). O VPL da cadeia infinita é

$$\text{VPL}_{\infty} = \text{VPL} \cdot \frac{1}{1-(1+i)^{-n}}$$

e o VPLA é simplesmente a prestação perpétua equivalente: $\text{VPLA} = \text{VPL}_{\infty} \cdot i$. Para os projetos X (3 anos, VPL de R$ 10.000) e Y (5 anos, VPL de R$ 14.000), a 10% a.a.: $\text{VPL}_{\infty(X)} = \text{R\$ } 40.211{,}48$ e $\text{VPL}_{\infty(Y)} = \text{R\$ } 36.931{,}65$. Apesar do VPL menor, o **X é melhor**: gera mais valor por unidade de tempo. A hipótese de repetição é forte — só faz sentido se for razoável replicar o projeto nas mesmas condições.

7.7. Análise de Sensibilidade

Testa como o VPL varia em função de mudanças nas variáveis-chave (receita, custo, taxa).

```
def analise_sensibilidade(fluxos_base, tma_base, variacao=0.3):
    """
    Analisa sensibilidade do VPL a variações nas principais variáveis.
    """
    importancias = {}
    variaveis = ['investimento', 'receita_anual', 'tma']

    print(f"{'Variável':<20} {'-30%':<12} {'-15%':<12} {'Base':<12} {'+15%':<12}
{'+30%':<12}")
    print("-" * 68)
    for var in variaveis:
        valores = []
        for delta in [-0.3, -0.15, 0, 0.15, 0.3]:
            fluxos_mod = fluxos_base.copy()
            tma_mod = tma_base
            if var == 'investimento':
                fluxos_mod[0] = fluxos_base[0] * (1 + delta)
            elif var == 'receita_anual':
                for t in range(1, len(fluxos_mod)):
                    fluxos_mod[t] = fluxos_base[t] * (1 + delta)
            elif var == 'tma':
                tma_mod = tma_base * (1 + delta)   # variação relativa da TMA
            vpl_ = vpl(tma_mod, fluxos_mod)
            valores.append(vpl_)
        print(f"{var:<20} " + " ".join(f"R${v:<8,.0f}" for v in valores))

analise_sensibilidade([-15000, 4500, 4500, 4500, 4500, 4500], 0.10)
```

```
Variável             -30%         -15%         Base         +15%         +30%
--------------------------------------------------------------------
investimento         R$6,559      R$4,309      R$2,059      R$-191       R$-2,441
receita_anual        R$-3,059     R$-500       R$2,059      R$4,617      R$7,176
tma                  R$3,451      R$2,733      R$2,059      R$1,424      R$828
```

Intuição

A análise de sensibilidade responde: **qual premissa, se estiver errada, mais prejudica a decisão?** Variam-se as hipóteses uma de cada vez e observa-se o efeito no VPL; as que movem mais o resultado merecem mais esforço de estimação (e mais cautela).

Exemplo resolvido — lendo a tabela de sensibilidade do Projeto A

Na tabela impressa acima (variações de ±15% e ±30%):

- **Investimento:** o VPL fica negativo se o investimento exceder 15.000 em mais de 13,7%: o limite é 17.058,54 (o VP das entradas).

- **Receita anual:** é a variável mais sensível. Uma queda de 15% já leva o VPL a −R$ 500; o **ponto de equilíbrio** está em $\frac{15.000}{3{,}79079} = \text{R\$ } 3.956{,}96$ por ano, 12,07% abaixo do previsto.
- **TMA:** a menos sensível dentro desse intervalo; mesmo com TMA 30% maior (13% a.a.), o VPL continua positivo (R$ 828).

```
cenarios_ex = {
    "Pessimista": dict(inv=16500, receita=3900, taxa=0.12),
    "Base":       dict(inv=15000, receita=4500, taxa=0.10),
    "Otimista":   dict(inv=13500, receita=5000, taxa=0.09),
}
for nome_c, c in cenarios_ex.items():
    fl_c = [-c["inv"]] + [c["receita"]] * 5
    print(f"{nome_c:<11} VPL = R${vpl(c['taxa'], fl_c):>9,.2f}")

fator_a5 = (1 - 1.10 ** -5) / 0.10
print(f"Receita mínima (VPL = 0): R${15000 / fator_a5:,.2f} ({15000 / fator_a5 / 4500 -
1:.2%})")
```

```
Pessimista  VPL = R$-2,441.37
Base        VPL = R$ 2,058.54
Otimista    VPL = R$ 5,948.26
Receita mínima (VPL = 0): R$3,956.96 (-12.07%)
```

No mercado brasileiro
Na prática profissional, a sensibilidade de uma variável por vez é só o começo. Os próximos passos são a **análise de cenários** (conjuntos coerentes de premissas, como no código acima), o **gráfico de tornado** (variáveis ordenadas pelo impacto no VPL) e a **simulação de Monte Carlo** (distribuições de probabilidade para as variáveis incertas, gerando uma distribuição de VPLs).

Em resumo
- $\text{VPL} = \sum \frac{\text{FC}_t}{(1+\text{TMA})^t}$: valor criado em reais de hoje; decisão: VPL > 0. É **aditivo** e é o critério soberano para projetos excludentes.
- TIR: taxa que zera o VPL. Calcula-se por tentativas. Problemas: múltiplas raízes em fluxos não convencionais; ignora escala; presume reinvestimento à própria TIR (a MTIR corrige).
- Payback: tempo de recuperação; mede liquidez, não valor. O descontado considera a TMA, e termina exatamente onde o VPL acumulado fica positivo.
- $\text{IL} = 1 + \frac{\text{VPL}}{|\text{FC}_0|}$: útil sob racionamento de capital.
- VPLA: compara projetos de prazos diferentes (hipótese de repetição).
- Sensibilidade e cenários mostram **quais premissas importam**.

Capítulo 8 — Tabela Resumo de Fórmulas

Conceito	Fórmula	Observação
Juros Simples	$J = \text{PV} \times i \times n$	Crescimento linear
Montante (Simples)	$\text{FV} = \text{PV} \times (1 + i \times n)$	PA
Juros Compostos	$J = \text{PV} \times [(1+i)^n - 1]$	Crescimento exponencial
Montante (Composto)	$\text{FV} = \text{PV} \times (1+i)^n$	PG
Valor Presente	$\text{PV} = \text{FV}/(1+i)^n$	Descapitalização
Taxa Equivalente (Comp)	$i_{\text{maior}} = (1 + i_{\text{menor}})^k - 1$	Não confundir com proporcional

Conceito	Fórmula	Observação
Nominal → Efetiva	$i_{\text{ef}} = (1 + i_{\text{nom}}/k)^k - 1$	k = capitalizações
Série Postecipada	$\text{PMT} = \text{PV} \times \frac{i(1+i)^n}{(1+i)^n - 1}$	Sem entrada
Série Antecipada	$\text{PV}_{\text{ant}} = \text{PV}_{\text{post}} \times (1+i)$	Com entrada
Perpetuidade	$\text{PV} = \text{PMT}/i$	Pagamentos infinitos
Gordon	$\text{PV} = \text{PMT}/(i - g)$	Com crescimento
Price	Prestações fixas	Juros decrescentes, amort. crescente
SAC	Amortização fixa	Prestações decrescentes
VPL	$\sum \text{FC}_t/(1+\text{TMA})^t$	> 0 = viável
TIR	$\sum \text{FC}_t/(1+\text{TIR})^t = 0$	$>$ TMA = viável
Payback	Tempo de recuperação	Sem considerar custo de capital
Fisher	$(1+i_n) = (1+i_r)(1+\pi)$	Relação nominal-real
IL	$\text{VP}_{\text{entradas}}/\lvert\text{investimento}\rvert$	> 1 = viável

Tabela 8.2 — Fórmulas dos sistemas de amortização e de análise de investimentos.

Conceito	Fórmula	Observação
Fator de valor presente da série	$a_{n\rceil i} = \frac{1-(1+i)^{-n}}{i}$	$\text{PV} = \text{PMT} \cdot a_{n\rceil i}$
Fator de valor futuro da série	$s_{n\rceil i} = \frac{(1+i)^n - 1}{i}$	$\text{FV} = \text{PMT} \cdot s_{n\rceil i}$
Saldo devedor (Price)	$\text{SD}_k = \text{PMT} \cdot a_{n-k\rceil i}$	Método prospectivo
Amortização (Price)	$A_k = A_1(1+i)^{k-1}$	$A_1 = \text{PMT} - \text{PV}\ i$
Prestação (SAC)	$\text{PMT}_k = \frac{\text{PV}}{n}(1 + i(n-k+1))$	PA decrescente
Total de juros (SAC)	$i \cdot \text{PV}\ \frac{n+1}{2}$	Soma de PA
Juros totais (Price)	$n \cdot \text{PMT} - \text{PV}$	Maior que o SAC
Fundo de amortização	$D = \text{PV} \cdot \frac{i_{\text{fundo}}}{(1+i_{\text{fundo}})^n - 1}$	Sistema Americano
Taxa de desconto → juros	$i = \frac{d}{1-dn}$	Desconto comercial simples
Inflação implícita	$\pi_{\text{be}} = \frac{1+i_{\text{pré}}}{1+r} - 1$	Pré × IPCA+
Índice de lucratividade	$\text{IL} = 1 + \frac{\text{VPL}}{\lvert\text{FC}_0\rvert}$	Racionamento de capital
VPLA	$\text{VPL} \cdot \frac{i}{1-(1+i)^{-n}}$	Prazos diferentes
Preço de Gordon	$P_0 = \frac{D_1}{i-g}$	Exige $i > g$

Capítulo 9 — Exercícios Resolvidos

E1. Um capital de R$ 8.000 foi aplicado a juros simples de 1,8% ao mês durante 15 meses. Calcule o montante e os juros.

$$\text{FV} = 8000 \times (1 + 0{,}018 \times 15) = 8000 \times 1{,}27 = 10.160$$

$$J = 10.160 - 8.000 = 2.160$$

```
pv, i, n = 8000, 0.018, 15
fv = pv * (1 + i * n)
print(f"Montante: R${fv:.2f}, Juros: R${fv - pv:.2f}")
```

```
Montante: R$10160.00, Juros: R$2160.00
```

E2. Qual o capital que, aplicado a 2,5% ao mês por 8 meses, rende juros compostos de R$ 2.500?

$$\text{PV} = \frac{J}{(1+i)^n - 1} = \frac{2500}{(1{,}025)^8 - 1}$$

```
j, i, n = 2500, 0.025, 8
pv = j / ((1 + i) ** n - 1)
print(f"Capital necessário: R${pv:.2f}")
```

```
Capital necessário: R$11446.73
```

E3. Uma loja vende um produto por R$ 1.200 à vista ou em 3 parcelas iguais sem entrada, a juros de 3% ao mês. Qual o valor de cada parcela?

$$\text{PMT} = 1200 \times \frac{0{,}03 \times (1{,}03)^3}{(1{,}03)^3 - 1}$$

```
pv, i, n = 1200, 0.03, 3
pmt = pv * (i * (1 + i) ** n) / ((1 + i) ** n - 1)
print(f"Valor de cada parcela: R${pmt:.2f}")
print(f"Total a prazo: R${pmt * n:.2f}")
print(f"Juros pagos: R${pmt * n - pv:.2f}")
```

```
Valor de cada parcela: R$424.24
Total a prazo: R$1272.71
Juros pagos: R$72.71
```

E4. Um título de R$ 10.000 com vencimento em 60 dias é descontado a uma taxa de desconto comercial de 3,5% ao mês. Qual o valor líquido?

$$\text{PV} = 10000 \times (1 - 0{,}035 \times 2)$$

```
fv, i, n = 10000, 0.035, 2
pv = fv * (1 - i * n)
print(f"Valor líquido: R${pv:.2f}")
```

```
Valor líquido: R$9300.00
```

E5. Um investimento de R$ 25.000 promete retornos de R$ 7.000/ano por 5 anos. TMA = 12% a.a. Calcule VPL, TIR e Payback.

```
fluxos = [-25000, 7000, 7000, 7000, 7000, 7000]
tma = 0.12

vpl_ = vpl(tma, fluxos)
tir_ = tir(fluxos)
pb = payback(fluxos)
pb_desc = payback(fluxos, tma)

print(f"VPL: R${vpl_:.2f}")
print(f"TIR: {tir_*100:.2f}% a.a.")
print(f"Payback simples: {pb:.2f} anos")
```

```
print(f"Payback descontado: {pb_desc:.2f} anos")
print(f"Decisão: {'VIÁVEL' if vpl_ > 0 and tir_ > tma else 'INVIÁVEL'}")
```

```
VPL: R$233.43
TIR: 12.38% a.a.
Payback simples: 3.57 anos
Payback descontado: 4.94 anos
Decisão: VIÁVEL
```

E6. Uma empresa tem WACC de 14%, capital próprio de R$ 80 milhões e dívida de R$ 40 milhões. O custo do capital próprio é 16% e o custo da dívida bruto é 10% (IR = 34%). Verifique se o WACC está correto.

$$\text{WACC} = \frac{80}{120} \times 0{,}16 + \frac{40}{120} \times 0{,}10 \times (1 - 0{,}34)$$

```
pl, divida = 80, 40
ke, kd, ir = 0.16, 0.10, 0.34
v = pl + divida
wacc_ = (pl/v) * ke + (divida/v) * kd * (1 - ir)
print(f"WACC: {wacc_*100:.2f}%")
```

```
WACC: 12.87%
```

E7. Um título pós-fixado rende 110% do CDI. Se o CDI está em 13,25% a.a. e o investimento é de R$ 5.000 por 252 dias, qual o valor de resgate?

```
pv, cdi, percentual = 5000, 0.1325, 1.10
cdi_diario = (1 + cdi) ** (1/252) - 1        # CDI diário (base 252 dias úteis)
fator_diario = 1 + cdi_diario * percentual   # 110% do CDI incide sobre a taxa DIÁRIA
vf = pv * fator_diario ** 252
taxa_efetiva = fator_diario ** 252 - 1
print(f"Taxa efetiva no período: {taxa_efetiva*100:.2f}% (e não {cdi*percentual*100:.2f}%,
que seria 1,10 x 13,25%)")
print(f"Valor de resgate: R${vf:.2f}")
```

```
Taxa efetiva no período: 14.67% (e não 14.58%, que seria 1,10 x 13,25%)
Valor de resgate: R$5733.38
```

E8. Um fundo de investimento rendeu 0,9%, 1,1% e −0,4% em três meses consecutivos. Calcule (a) a rentabilidade acumulada no trimestre, (b) a rentabilidade mensal média e (c) a rentabilidade anualizada, supondo que o ritmo médio se mantenha.

Solução.

- **(a) Acumulada:** taxas sucessivas **multiplicam-se**: $1{,}009 \times 1{,}011 \times 0{,}996 = 1{,}016019$, ou seja, **1,6019%** no trimestre.
- **(b) Média mensal:** a média correta é a **geométrica**: $1{,}016019^{1/3} - 1 = 0{,}5311\%$ a.m. A média aritmética, $\frac{0{,}9 + 1{,}1 - 0{,}4}{3} = 0{,}5333\%$, está **errada** para rentabilidades: ela superestima o ganho porque ignora o efeito de a perda de −0,4% ter incidido sobre um capital já maior.
- **(c) Anualizada:** $1{,}016019^{4} - 1 = 6{,}56\%$ a.a. (e não $12 \times 0{,}5311\% = 6{,}37\%$).

```
rent_mensais = [0.009, 0.011, -0.004]
fator_trim = 1.0
for r_m in rent_mensais:
    fator_trim *= 1 + r_m
media_geo = fator_trim ** (1 / len(rent_mensais)) - 1
print(f"Acumulada no trimestre: {(fator_trim - 1)*100:.4f}%")
print(f"Média geométrica mensal: {media_geo*100:.4f}% | média aritmética (errada):
{sum(rent_mensais)/3*100:.4f}%")
print(f"Anualizada: {((1 + media_geo) ** 12 - 1)*100:.2f}% a.a.")
```

```
Acumulada no trimestre: 1.6019%
Média geométrica mensal: 0.5311% | média aritmética (errada): 0.5333%
Anualizada: 6.56% a.a.
```

E9. Um eletrodoméstico é vendido em 10 parcelas mensais de R$ 480, sem entrada, com juros de 2,2% a.m. (a) Qual é o preço à vista implícito? (b) Quanto de juros está embutido? (c) E se a primeira parcela fosse paga no ato?

Solução.

1. **Tipo:** série postecipada. $\text{PV} = 480 \cdot a_{10|2{,}2\%} = 480 \times \frac{1-1{,}022^{-10}}{0{,}022} =$ **R$ 4.266,87**.
2. **Juros embutidos:** $10 \times 480 - 4.266{,}87 =$ **R$ 533,13** (cerca de 11% do valor à vista).
3. **Primeira parcela no ato:** série antecipada, $\text{PV}_{\text{ant}} = \text{PV} \cdot (1+i) = 4.266{,}87 \times 1{,}022 =$ **R$ 4.360,74**. O produto "vale mais" porque cada parcela foi paga um mês mais cedo.

```
parc_e9, taxa_e9, n_e9 = 480, 0.022, 10
vp_e9 = parc_e9 * (1 - (1 + taxa_e9) ** -n_e9) / taxa_e9
print(f"(a) Preço à vista implícito: R${vp_e9:,.2f}")
print(f"(b) Juros embutidos: R${parc_e9 * n_e9 - vp_e9:,.2f}")
print(f"(c) Valor presente se a 1ª parcela for no ato: R${vp_e9 * (1 + taxa_e9):,.2f}")
```

```
(a) Preço à vista implícito: R$4,266.87
(b) Juros embutidos: R$533.13
(c) Valor presente se a 1ª parcela for no ato: R$4,360.74
```

E10. Um financiamento de R$ 18.000 será pago em 10 parcelas pelo Sistema Price, a 1,6% a.m. Passados 4 pagamentos, o cliente quer quitar a dívida. (a) Qual o valor da prestação? (b) Qual o valor da quitação? (c) Quanto de juros ele economiza em relação a pagar as 6 parcelas restantes?

Solução.

1. $\text{PMT} = 18.000 \cdot \frac{0{,}016}{1-1{,}016^{-10}} =$ **R$ 1.962,17**.
2. Pelo método prospectivo (seção 5.1), o saldo após o 4º pagamento é o VP das 6 prestações restantes: $\text{SD}_4 = 1.962{,}17 \cdot a_{6|1{,}6\%} =$ **R$ 11.140,88**.
3. Pagando as 6 parcelas: $6 \times 1.962{,}17 = 11.773{,}02$. Economia: $11.773{,}02 - 11.140{,}88 =$ **R$ 632,14** — os juros dos meses futuros, que deixam de ser devidos.

```
parc_e10 = 18000 * 0.016 / (1 - 1.016 ** -10)
saldo_prosp = parc_e10 * (1 - 1.016 ** -6) / 0.016
print(f"(a) Prestação: R${parc_e10:,.2f}")
print(f"(b) Saldo para quitação após 4 pagamentos (prospectivo): R${saldo_prosp:,.2f}")
print(f"(c) Economia vs. pagar as 6 parcelas: R${6 * parc_e10 - saldo_prosp:,.2f}")
tab_e10, _ = tabela_price(18000, 0.016, 10)
print(f"Conferência pela tabela Price (saldo após a 4ª): R${tab_e10[3]
['saldo_devedor']:,.2f}")
```

```
(a) Prestação: R$1,962.17
(b) Saldo para quitação após 4 pagamentos (prospectivo): R$11,140.88
(c) Economia vs. pagar as 6 parcelas: R$632.14
Conferência pela tabela Price (saldo após a 4ª): R$11,140.88
```

E11. Um CDB prefixado rende 11% a.a. brutos e fica aplicado por 365 dias (alíquota de IR de 17,5%, conforme a tabela regressiva da seção 12.2). A inflação do período foi de 4,5%. Qual a rentabilidade **líquida real**?

Solução. Passos: (1) rendimento bruto $= 11\%$; (2) IR sobre o **rendimento**: $11\% \times 17{,}5\% = 1{,}925\%$; (3) rendimento líquido nominal $= 11\% - 1{,}925\% = 9{,}075\%$; (4) taxa real pela equação de Fisher, $\frac{1{,}09075}{1{,}045} - 1 =$ **4,38%** a.a.

> **Atenção — erro comum**
> O imposto incide sobre o **rendimento**, não sobre o valor resgatado: ao calcular o líquido, aplique a alíquota sobre o *ganho* (11%), e não sobre 1,11.

```python
bruto_e11, aliq_e11, infl_e11 = 0.11, 0.175, 0.045
liquido_nominal = bruto_e11 * (1 - aliq_e11)
liquido_real = (1 + liquido_nominal) / (1 + infl_e11) - 1
print(f"Líquido nominal: {liquido_nominal*100:.3f}% | líquido real: {liquido_real*100:.2f}%
a.a.")
```

```
Líquido nominal: 9.075% | líquido real: 4.38% a.a.
```

E12. Um projeto exige R$ 50.000 e promete fluxos de R$ 12.000, R$ 18.000, R$ 22.000 e R$ 25.000 nos quatro anos seguintes. TMA = 12% a.a. Calcule o VPL, a TIR e os paybacks, e conclua sobre a viabilidade.

Solução. O fluxo **não é uniforme**, portanto não há atalho com $a_{n|i}$: desconta-se ano a ano. VPL $= -50.000 + \frac{12.000}{1{,}12} + \frac{18.000}{1{,}12^2} + \frac{22.000}{1{,}12^3} + \frac{25.000}{1{,}12^4}$. A TIR (por tentativas ou pela função `tir`) e os paybacks saem do código:

```python
fluxos_e12 = [-50000, 12000, 18000, 22000, 25000]
print(f"VPL a 12%: R${vpl(0.12, fluxos_e12):,.2f}")
print(f"TIR: {tir(fluxos_e12)*100:.2f}% a.a.")
print(f"Payback simples: {payback(fluxos_e12):.2f} anos | descontado: {payback(fluxos_e12,
0.12):.2f} anos")
```

```
VPL a 12%: R$6,610.89
TIR: 17.43% a.a.
Payback simples: 2.91 anos | descontado: 3.58 anos
```

Conclusão. VPL positivo e TIR (17,43%) acima da TMA (12%): o projeto é **viável**. O payback simples (2,91 anos) é menor que o descontado (3,58 anos), como deve ser: descontar os fluxos adia a recuperação. Como o horizonte é de 4 anos, o investimento é recuperado (já em termos de valor presente) com folga de cerca de 5 meses.

E13. Uma fundação quer pagar uma bolsa de R$ 60.000 por ano, **para sempre**, com recursos aplicados a 7% a.a. (a) Qual capital inicial, se a primeira bolsa for paga daqui a um ano? (b) E se o primeiro pagamento for hoje? (c) E se a bolsa crescer 3% a.a., com a primeira em um ano?

Solução. (a) Perpetuidade postecipada: $\frac{60.000}{0{,}07} =$ **R$ 857.142,86**. (b) Antecipada: soma-se o pagamento de hoje: $60.000 + 857.142{,}86 =$ **R$ 917.142,86**. (c) Perpetuidade crescente (Gordon): $\frac{60.000}{0{,}07 - 0{,}03} =$ **R$ 1.500.000,00**. Repare que um crescimento de 3% quase dobra o capital exigido: a "proteção contra a inflação" é cara.

```python
bolsa, taxa_f, cresc_f = 60000, 0.07, 0.03
print(f"(a) Primeira bolsa em 1 ano: R${bolsa / taxa_f:,.2f}")
print(f"(b) Primeira bolsa hoje    : R${bolsa + bolsa / taxa_f:,.2f}")
print(f"(c) Bolsa crescendo {cresc_f:.0%}   : R${bolsa / (taxa_f - cresc_f):,.2f}")
```

```
(a) Primeira bolsa em 1 ano: R$857,142.86
(b) Primeira bolsa hoje    : R$917,142.86
(c) Bolsa crescendo 3%   : R$1,500,000.00
```

Capítulo 10 — Exercícios para Executar (na mão)

Estes exercícios não exigem computador: o objetivo é treinar o raciocínio e a manipulação das fórmulas antes de automatizá-las. Resolva no papel (ou calculadora simples), mostrando as etapas do cálculo. **As soluções não estão neste documento** — quando terminar, peça para eu conferir suas respostas.

Exercício 1 — Juros simples na mão
Um capital de R$ 3.600,00 foi aplicado a juros simples de 2,2% ao mês durante 7 meses. Calcule, mostrando cada etapa: (a) os juros totais; (b) o montante final; (c) o montante que seria obtido se a taxa fosse proporcional de 26,4% ao ano aplicada pelo mesmo prazo (converta o prazo para anos antes de aplicar a fórmula).

Exercício 2 — Desconto comercial vs. desconto racional
Um título de valor nominal R$ 12.000,00, com vencimento em 3 meses, é descontado a uma taxa de 4% ao mês. Calcule à mão o valor líquido recebido (a) pelo desconto comercial (por fora) e (b) pelo desconto racional (por dentro). Diga qual dos dois é mais vantajoso para quem antecipa o recebimento e explique por quê, usando a relação entre D_c e D_r do Capítulo 2.

Exercício 3 — Juros compostos mês a mês
Calcule, sem usar código, o valor futuro de um capital de R$ 2.000,00 aplicado a uma taxa de 3% ao mês, por 4 meses, usando $\mathrm{FV} = \mathrm{PV} \times (1+i)^n$. Monte uma pequena tabela mostrando o saldo ao final de cada um dos 4 meses (mês 1, 2, 3 e 4), e confira que o saldo do mês 4 bate com o valor obtido pela fórmula fechada.

Exercício 4 — Taxa nominal para taxa efetiva
Um banco anuncia uma taxa nominal de 18% ao ano, capitalizada mensalmente. Usando a fórmula do Capítulo 3 ($i_{\text{ef}} = (1 + i_{\text{nom}}/k)^k - 1$), calcule à mão a taxa efetiva anual. Em seguida, repita o cálculo supondo capitalização trimestral ($k = 4$) e compare os dois resultados, explicando por que a taxa efetiva aumenta quando o número de capitalizações por ano aumenta.

Exercício 5 — Prestação de uma série uniforme
Um consórcio de R$ 18.000,00 será pago em 6 parcelas mensais iguais, postecipadas, a uma taxa de 2,5% ao mês. Usando o Fator de Recuperação de Capital (FRC) do Capítulo 4, calcule à mão o valor de cada parcela e o total de juros pagos ao final do contrato.

Exercício 6 — Decisão entre Price e SAC
Uma financiadora oferece dois planos para um empréstimo de R$ 24.000,00 em 4 parcelas mensais a 3% ao mês: Plano A (Sistema Price) e Plano B (SAC). Calcule à mão, para cada plano, apenas a **primeira** e a **última** prestação (não é necessário montar a tabela completa dos 4 períodos — mas mostre o cálculo de cada uma das quatro prestações de cada plano). Depois responda: qual plano tem prestação inicial menor? Qual tem o menor total de juros pagos ao final? Justifique com base nas características descritas no Capítulo 5.

Exercício 7 — Taxa real pela equação de Fisher
Uma aplicação rendeu 9,80% ao ano (taxa nominal) em um período no qual o IPCA acumulado foi de 4,20%. Calcule à mão a taxa real de retorno usando a equação de Fisher completa (Capítulo 6), e compare com o resul-

tado que seria obtido pela aproximação linear $i_r \approx i_n - \pi$. Qual foi o erro (em pontos percentuais) cometido pela aproximação?

Exercício 8 — Interpretando VPL e TIR já calculados
Um projeto de investimento tem TMA de 11% ao ano. Foram calculados os seguintes resultados para dois projetos concorrentes:

Indicador	Projeto M	Projeto N
Investimento inicial	R$ 40.000	R$ 40.000
VPL	R$ 3.150,00	–R$ 890,00
TIR	13,4% a.a.	10,2% a.a.
Payback simples	3,8 anos	3,2 anos

Sem recalcular nada, apenas interpretando os números da tabela: (a) qual projeto é viável, segundo o critério do VPL? (b) o critério da TIR concorda com o do VPL? (c) o Projeto N tem payback mais curto — isso é suficiente para recomendá-lo? Justifique com base nas limitações de cada indicador discutidas no Capítulo 7.

Capítulo 11 — Exercícios para Executar (em código)

Implemente e execute cada um no seu editor. Vários exercícios pedem para reaproveitar funções definidas em capítulos anteriores (`juros_simples`, `serie_postecipada`, `tabela_price`, `tabela_sac`, `fisher`, `vpl`, `tir`, `payback`) — copie-as para o seu arquivo antes de começar. **As soluções não estão neste documento** — o objetivo é você rodar de verdade e ver o resultado; quando terminar, peça para eu revisar seu código.

Exercício 1 — Juros simples multiuso
Reaproveite (ou reescreva) a função `juros_simples(pv, fv, i, n)` do Capítulo 2. Use-a para resolver, sem alterar sua assinatura: (a) o montante de R$ 4.500 a 1,5% a.m. por 10 meses; (b) a taxa necessária para que R$ 6.000 vire R$ 7.800 em 12 meses; (c) o capital necessário para chegar a R$ 20.000 em 8 meses a 2% a.m.

Exercício 2 — Ponto de cruzamento entre regimes
Escreva uma função `mes_do_cruzamento(pv, i, n_max=60)` que, para um mesmo capital e mesma taxa, calcula mês a mês o montante em juros simples e em juros compostos e retorna o primeiro período em que o montante composto ultrapassa o montante simples. Teste com `pv=1000`, `i=0.02`.

Exercício 3 — Taxas equivalentes compostas
Crie uma função `taxa_equivalente(i_menor, k)` que recebe uma taxa referente a um período menor e o número de períodos menores contidos no período maior, retornando a taxa equivalente composta ($i_{\text{maior}} = (1 + i_{\text{menor}})^{k} - 1$). Use-a para converter 1,8% ao mês em taxa equivalente ao ano ($k = 12$) e compare o resultado com a taxa proporcional simples ($1{,}8\ \% \times 12$), explicando a diferença.

Exercício 4 — Nominal para efetiva com capitalizações diferentes
Escreva `nominal_para_efetiva(i_nominal_aa, k)` implementando $i_{\text{ef}} = (1 + i_{\text{nom}}/k)^{k} - 1$. Gere uma tabela comparando a taxa efetiva anual resultante de uma taxa nominal de 24% a.a. capitalizada: anualmente ($k = 1$), semestralmente ($k = 2$), trimestralmente ($k = 4$), mensalmente ($k = 12$) e diariamente ($k = 252$, dias úteis). O que acontece com a taxa efetiva à medida que $k \to \infty$?

Exercício 5 — Simulação com aportes mensais
Escreva `simular_aportes(aporte_mensal, i, n)` que simula uma aplicação em que, além do rendimento composto, o investidor deposita um novo aporte fixo ao final de cada mês (sem capital inicial). Mostre o saldo acumulado mês a mês para 24 meses, com `aporte_mensal=500` e `i=0.008`, e verifique que o saldo final bate com o valor futuro de uma série uniforme postecipada calculado pela fórmula do Capítulo 4.

Exercício 6 — Prestação de série postecipada
Reaproveite `serie_postecipada` do Capítulo 4. Um equipamento de R$ 45.000,00 será financiado em 18 parcelas mensais postecipadas a 2,2% a.m. Calcule a prestação, o total pago e os juros totais.

Exercício 7 — Antecipada vs. postecipada
Reaproveite `serie_antecipada` e `serie_postecipada` do Capítulo 4. Para o mesmo financiamento do Exercício 6 (R$ 45.000,00, 18 parcelas, 2,2% a.m.), calcule a prestação em série antecipada (com entrada) e compare com a postecipada. Calcule também a economia mensal e a economia total ao longo do contrato.

Exercício 8 — Série diferida com carência
Reaproveite `serie_diferida` do Capítulo 4. Um financiamento de R$ 60.000,00 em 24 parcelas mensais a 1,9% a.m. tem carência de 4 meses antes do início dos pagamentos. Calcule a prestação com carência e compare com a prestação sem carência, explicando de onde vem a diferença (dica: os juros da carência não desaparecem — eles são incorporados a quê?).

Exercício 9 — Perpetuidade simples
Reaproveite `perpetuidade` do Capítulo 4. Uma pessoa quer se aposentar recebendo R$ 8.000,00 por mês, para sempre, com uma aplicação que rende 0,7% a.m. Calcule o capital necessário. Em seguida, calcule quanto essa mesma pessoa poderia sacar por mês se juntasse R$ 1.500.000,00 a uma taxa de 0,55% a.m.

Exercício 10 — Perpetuidade com crescimento (Gordon)
Reaproveite `perpetuidade_crescimento` do Capítulo 4. Uma ação pagará dividendo de R$ 3,50 no próximo ano, com crescimento esperado de 5% a.a., e o investidor exige retorno (Ke) de 12% a.a. Calcule o preço justo pelo modelo de Gordon. Em seguida, dado que a ação está sendo negociada a R$ 42,00 com o mesmo dividendo esperado, calcule a taxa de crescimento implícita no preço de mercado.

Exercício 11 — Tabela Price completa
Implemente `tabela_price(pv, i, n)` do zero (sem colar o código do Capítulo 5 — escreva por conta própria) e gere a tabela completa de amortização para um financiamento de R$ 36.000,00 em 15 parcelas a 2,3% a.m. Imprima a tabela formatada com período, prestação, juros, amortização e saldo devedor, e confirme que o saldo devedor do último período é (aproximadamente) zero.

Exercício 12 — Tabela SAC completa
Implemente `tabela_sac(pv, i, n)` do zero, usando os mesmos dados do Exercício 11 (R$ 36.000,00, 15 parcelas, 2,3% a.m.). Imprima a tabela completa e identifique a amortização constante, a primeira e a última prestação.

Exercício 13 — Comparação Price x SAC
Reaproveite as duas tabelas dos Exercícios 11 e 12. Escreva uma função `comparar_sistemas(pv, i, n)` que imprime lado a lado: prestação inicial, prestação final, total pago e juros totais de cada sistema, e conclui qual sistema é mais barato no total e qual tem prestação inicial menor.

Exercício 14 — Sistema de Amortização Americano
Implemente `tabela_americano(pv, i, n)` seguindo as regras do Capítulo 5 (juros constantes pagos a cada período, principal quitado integralmente na última parcela). Use R$ 50.000,00, 10 períodos, 1,5% a.m., e imprima a tabela completa, destacando que a última prestação é muito maior que as demais.

Exercício 15 — SAM (Sistema de Amortização Mista)
Reaproveite `tabela_price` e `tabela_sac`. Implemente `tabela_sam(pv, i, n)` que calcula, para cada período, a prestação como a **média aritmética** entre a prestação Price e a prestação SAC daquele período (conforme a fórmula do Capítulo 5). Use os mesmos dados do Exercício 11 e mostre que a prestação inicial do SAM fica entre a do Price e a do SAC.

Exercício 16 — Custo Efetivo Total (CET)
Reaproveite `calcular_cet` do Capítulo 5. Um empréstimo de R$ 15.000,00 é pago em 10 prestações mensais de R$ 1.680,00 (esse valor já inclui tarifa de cadastro e seguro). Calcule o CET mensal e anual, e compare com a taxa de juros "pura" que seria necessária para gerar uma prestação de R$ 1.680,00 sobre R$ 15.000,00 em 10 meses (sem tarifas). Qual é o custo extra, em pontos percentuais, gerado pelas tarifas?

Exercício 17 — Equação de Fisher completa
Reaproveite `fisher` do Capítulo 6. Um CDB rendeu 11,50% a.a. nominal em um ano em que o IPCA fechou em 5,80%. Calcule a taxa real. Em seguida, calcule qual seria a taxa nominal necessária para se obter uma taxa real de 6% a.a., supondo uma inflação projetada de 4,5% a.a.

Exercício 18 — Correção monetária de uma série
Reaproveite `corrigir_por_indice` do Capítulo 6. Um contrato de aluguel de R$ 3.200,00 é reajustado mês a mês pela seguinte série de índices mensais (decimais): `[0.0061, 0.0055, 0.0048, 0.0070, 0.0052, 0.0044, 0.0058, 0.0039, 0.0061, 0.0050, 0.0047, 0.0053]`. Calcule o valor corrigido ao final de 12 meses e a inflação acumulada no período (em %).

Exercício 19 — Pré-fixado vs. pós-fixado
Reaproveite `comparar_prefixado_posfixado` do Capítulo 6. Um investidor tem R$ 20.000,00 para aplicar por 180 dias e compara: título pré-fixado a 11,8% a.a., ou título pós-fixado a 100% do CDI projetado em 12,4% a.a. (sem spread). Calcule o valor final de cada opção e diga qual é preferível no cenário informado.

Exercício 20 — VPL de um fluxo de caixa
Reaproveite `vpl` do Capítulo 7. Um projeto exige investimento de R$ 32.000,00 e promete fluxos de R$ 9.000,00 ao final de cada um dos próximos 5 anos. Calcule o VPL para TMA de 9% a.a. e para TMA de 15% a.a., e explique por que o VPL cai quando a TMA aumenta.

Exercício 21 — TIR implementada do zero
Implemente `tir(fluxos)` do zero (por bisseção ou Newton-Raphson, sem colar o código do Capítulo 7) e calcule a TIR do fluxo `[-32000, 9000, 9000, 9000, 9000, 9000]`. Confirme, substituindo a TIR encontrada de volta na fórmula do VPL, que o resultado é (aproximadamente) zero.

Exercício 22 — Payback simples e descontado
Reaproveite `payback` do Capítulo 7. Para o mesmo fluxo do Exercício 21, calcule o payback simples e o payback descontado a 9% a.a. Explique por que o payback descontado é sempre maior ou igual ao payback simples.

Exercício 23 — Índice de Lucratividade e WACC
Reaproveite `indice_lucratividade` do Capítulo 7. Calcule o IL do projeto do Exercício 20 (TMA = 9% a.a.) e diga se ele é viável pelo critério IL > 1. Em seguida, calcule o WACC de uma empresa com capital próprio de R$ 60 milhões (custo de 15% a.a.), dívida de R$ 30 milhões (custo bruto de 9% a.a., IR de 34%), e discuta se essa empresa deveria usar seu WACC como TMA para avaliar o projeto do Exercício 20 (ou se uma outra taxa seria mais apropriada).

Exercício 24 — Projeto integrador: relatório de financiamento
Combine várias funções deste capítulo em um único programa. Dado um financiamento de R$ 80.000,00 em 30 parcelas mensais a 1,7% a.m.: (1) gere as tabelas Price e SAC completas; (2) calcule o CET supondo que o banco cobra uma tarifa de cadastro de R$ 800,00 somada à primeira parcela; (3) calcule a taxa efetiva anual equivalente à taxa mensal contratada; (4) imprima um resumo comparando, ao final, qual sistema (Price ou SAC) esse cliente deveria escolher se seu objetivo é ter a **menor prestação possível nos primeiros 12 meses**, e qual deveria escolher se o objetivo é **pagar o menor total de juros**.

Capítulo 12 — Aplicações no Mercado Brasileiro

Este capítulo reúne situações do dia a dia financeiro, resolvidas com as ferramentas dos capítulos anteriores. A ideia é treinar o olhar: reconhecer em um anúncio, em um extrato ou em uma proposta qual é o **fluxo de caixa** escondido por trás e qual é a **taxa efetiva** da operação.

12.1. "Sem juros" é sempre sem juros?

O parcelamento "sem juros" é um dos produtos de crédito mais comuns no varejo. Há dois fatos a considerar. Primeiro: se a loja oferece **desconto para pagamento à vista**, o parcelamento esconde uma taxa de juros.

Segundo: mesmo quando o preço é idêntico, o parcelamento tem valor para o comprador, porque o dinheiro *fica aplicado* durante o prazo.

> **Exemplo resolvido — o desconto à vista revela a taxa**
> Um produto custa R$ 1.200 em 3 parcelas "sem juros" de R$ 400, ou R$ 1.140 à vista (5% de desconto). Qual a taxa de juros implícita?
>
> O fluxo de quem parcela, comparado ao pagamento à vista, é: recebe R$ 1.140 em valor de hoje (o que deixou de pagar) e paga 3 parcelas de R$ 400. A taxa que iguala $1.140 = 400 \cdot a_{3|i}$ é
>
> **2,61% a.m.** (36,2% a.a.). Parcelar nesse caso **é caro**: seria melhor pagar à vista, a menos que o dinheiro rendesse mais de 2,61% a.m.
>
> Já se o preço à vista for igual (R$ 1.200), a taxa implícita é 0% e o parcelamento é vantajoso para quem consegue aplicar o dinheiro: a 1% a.m., as parcelas valem hoje $400 \cdot a_{3|1\%} =$ R$ 1.176,39, ou seja, uma economia de R$ 23,61.

```
a_vista, parcela_loja, n_loja = 1140, 400, 3
taxa_implicita_loja = bisseccao(lambda r: vp_das_parcelas(r, parcela_loja, n_loja) -
a_vista, 1e-9, 1)
print(f"Taxa implícita: {taxa_implicita_loja*100:.2f}% a.m. = {((1 + taxa_implicita_loja)
** 12 - 1)*100:.1f}% a.a.")
print(f"Preço igual: VP das 3 parcelas a 1% a.m. = R${vp_das_parcelas(0.01, 400, 3):,.2f}
(vs. R$1.200 à vista)")
```

```
Taxa implícita: 2.61% a.m. = 36.2% a.a.
Preço igual: VP das 3 parcelas a 1% a.m. = R$1,176.39 (vs. R$1.200 à vista)
```

12.2. CDB, LCI e imposto de renda regressivo

Os rendimentos de renda fixa tributados têm Imposto de Renda **regressivo no tempo**: quanto mais tempo o dinheiro fica aplicado, menor a alíquota sobre o *rendimento* (não sobre o principal).

Prazo da aplicação	Alíquota de IR
até 180 dias	22,5%
de 181 a 360 dias	20,0%
de 361 a 720 dias	17,5%
acima de 720 dias	15,0%

Papéis como **LCI** e **LCA** (e, para pessoa física, debêntures incentivadas) são **isentos** de IR, e por isso pagam taxas brutas menores. Para compará-los com um CDB, calcula-se a **taxa bruta equivalente**: $i_{\text{bruta}} = \frac{i_{\text{líq}}}{1 - \text{IR}}$.

> **Exemplo resolvido — qual rende mais em um ano?**
> Com CDI de 13,25% a.a. e aplicação por 365 dias (alíquota 17,5%): um CDB a 100% do CDI rende 13,25% brutos e $13{,}25\% \times (1 - 0{,}175) \approx 10{,}93\%$ líquidos. Uma LCI a 90% do CDI rende **cerca de 11,85% líquidos**, isentos. Portanto, a LCI é melhor. Qual percentual do CDI um CDB precisaria pagar para igualar a LCI de 90%? $\frac{0{,}90}{1 - 0{,}175} \approx 1{,}09$, ou seja, cerca de **109% do CDI**.

```
def aliquota_ir(dias):
    if dias <= 180: return 0.225
    if dias <= 360: return 0.20
    if dias <= 720: return 0.175
    return 0.15

def rendimento_renda_fixa(valor, pct_cdi, cdi_aa, dias_corridos, isento=False):
```

```
    dias_uteis = round(dias_corridos * 252 / 365)
    cdi_dia = (1 + cdi_aa) ** (1 / 252) - 1
    bruto = valor * ((1 + cdi_dia * pct_cdi) ** dias_uteis - 1)     # % do CDI incide sobre
a taxa diária
    ir = 0 if isento else bruto * aliquota_ir(dias_corridos)
    return bruto, ir, bruto - ir

for nome_t, pct_t, iso_t in [("CDB 100% CDI", 1.00, False), ("CDB 110% CDI", 1.10, False),
("LCI 90% CDI (isenta)", 0.90, True)]:
    b_t, ir_t, liq_t = rendimento_renda_fixa(10000, pct_t, 0.1325, 365, iso_t)
    print(f"{nome_t:<22} bruto R${b_t:8.2f} | IR R${ir_t:7.2f} | líquido R${liq_t:8.2f}
({liq_t / 10000:.2%})")
```

```
CDB 100% CDI           bruto R$ 1325.00 | IR R$ 231.87 | líquido R$ 1093.12 (10.93%)
CDB 110% CDI           bruto R$ 1466.76 | IR R$ 256.68 | líquido R$ 1210.07 (12.10%)
LCI 90% CDI (isenta)   bruto R$ 1184.99 | IR R$   0.00 | líquido R$ 1184.99 (11.85%)
```

No mercado brasileiro
Poupança. A caderneta rende, mensalmente, 0,5% mais a Taxa Referencial (TR) quando a meta Selic é superior a 8,5% a.a.; quando a Selic é igual ou inferior a 8,5%, rende 70% da Selic mais a TR. É isenta de IR para pessoas físicas. Como a regra pode ser alterada por lei, confirme sempre o texto vigente.

12.3. O custo do rotativo do cartão de crédito

Juros mensais altos, compostos, produzem números impressionantes. Uma dívida de R$ 1.000 a 12% a.m. (taxa comum em linhas de crédito rotativo e cheque especial no passado recente) cresce, sem pagamentos, para $1.000 \times 1{,}12^{6} = \text{R\$ } 1.973{,}82$ em 6 meses e para $1.000 \times 1{,}12^{12} = \text{R\$ } 3.895{,}98$ em um ano — um acréscimo de **289,6% ao ano**. Lembre-se do Capítulo 3: a taxa anual **não** é 12 × 12% = 144%, e sim $1{,}12^{12} - 1$.

Atenção — erro comum
A legislação mais recente passou a limitar o custo total do rotativo, e a regulamentação do cheque especial também tem limites. Esses tetos mudam com o tempo: consulte a regra vigente. Mas a **matemática** do composto, mostrada aqui, não muda — é por ela que as dívidas de juros altos crescem tão depressa.

12.4. Financiamento imobiliário: Price × SAC em 30 anos

Exemplo resolvido — financiamento de 300 mil em 360 meses a 10% a.a.
A taxa mensal equivalente é $1{,}10^{1/12} - 1 = 0{,}7974\%$ a.m. (e não $10\frac{\%}{12} = 0{,}8333\%$). Sem considerar TR, seguros e tarifas:

- **SAC:** amortização de R$ 833,33 por mês; 1ª prestação $833{,}33 + 300.000 \times 0{,}007974 = \text{R\$ } 3.225{,}58$; última, R$ 839,98.
- **Price:** prestação fixa de R$ 2.537,67.

```
imob_pv, imob_aa, imob_n = 300000, 0.10, 360
imob_i = (1 + imob_aa) ** (1 / 12) - 1
tab_p, juros_p = tabela_price(imob_pv, imob_i, imob_n)
tab_s, juros_s = tabela_sac(imob_pv, imob_i, imob_n)
print(f"Taxa mensal equivalente: {imob_i*100:.4f}% a.m.")
print(f"Price: prestação R${tab_p[0]['prestacao']:,.2f} | juros totais R${juros_p:,.2f}")
print(f"SAC  : 1ª R${tab_s[0]['prestacao']:,.2f} -> última R${tab_s[-1]['prestacao']:,.2f}
| juros totais R${juros_s:,.2f}")
print(f"Diferença de juros (Price - SAC): R${juros_p - juros_s:,.2f}")
print(f"Renda mínima (30%): Price R${tab_p[0]['prestacao']/0.3:,.2f} | SAC R${tab_s[0]
['prestacao']/0.3:,.2f}")
```

```
Taxa mensal equivalente: 0.7974% a.m.
Price: prestação R$2,537.67 | juros totais R$613,562.09
SAC  : 1ª R$3,225.58 -> última R$839.98 | juros totais R$431,799.70
Diferença de juros (Price - SAC): R$181,762.39
Renda mínima (30%): Price R$8,458.90 | SAC R$10,751.93
```

Em prazos longos, a diferença de juros totais entre Price e SAC é enorme (mais de R$ 180 mil neste exemplo), mas o SAC exige uma renda inicial maior. Quem escolhe o Price, em geral, paga o preço da entrada mais fácil ao longo de décadas.

12.5. Planejamento de aposentadoria: acumular e depois consumir

O planejamento de aposentadoria combina duas séries: a **fase de acumulação** (valor futuro de aportes) e a **fase de consumo** (valor presente de saques, ou perpetuidade). Trabalhar em **termos reais** (descontando a inflação) evita erros: usa-se uma taxa **real** de retorno e valores em reais de hoje.

Exemplo resolvido — de quanto poupar por mês?
Meta: viver de uma renda de R$ 5.000 por mês (em reais de hoje), para sempre, com retorno real de 0,5% a.m. (cerca de 6,2% a.a. reais).

1. **Capital necessário** (perpetuidade): $\frac{5.000}{0{,}005} = \text{R\$ } 1.000.000$.
2. **Aporte para acumulá-lo em 30 anos** (360 meses), à mesma taxa real: $\text{PMT} = \text{FV} \cdot \frac{i}{(1+i)^n - 1} = 1.000.000 \times \frac{0{,}005}{1{,}005^{360} - 1} = \textbf{R\$ 995,51 por mês}$.

O resultado surpreende: cerca de 64% do patrimônio final vem de *rendimentos* (os aportes somam R$ 358 mil dos R$ 1 milhão). Atrasar o início por 10 anos (acumular em 20 anos) mais que dobra o aporte necessário — é o efeito do tempo sobre o composto.

```python
renda_alvo, taxa_real_m = 5000, 0.005
capital_alvo = renda_alvo / taxa_real_m
for anos_ac in (30, 20, 10):
    n_m = anos_ac * 12
    aporte_ac = capital_alvo * taxa_real_m / ((1 + taxa_real_m) ** n_m - 1)
    print(f"Acumular em {anos_ac} anos: aporte de R${aporte_ac:,.2f}/mês "
          f"(total aportado R${aporte_ac * n_m:,.0f} = {aporte_ac * n_m / capital_alvo:.0%}
do capital)")
```

```
Acumular em 30 anos: aporte de R$995.51/mês (total aportado R$358,382 = 36% do capital)
Acumular em 20 anos: aporte de R$2,164.31/mês (total aportado R$519,435 = 52% do capital)
Acumular em 10 anos: aporte de R$6,102.05/mês (total aportado R$732,246 = 73% do capital)
```

Capítulo 13 — Erros Comuns e Teste Rápido

13.1. Os dez erros que mais reprovam

#	Erro	Como evitar
1	Taxa mensal com prazo em anos (ou o contrário)	Escreva a unidade ao lado de cada número; padronize antes de calcular
2	Dividir a taxa anual por 12 em juros compostos	Use a taxa **equivalente**: $(1 + i_a)^{1/12} - 1$
3	Somar inflações mensais em vez de multiplicar fatores	Acumule com $\prod(1 + \pi_t)$
4	Subtrair inflação da taxa nominal em taxas altas	Use Fisher: $\frac{1+i_n}{1+\pi} - 1$

#	Erro	Como evitar
5	Somar valores de datas diferentes	Leve tudo para a **mesma data focal**
6	Confundir "com entrada" com série antecipada	Antecipada só se a entrada for igual às demais parcelas
7	Trocar o primeiro pagamento: série postecipada × antecipada	O VP da postecipada fica **um período antes** do 1º pagamento
8	Usar a taxa de desconto comercial como se fosse taxa de juros	Converta: $i = \frac{d}{1-dn}$
9	Escolher projeto pela maior TIR (excludentes)	Decida pelo **maior VPL** à TMA
10	Arredondar taxas intermediárias	Guarde todas as casas até o resultado final

13.2. Teste rápido: verdadeiro ou falso (com gabarito)

1. **V ou F:** 2% a.m. em juros compostos equivalem a 24% a.a. **Falso.** Equivalem a $1{,}02^{12} - 1 = 26{,}82\%$ a.a.; 24% é a taxa proporcional (nominal).
2. **V ou F:** No desconto comercial, a taxa efetiva de juros é maior que a taxa de desconto. **Verdadeiro.** $i = \frac{d}{1-dn} > d$.
3. **V ou F:** Em um financiamento SAC, a 1ª prestação é menor que no Price. **Falso.** A 1ª prestação do SAC é **maior** é a última que é menor.
4. **V ou F:** O SAC e o Price têm o mesmo custo efetivo (taxa i) quando a taxa contratada é a mesma. **Verdadeiro.** Os valores presentes das prestações, descontados a i, são iguais ao valor financiado.
5. **V ou F:** Quanto maior a TMA, maior o VPL. **Falso.** O VPL é **decrescente** em relação à taxa de desconto (fluxos positivos futuros).
6. **V ou F:** Um projeto com TIR de 30% é sempre melhor que outro com TIR de 18%. **Falso.** Para projetos excludentes, a escala e o prazo importam; decide-se pelo VPL.
7. **V ou F:** Se a inflação é 6% e a taxa nominal 6%, o ganho real é zero. **Verdadeiro.** $\frac{1{,}06}{1{,}06} - 1 = 0$.
8. **V ou F:** Em perpetuidade com crescimento, o valor presente existe para qualquer g. **Falso.** Exige $g < i$.
9. **V ou F:** Pagamentos de aluguel feitos no início do mês formam uma série postecipada. **Falso.** São **antecipados**.
10. **V ou F:** A média aritmética das rentabilidades mensais é a rentabilidade mensal média correta. **Falso.** Use a média **geométrica**, pois rentabilidades se compõem.

Capítulo 14 — Glossário

Termos em ordem alfabética. Os números entre parênteses remetem ao capítulo onde o conceito é desenvolvido.

Termo	Definição
Amortização (5)	Parcela da prestação que abate o saldo devedor (o principal).
Anatocismo (1)	Cobrança de juros sobre juros, isto é, capitalização composta.
Capitalização (1, 3)	Incorporação dos juros ao capital; define o regime (simples ou composto).
Carência (4)	Período inicial de um financiamento em que não há pagamento de principal (e, às vezes, nem de juros).
CDI (3, 6)	Taxa média dos empréstimos interbancários de um dia; referência de rentabilidade de renda fixa pós-fixada.
CET (5)	Custo Efetivo Total: taxa interna de retorno do fluxo líquido do cliente, incluindo tarifas, tributos e seguros.
Data focal (1, 2)	Data escolhida para comparar valores de datas diferentes.

Termo	Definição
Deflator (6)	Índice de preços usado para converter valores nominais em reais de uma data-base.
Desconto comercial (2)	Desconto calculado sobre o valor nominal (por fora): $D_c = \text{FV} \cdot d \cdot n$.
Desconto racional (2)	Desconto calculado sobre o valor atual (por dentro): $D_r = \frac{D_c}{1+in}$.
Dias úteis / base 252 (3, 6)	Convenção do mercado de renda fixa brasileiro: taxas anuais referem-se a 252 dias úteis.
Equivalência de taxas (3)	Duas taxas são equivalentes quando produzem o mesmo montante no mesmo prazo total.
Fator de capitalização (3)	$(1+i)^n$: multiplica um valor presente para obtê-lo no futuro.
Fluxo de caixa (1)	Representação das entradas e saídas de dinheiro no tempo.
FRC (4)	Fator de Recuperação de Capital: $\frac{1}{a_{n\|i}}$, usado para calcular prestações.
Fisher (equação de) (6)	$(1+i_n) = (1+i_r)(1+\pi)$: relação entre taxa nominal, real e inflação.
Fisher (taxa de) (7)	Taxa em que os VPLs de dois projetos se igualam.
Gordon (modelo de) (4)	Valor presente de uma perpetuidade com crescimento: $P_0 = \frac{D_1}{i-g}$.
IL (7)	Índice de Lucratividade: $1 + \frac{\text{VPL}}{\|\text{FC}_0\|}$.
Inflação implícita (6)	Inflação que iguala o retorno de um título pré-fixado e o de um IPCA+.
IOF (5, 12)	Imposto sobre Operações Financeiras; incide sobre crédito e entra no CET.
IPCA / IGP-M (6)	Índices de preços: IPCA é o índice oficial (IBGE); IGP-M é da FGV, usado em aluguéis e contratos.
Juros compostos (3)	Juros calculados sobre o montante do período anterior; crescimento exponencial.
Juros simples (2)	Juros calculados sempre sobre o capital inicial; crescimento linear.
LCI / LCA (12)	Letras de crédito imobiliário e do agronegócio; renda fixa isenta de IR para pessoa física.
Marcação a mercado (6)	Reavaliação do preço de um título pelo valor de mercado atual (taxa vigente).
Montante (1)	Capital mais juros acumulados (valor futuro).
Payback (7)	Tempo necessário para recuperar o investimento (simples ou descontado).
Perpetuidade (4)	Série infinita de pagamentos iguais; $\text{PV} = \frac{\text{PMT}}{i}$.
Pós-fixado (6)	Título cuja remuneração depende de um indexador (CDI, Selic, IPCA) conhecido só no futuro.
Postecipada / antecipada / diferida (4)	Séries cujo primeiro pagamento ocorre ao fim do 1º período, no início ou após carência, respectivamente.
Pré-fixado (6)	Título cuja taxa é conhecida no momento da aplicação.
Prazo médio (2)	Média ponderada dos prazos de vários títulos, pelos valores nominais.
Price (5)	Sistema de amortização com prestações constantes (Sistema Francês).
Regra dos 72 (3)	Aproximação do tempo de duplicação: $\frac{72}{i}\%$.
Racionamento de capital (7)	Situação em que o orçamento limita os projetos que podem ser executados.
SAC (5)	Sistema de Amortização Constante: amortização fixa, prestações decrescentes.
SAM (5)	Sistema de Amortização Mista: média aritmética de Price e SAC.
Saldo devedor (5)	Valor presente das prestações que ainda faltam; dívida remanescente.
Selic (3, 6)	Taxa básica de juros da economia, definida pelo Copom.
Spread (6)	Margem somada ou multiplicada a um indexador (por exemplo, CDI + 0,5%).
Taxa efetiva (3)	Taxa que realmente incide no período considerado.
Taxa nominal (3)	Taxa declarada cujo período de capitalização difere do período a que se refere.
Taxa over (3)	Taxa diária multiplicada por 30, convenção de operações compromissadas.

Termo	Definição
Taxa proporcional (2, 3)	Taxas na mesma razão que seus períodos; equivalentes só em juros simples.
Taxa real (6)	Taxa de retorno descontada a inflação (ganho em poder de compra).
TIR (7)	Taxa Interna de Retorno: taxa que zera o VPL.
TMA (7)	Taxa Mínima de Atratividade: custo de oportunidade do capital.
VPL (7)	Valor Presente Líquido: soma dos fluxos descontados a valor presente.
VPLA (7)	VPL Anualizado: série uniforme equivalente ao VPL, para comparar prazos diferentes.
WACC (7)	Custo Médio Ponderado de Capital.